Pauta

UI, UX and platform rules for adl's JavaFX desktop apps. Editora is the reference implementation; every rule names the class or file to copy from.

Starting a new app? Use adl-desktop-app-template, a GitHub template repository built to the edition its README names: click **Use this template**, run `scripts/rename.sh <AppName> <package>`, then add any rule from a later edition.

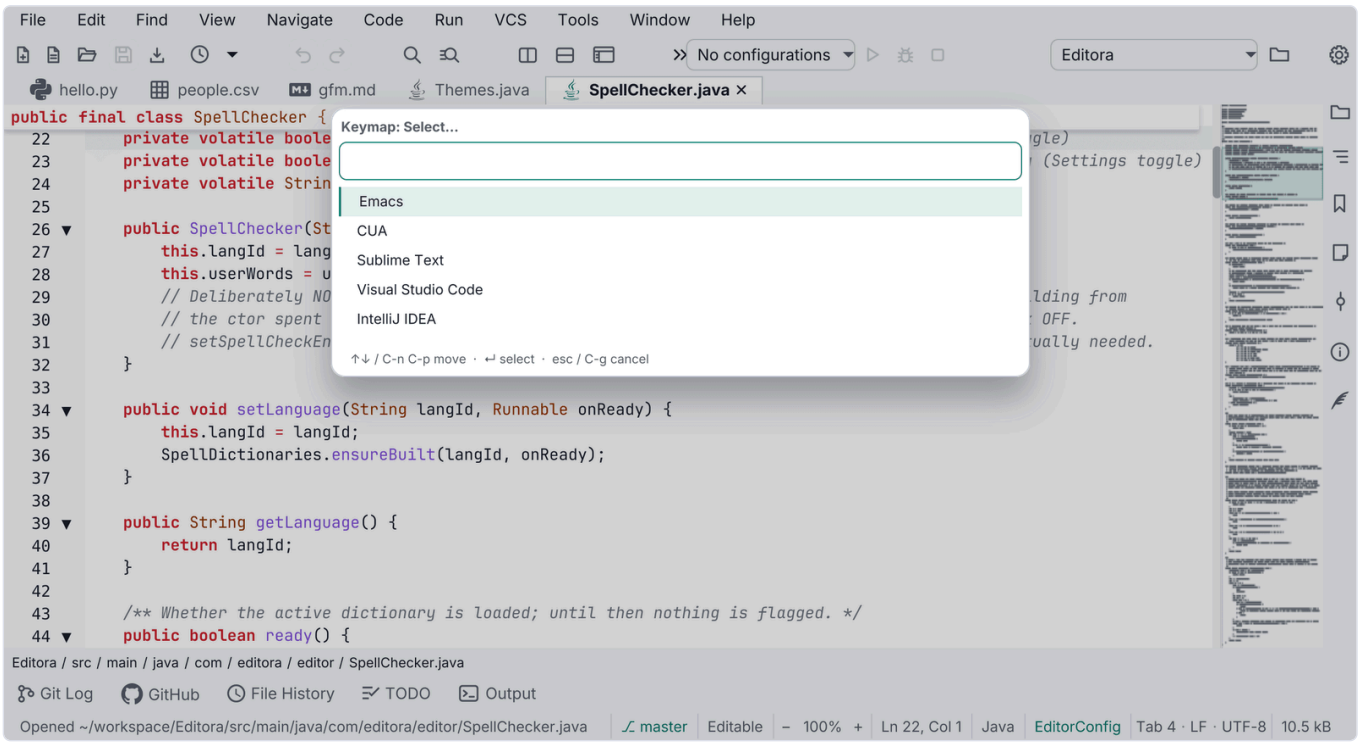
Editora · reference | Torrenta | Termina | Modula | Mirada | nux

`MUST` release blocker for a new app `SHOULD` expected unless the app says why not `MAY` copy when the app grows into it

CONTENTS

Principles
New app checklist
Conformance
Coding agents and LLMs
Platform baseline
Repository documents
Documentation
Code quality gates
Performance
Keyboard first
Window shell
Icons and color
Toolbars and tooltips
Tabs and tool panels
Menus
Tray and background
Settings
Diagnostics and support
Command line and launch
Internationalization
Command Palette
QuickOpen picker
Welcome page
About window
Pauta window
Confirm dialog
Find bar
Message Log
Settings Sync
Packaging, signing and release
Feature matrix
Shared features
Gaps per app

Principles



Editora's keymap picker: five keymaps bind keys to the same commands.

These come from what the six apps already do. The first is the reason the family and this guide exist: one app that is the same on Windows, macOS and Linux. The other twelve are how the apps get there and stay there. Editora is the reference; each principle names where else it is written down.

For the people using the apps

1. **Same app on every OS.** A user who moves between Windows, macOS and Linux finds the same features, the same commands and keymaps, the same look and the same installer quality, and never relearns the app. Platform differences live behind one class, never scattered `if (isMac)` checks. Every other principle serves this one.
2. **Keyboard first.** Every action works without a mouse. The mouse is a shortcut to commands, not the other way round.
3. **Every feature is a command.** One registry of commands with ids. Menus, toolbar buttons, the command palette and key bindings all run commands by id. Nothing is wired straight to logic.
4. **Speed comes first.** Start-up and the app's core action (typing in Editora, stepping to the next image in Mirada, a keystroke reaching the shell in Termina) are the product. When a feature pulls against them, the feature loses or waits until it can be made cheap. *(Mirada's "Priorities"; "fast first" in Termina and Mirada.)*
5. **The UI never waits.** The JavaFX application thread only touches the scene graph. Disk, network, processes, parsing and searching run on other threads and hand their result back; a keystroke, a scroll or a window opening never waits for them. See Performance.
6. **Changes apply as they are made.** Settings have no Save button. A change reaches every open window.
7. **Nothing fails silently.** Every failure reaches the status bar, the Message Log or the Debug Log, and the user can turn any of it into a GitHub issue.
8. **Never claim what is not true.** No link that looks like a link and does nothing, no "up to date" before an update check has answered, no setting that seems to apply when it cannot: a setting that waits for a restart says so on its own row. *(Modula's About, Editora's update status, Termina's settings rows.)*
9. **Local first, nothing sent without asking.** No telemetry. The only automatic network call is a once-a-day release check the user can turn off. Problem reports are built locally and submitted by the user.
10. **Users' data outlives versions.** Config files carry a schema version and migrate forward. An older build never overwrites a newer config.

For the people building them

11. **One source of truth for each fact.** The version lives in `pom.xml` and reaches About, `--version`, the update check and problem reports through a filtered `build-info.properties`. A key binding is declared once, so the menu and the accelerator cannot disagree. A list that repeats a fact (About's dependencies, the coverage table in the docs) has a test that fails when the two drift, in both directions. *(Modula's `DependenciesTest`, nux and Torrenta's `AppInfo`, Editora's `BuildHygieneTest`.)*
12. **Separate the decision from the UI, then test it.** Anything with a rule in it (which key does what, sort order, when a bar shows, whether a color is legible) is a small pure method or class with its own unit tests. UI classes stay thin and only wire decisions to controls. *(Termina, Torrenta, Mirada and nux "Conventions"; Editora's pure decision helpers.)* See Code quality gates.
13. **Verify by driving the real app.** UI bugs are visual and none of them throw. Before calling a UI change done, run the real window, press the real keys and look at the result, through a capture hook or a headless screenshot; reasoning about event order is not evidence. *(Termina and Torrenta's `DevCapture`, nux's `--screenshot`, the headless harness behind Editora's website screenshots.)*

New app checklist

Each group links to the page with the rules and the Editora code to copy. Everything here is a `MUST` unless it says `SHOULD`.

Start from the template. adl-desktop-app-template is a GitHub template repository: a small runnable app with its tests, CI and release workflows and every repository document, built to the edition its README names. Click **Use this template**, run `scripts/rename.sh <AppName> <package>`, then add any rule on this list from a later edition.

Conformance

Conformance · Pauta window

- [ ] `pauta.version` in `pom.xml` names the edition; About shows it and its conformance, *Built to Pauta 4.1.0* and *Pauta conformance: Core*
- [ ] Every rule the app does not meet is an exception in `pauta.json`: not applicable, a deviation (SHOULDs only) with a decision record, or pending with an issue
- [ ] The conformance (Complete, Core or Partial) is computed from the exceptions by one tested helper, never stored
- [ ] `PautaConformanceTest` checks `pauta.json` and the README against the edition's `rules.json`
- [ ] A Pauta window, `help.pauta`, in the palette, the Help menu and About's link, listing every exception
- [ ] A *Pauta conformance* page in the user docs with the same list, generated from `pauta.json` (generated: `SHOULD`)

Platform

Platform baseline

- [ ] JDK 25 or newer (`release` 25 is the floor) and JavaFX 27 or newer
- [ ] One JPMS module, `com.<app>`
- [ ] Maven Wrapper committed (`mvnw`, `mvnw.cmd`, `.mvn/wrapper/`)
- [ ] AtlantaFX light and dark themes, following the system by default
- [ ] Inter bundled and attached to every scene and dialog `SHOULD`

Code quality

Code quality gates

- [ ] The version comes only from `pom.xml` (filtered `build-info.properties`); lists that repeat a fact have a drift test
- [ ] Rules live in pure, unit-tested helpers; UI classes stay thin
- [ ] A capture hook or headless screenshot drives the real window for UI checks
- [ ] Every package `@NullMarked`; NullAway on Error Prone fails the build
- [ ] Spotless with Palantir Java Format
- [ ] JUnit 5 and headless FX tests (`-Dglass.platform=Headless`), FX tests tagged `fx`
- [ ] JaCoCo check at 0.90 lines and 0.90 methods

Performance

Performance

- [ ] No I/O, process, parsing or waiting on the JavaFX application thread; results return through `Platform.runLater`
- [ ] Background results applied under a generation guard; named daemon threads
- [ ] Redraws coalesced to one per pulse; streams delivered in batches
- [ ] Logs, histories and caches bounded; large inputs guarded
- [ ] Only the first frame built before it paints; update check and extras deferred

Keyboard and commands

Keyboard first

- [ ] One `CommandRegistry`: every action is a command with an id
- [ ] Command Palette as an in-scene overlay, not a `Popup`
- [ ] Emacs and CUA keymaps at least, with user overrides on top
- [ ] One QuickOpen-style picker for every list choice (themes, keymaps, recent items, a setting's values), in the overlay host `SHOULD`
- [ ] A find bar in the scene wherever there is text to search: searches as you type with a count, Enter and Shift+Enter step, Escape closes it and returns focus (`FindReplaceBar`, Termina's `SearchBar`)

Window shell

Window shell · Welcome page · Confirm dialog

- [ ] A Welcome page when there is nothing to restore, or an empty state that names the ways in with their chords `SHOULD`
- [ ] Every destructive action asks first through one helper: the question names the thing, the button names the action in the `danger` style, Escape cancels; or the action is undoable instead

Icons and color

Icons and color

- [ ] Inter attached to every scene by a scene stylesheet; state colors named once on `.root` from the theme's roles, no fixed hex in a component rule
- [ ] Readable text in `-color-fg-muted` or stronger, `-color-fg-emphasis` on an emphasis fill; a pure contrast test over the stylesheets `SHOULD`
- [ ] Icons as themed SVG paths in one `Icons` class, one style class per family, the same glyph for an action everywhere it is reached

Toolbars and tooltips

Toolbars and tooltips

- [ ] Icon-only buttons built by one helper that sets the tooltip and the accessible name (`Icons.button` , `Icons.name`)
- [ ] Toolbar buttons out of the Tab order; every tooltip names the command and its chord from the active keymap
- [ ] Groups separated by spacing on one chrome band, no hairlines; a toggle wears `:open` while it is on `SHOULD`

Tabs and panels

Tabs and tool panels

- [ ] One tab header carries the states: `•` and amber italic for unsaved, a pin and bold for pinned, muted italic for read-only (`TabHeaders`)
- [ ] Bulk close skips pinned tabs, and Reopen Closed Tab brings the last one back with its caret (`ClosedTabs`)
- [ ] Panels toggled by commands, with side, order and divider positions restored on the next launch (`ToolWindowManager`) `SHOULD`

Menus

Menus

- [ ] Menu rows are commands: icon, title and chord from the registry and the active keymap, no `KeyCombination` accelerators
- [ ] Every row of every menu, right-click menus and drop-downs included, has an icon that depicts it and its chord when bound, in the 11 px periwinkle column
- [ ] Menus draw in Inter 14 px whatever they were opened from (`.context-menu` pins the font in pixels)
- [ ] Right-click menus close on a press anywhere outside them and never stack (`ContextMenuDismissal` installed at startup)
- [ ] Right-click menus open from the Menu key and Shift+F10 at the selected row (`RowContextMenu`)

Tray and background

Tray and background (for an app that keeps working without its window)

- [ ] The shared `tray/` package: StatusNotifierItem on Linux, AWT elsewhere, created off the FX thread, and the app fully usable without one
- [ ] Closing the window hides it only while an icon is up and the setting says so; Quit is explicit and one path
- [ ] Start at login writes the OS entry (XDG autostart, LaunchAgent, `HKCU` Run) with no setting beside it, `--minimized` behind the icon `SHOULD`
- [ ] Desktop notifications through one `Notifier` , off the FX thread, one switch per event

Settings

Settings

- [ ] Settings window with a page sidebar, search and live apply
- [ ] A palette command for every setting
- [ ] Versioned config in `~/.<app>` , with migrations and downgrade backups
- [ ] Export and import of the config folder
- [ ] Config read with suspicion: values clamped on load, an unreadable file kept as a backup and defaults loaded
- [ ] No secret in a config file: keys and tokens in the OS keychain, a credential the app can ask for again in memory only
- [ ] A secret in the wrong place refused (a sync URL with a password, an unredacted problem report), and an option that weakens security only a config key
- [ ] Settings sync through a Git/GitHub repository `SHOULD`

Diagnostics

Diagnostics and support

- [ ] Status-bar echo and Message Log
- [ ] Debug Log with a session file, and an uncaught exception handler
- [ ] Daily update check with an off switch; About shows versions, the Pauta edition and conformance, and update status
- [ ] Report a Problem (redacted, prefilled GitHub issue) and `--report`

Command line and languages

Command line · Internationalization

- [ ] `--help`, `--version`, `--config-dir`, `--dev`
- [ ] Single instance per config folder
- [ ] At least six message catalogs, with a parity test

Repository

Repository documents

- [ ] `README.md` with the badge row and a light/dark hero screenshot
- [ ] `CHANGELOG.md` in Keep a Changelog format
- [ ] MIT `LICENSE`, and `NOTICE` with every third-party license
- [ ] `AGENTS.md`, with a one-line `CLAUDE.md` pointing to it
- [ ] `CONTRIBUTING.md`, `docs/` and `branding/`
- [ ] `.github/ISSUE_TEMPLATE/bug_report.yml`

Documentation

Documentation

- [ ] User docs on the app's website under `/docs/`: Getting started, a page per feature area, the keyboard reference, Settings and an FAQ
- [ ] Help → Documentation as a command (`help.documentation`), first in the Help menu after the Command Palette, and on the Welcome page
- [ ] A one-line `command.<id>.desc` for every command, held by the parity test
- [ ] `docs/README.md` indexes the contributor guides; `AGENTS.md` and `CONTRIBUTING.md` point into it
- [ ] Docs change in the same PR as the code; when they disagree with the code, the code wins and the docs are fixed
- [ ] Deep links from the app open the running version's docs, `/docs/v—<version>/` `SHOULD`
- [ ] Command and keybinding references generated from the source, with a check that quoted ids and chords exist `SHOULD`

Packaging and signing

Packaging, signing and release

- [ ] jpackage installers through JReleaser: Linux x64 and arm64, macOS arm64 and x64, Windows x64
- [ ] macOS `.dmg` signed with Developer ID, notarized and stapled
- [ ] Linux downloads with `.asc` signatures from the shared release key
- [ ] Build provenance attestations for every download
- [ ] Windows `.msi` Authenticode, once the certificate is chosen `SHOULD`
- [ ] Homebrew cask in `adriandeleon/homebrew-tap` and a winget package `AdrianDeLeon.<App>` `SHOULD`

CI/CD (GitHub Actions)

CI/CD

- [ ] `ci.yml` on every push and pull request: Format and Build & Test, both required checks for `master`
- [ ] `full-tests.yml` on Linux, macOS and Windows: daily, on a `full-ci` label, before every release `SHOULD`
- [ ] `release.yml` on a `v*` tag that matches the pom, with a `workflow_dispatch` dry run
- [ ] Every third-party action pinned to a commit SHA
- [ ] Dependabot for `github-actions` and `maven`
- [ ] `permissions: contents: read` by default; write only in publishing jobs
- [ ] `timeout-minutes` on every job; superseded pull-request runs canceled
- [ ] Test and coverage reports uploaded; coverage totals in the job summary

- [ ] Signing skipped without secrets, but a tag without them fails
- [ ] A smoke launch of the packaged app on each OS before release

Conformance

An app implements one edition of Pauta, says how fully it conforms (Complete, Core or Partial), and shows both in its About window. The rules it does not meet are not hidden: each one is an **exception**, written down once in the app's `pauta.json`, shown in the app's Pauta window and listed in its user docs. Its conformance is worked out from that list, never typed in by hand, so the claim and the evidence can't drift apart.

No app does this yet. The audit in Coding agents and LLMs produces the first list: its `fail` and `n/a` lines are the exceptions.

Complete, Core and Partial

Conformance	The app meets...	About says
Complete	every `MUST` of its edition, met or not applicable, and every `SHOULD`, met, not applicable or a recorded deviation	Built to Pauta 4.1.0 · Pauta conformance: Complete
Core	every `MUST` in the same way; one or more SHOULDs are still pending	Built to Pauta 4.1.0 · Pauta conformance: Core
Partial	not every `MUST`: one or more MUSTs are still pending	Working toward Pauta 4.1.0 · Pauta conformance: Partial

"Built to an edition" keeps its meaning: the app meets that edition's MUSTs. Core and Complete are the two ways to be built to it; a Partial app is working toward it and says so. `MAY` rules never count. A deviation, which only a `SHOULD` can have, leaves the conformance as it is but is always on show: About gives the number of exceptions, and the Pauta window lists every one.

Partial is how an app that already ships says it is on its way: it took up an edition after its first release, or moved to a newer one with new MUSTs. A new app is not released at Partial.

- **PAUTA-01** `MUST` Every app implements Pauta: it names one edition (the `pauta.version` of ABOUT-08) and states its conformance, Complete, Core or Partial, as defined above.
  - *Why:* an app that is in the family answers to these rules; saying which ones, and how far, is the first of them.
  - *Check (code):* `pom.xml` defines `pauta.version` and `src/main/resources/com/<app>/pauta.json` exists.
  - *Reference:* no app yet.
- **PAUTA-02** `MUST` The conformance is computed, never stored: one pure helper reads the exceptions and returns Partial when any `MUST` is pending, Core when any `SHOULD` is pending, and Complete otherwise. About, the Pauta window and the docs page take it from there; the README, which can't call the helper, is held to it by a test (PAUTA-15).
  - *Why: One source of truth for each fact.* A conformance typed in by hand stays "Complete" after a pending exception is added.
  - *Check (test):* a unit test of the helper (`PautaConformance`, or the app's equivalent) covers all three and shows that deviations and not-applicable entries leave the result alone.
  - *Reference:* no app yet.
- **PAUTA-03** `MUST` A new app's first release is at Core or Complete: the New app checklist's MUSTs are release blockers, and recording one as pending does not lift that.
  - *Check (by hand):* the About window of the app's first release says Core or Complete.
  - *Reference:* no app yet.

Exceptions

An exception is one rule the app does not meet as written. There are three kinds, and each one asks for something different.

Kind	Means	Needs
`not-applicable`	The rule applies to every app, but its subject does not exist in this one: no tabs to reopen, no setting to sync	A reason
`deviation`	A `SHOULD` only: the app does it differently on purpose, and would make the same choice again	A reason and a decision record
`pending`	The app does not do it yet	A reason and an issue

- **PAUTA-04** `MUST` Every rule of the edition that the app does not meet has an exception, and every exception names a rule the app does not meet. An unmet rule with no entry is a failed audit, not an exception; an entry for a rule the app now meets is removed in the same change that meets it.
  - *Why: Never claim what is not true.* The list is the app's claim; it has to be whole and current.
  - *Check (by hand):* audit the app against its edition (Coding agents and LLMs); every `fail` and `n/a` line is in `pauta.json`, and no entry is for a rule that passes.
  - *Reference:* no app yet.
- **PAUTA-05** `MUST` A rule whose own text or page limits it to apps with a feature ("An app with user data worth carrying between computers...", the Tray and background page) needs no entry in an app without that feature. `not-applicable` is for a rule that reads as applying to every app.
  - *Check (code):* no `not-applicable` entry names a rule whose text or page already scopes it.
  - *Reference:* no app yet.
- **PAUTA-06** `MUST` A `deviation` is a choice, not a gap, and only a `SHOULD` can have one: a `SHOULD` is expected unless the app has a stated reason, and the deviation is that reason. It links a decision record in `docs/decisions/` (DOCS-13) that gives the context, what the app does instead, and what it costs. A `MUST` the app does not meet is `pending`, as is any work it has not done yet.

- *Why:* a deviation keeps the conformance; a deviation from a `MUST`, or missing work called a deviation, would let the app say it is built to an edition it does not meet.
- *Check (code):* every `deviation` entry names a `SHOULD`, and its `link` opens a file under `docs/decisions/` that names the rule's ID.
- *Reference:* no app yet.

- **PAUTA-07** `MUST` A `pending` exception links an open issue in the app's repository that names the rule's ID and edition, "MENU-14 (Pauta 4.1.0)". Closing the issue removes the entry.
  - *Check (code):* every `pending` entry's `link` is an issue URL in the app's repository.
  - *Reference:* no app yet.
- **PAUTA-08** `MUST` Each exception's `reason` is one plain sentence for the person using the app, in English like the changelog: what is different and, for a deviation, what they get instead. No rule text, no class names.
  - *Check (by hand):* read each reason in the Pauta window as a user would.
  - *Reference:* no app yet.

The file

The exceptions live in one file, bundled with the app so the Pauta window can read it: `src/main/resources/com/<app>/pauta.json`. The edition stays in `pom.xml` (ABOUT-08). Each entry copies the rule's `level` and `url` from the edition's `rules.json` so the app needs nothing else at run time, and a test keeps the copies honest.

```
{
  "exceptions": [
    {
      "rule": "MENU-14",
      "level": "MUST",
      "url": "https://pauta-project.dev/4.1.0/menus/#menu-14",
      "kind": "pending",
      "reason": "Some menu rows have no icon yet.",
      "link": "https://github.com/adriandeleon/<App>/issues/212"
    },
    {
      "rule": "TABS-06",
      "level": "MUST",
      "url": "https://pauta-project.dev/4.1.0/tabs-and-panels/#tabs-06",
      "kind": "not-applicable",
      "reason": "<App> has no tabs: it shows one library in one window."
    },
    {
      "rule": "WIN-06",
      "level": "SHOULD",
      "url": "https://pauta-project.dev/4.1.0/window-shell/#win-06",
      "kind": "deviation",
      "reason": "<App> opens straight to your library instead of a Welcome page; its empty state names the ways in.",
      "link": "https://github.com/adriandeleon/<App>/blob/master/docs/decisions/0007-no-welcome-page.md"
    }
  ]
}
```

- **PAUTA-09** `MUST` The exceptions have one source, `pauta.json` as above: `rule`, `level`, `url`, `kind` and `reason` on every entry, `link` on every deviation and pending entry, one entry per rule.
  - *Check (code):* the file exists at that path, and each entry has the fields its kind needs.
  - *Reference:* no app yet.
- **PAUTA-10** `MUST` A test reads `pauta.json` against a copy of the edition's `rules.json` in `src/test/resources/pauta-rules.json` and fails when the copy's `edition` is not `pauta.version`, when an entry names a rule the edition does not have, when its `level` or `url` differs from the edition's, when a rule appears twice, when a deviation names a `MUST`, or when a deviation or pending entry has no `link`.
  - *Why:* moving to a newer edition changes levels and retires IDs; the test turns each stale entry into a failure.
  - *Check (test):* `PautaConformanceTest`, or the app's equivalent, runs in the build.
  - *Reference:* no app yet.
- **PAUTA-11** `SHOULD` Moving to a newer edition is one change: replace `pauta-rules.json`, run the audit again, update `pauta.json`, then change `pauta.version`, the README and `AGENTS.md` together.
  - *Check (by hand):* the change that raises `pauta.version` also replaces `pauta-rules.json` and touches `pauta.json` where the audit found something new.
  - *Reference:* Move an app to a newer edition.

Where it shows

The same list reaches three readers: the person using the app, the person reading its docs, and whoever reviews the repository.

- **PAUTA-12** `MUST` In the app: About names the edition and the conformance (ABOUT-09), and the Pauta window, opened from About, the Help menu and the Command Palette, lists every exception (Pauta window).
  - *Check (code):* `help.pauta` is registered and About's *Pauta conformance* link runs it.
  - *Reference:* no app yet.

- **PAUTA-13** `MUST` In the user docs: the app's website has a *Pauta conformance* page under `/docs/` (DOCS-02) with the edition, the conformance and every exception, grouped by kind, each with its rule ID linked to the edition's rule, its reason and its link. The Pauta window links to it.
  - *Check (code):* the website's `src/content/docs/` has the page, and it lists the same rules as `pauta.json`.
  - *Reference:* no app yet.
- **PAUTA-14** `SHOULD` The docs page is generated from `pauta.json`, like the Commands page (DOCS-04), so it cannot list a different set from the app.
  - *Check (code):* the website's `scripts/` has a generator that reads the app's `pauta.json`, and its output is committed.
  - *Reference:* no app yet; editora-website's `scripts/GenCommands.java` is the pattern.
- **PAUTA-15** `MUST` In the repository: the README's Pauta badge (README badges) and its "Built to" line (at Partial, "Working toward") name the edition and the conformance, and both link to the docs page.
  - *Check (test):* `PautaConformanceTest` fails when the README's badge or line names another edition than `pauta.version` or another conformance than `PautaConformance` returns.
  - *Reference:* no app yet; the app template's README names its edition.

Coding agents and LLMs

Pauta has two readers: the developer, and the coding agent working beside them. The pages are written for the developer. Every edition also serves the same rules as plain text and as JSON, so an agent (Claude Code, Codex, Copilot, or any LLM that can fetch a URL) reads the exact rules of the edition the app is built to, instead of what it remembers of some other one.

What each edition serves

File	What it holds	Read it when
`llms.txt`	The index, in the llms.txt format: what Pauta is, how to start an app, build a feature and audit an app, and every page with its rule count and prefix.	First. It is short.
`llms-full.txt`	Every rule of the edition as Markdown, grouped by page and section, with every link made absolute. No prose, screenshots or mockups.	Building or auditing: the whole rule set in one read.
`rules.json`	Every rule as data, with the levels and the three kinds of check explained.	Scripts, filtering by page or level, comparing editions.
The page itself	The prose, screenshots, mockups and spec tables around the rules.	When the rule alone isn't enough, as on a template page.

Each one lives under its edition: `https://pauta-project.dev/2.0.2/llms.txt`, `.../2.0.2/llms-full.txt` and `.../2.0.2/rules.json`. `latest/` always holds the newest edition, and `https://pauta-project.dev/llms.txt`, at the site root, is a copy of the latest one, where agents look for it first. Every link inside these files points at the numbered edition, so a rule an agent cites keeps its address. The first edition that serves them is 2.0.0; earlier editions have their pages only. `versions.json` at the site root lists every published edition.

An agent that also needs the prose can read the single Markdown file, the whole guide in one file. It is large, and its rules are the same.

What a rule looks like

Every rule is one testable statement with a permanent ID. In `llms-full.txt` it is the same list item as on its page: the ID and level in bold, the statement, then *Why*, *Check* and *Reference* lines. In `rules.json` the same rule is:

```
{
  "id": "KBD-07",
  "level": "MUST",
  "page": "keyboard",
  "pageTitle": "Keyboard first",
  "section": "Command Palette",
  "url": "https://pauta-project.dev/2.0.2/keyboard/#kbd-07",
  "text": "The palette is an in-scene overlay in the window's root `StackPane`, not a `javafx.stage.Popup`.",
  "why": "on Windows a Popup does not reliably take OS keyboard focus, and the whole app can stop receiving keystrokes.",
  "check": {
    "kind": "code",
    "text": "the palette and every picker add their node to the root `StackPane`; none creates a `Popup`."
  },
  "reference": "Editora `ui/CommandPalette.java`."
}
```

- `id` is permanent. It is never renumbered or reused; a removed rule leaves a gap. The prefix names the page (`KBD` is Keyboard first), and `llms.txt` lists every page's prefix.
- `level` is `MUST` (a release blocker for a new app), `SHOULD` (expected unless the app has a stated reason) or `MAY` (worth copying when the app grows into it).
- `check.kind` says how the rule is verified: `code` by reading or searching the source, `test` by a named test the build runs, `by hand` in the running app, which source alone can't settle.
- `reference` names the Editora file, or the other app, that does it. The agent copies from there.
- `why` is `null` when the rule has no reason worth stating.

Point the app's agent at its edition

An app is built to one edition, the one `pauta.version` in its `pom.xml` names (see About window). Its agent should read that edition, not `latest/`: a newer edition may hold MUSTs the app doesn't meet yet, and an agent that reads them will "fix" what nobody asked it to.

Say so in the app's `AGENTS.md` (Repository documents), next to the line that names the edition. The app template's `AGENTS.md` already names its edition in its first lines; add the addresses:

```
### Pauta

This app is built to Pauta <edition>. Its rules: https://pauta-project.dev/<edition>/llms.txt
(every rule in one file: https://pauta-project.dev/<edition>/llms-full.txt).

- Before building a feature, read the rules on its page and copy from the code each rule's Reference names.
- Cite a rule by its ID and edition: "KBD-07 (Pauta <edition>)".
- Read the numbered edition, never latest/. Moving to a newer edition is its own change: pauta.version in
  pom.xml, this file and the README change together.
```

Use the same number everywhere the app names its edition: `pom.xml`, `AGENTS.md`, the README and the About window.

Build a feature to the rules

Give the agent the rules of the page the feature belongs to, the page itself when it has a mockup or a spec table, and the reference code. For example:

```
Add a find bar to the editor. Read the Find bar rules (FIND-*) in
https://pauta-project.dev/<edition>/llms-full.txt and the page
https://pauta-project.dev/<edition>/templates/find-bar/ for its layout. Copy from the
Editora files each rule's Reference names. Meet every MUST; list the SHOULDs you
skipped and why.
```

- **Template pages** keep their mockups and spec tables on the page; `llms-full.txt` holds only their rules. Give the agent the page's address too.
- **Reference code** is in Editora, or the app the reference names. An agent with Editora checked out beside the app reads it faster and more reliably than over the web.
- **When the rule and Editora disagree,** the code is right until someone checks. Ask the agent to report the difference instead of picking a side, and fix the guide or the app in the same change.

Audit an app

`llms.txt` and `llms-full.txt` both carry the procedure, so an agent pointed at either one knows it:

1. Find the edition the app implements: `pauta.version` in its `pom.xml`, the About window, or its README. Read that edition's rules. If the app names none, use the newest.
2. Take each rule in turn. A *code* check: read or search the source as it says. A *test* check: find the named test, or the app's equivalent, and confirm the build runs it. A *by hand* check: report it as manual.
3. Report one line per rule: its ID, its level, pass, fail, not applicable or manual, and the evidence as `file:line`. A failed `MUST` blocks a release.
4. A rule applies to every app unless its text names the apps it covers ("An app with user documents...").
5. Read the app's `pauta.json` (Conformance). A rule it lists is reported as its kind (`not-applicable`, `deviation` or `pending`), not as a failure. An unmet rule it doesn't list, an entry for a rule that passes, and a deviation or pending entry with no link are findings.

A prompt as short as this one is enough:

```
Audit this app against Pauta. Start at https://pauta-project.dev/llms.txt and follow
its "Auditing an app" steps. Write the report to pauta-audit.md, failed MUSTs first.
```

The report reads like this:

```
KBD-07   MUST    pass     src/main/java/com/example/app/ui/CommandPalette.java:57
KBD-17   MUST    fail     src/main/java/com/example/app/ui/MainToolbar.java:88 builds an icon button by hand
KBD-15   SHOULD  manual   Tab order needs the running app
TRAY-01  MUST    n/a      the app has no tray icon
MENU-14  MUST    pending  pauta.json: issue #212
```

- **Split a large audit.** An edition has hundreds of rules. Give each page, or each prefix, its own pass or its own agent, and start with the MUSTs (`jq '.rules[] | select(.level == "MUST")' rules.json`).
- **Check the evidence.** A verdict without `file:line` is a guess. Open a sample of the passes as well as the failures before you act on the report.
- **Manual means a person.** A *by hand* check needs the running app, often on a particular OS. The agent lists them; someone runs them.
- **Gaps per app** on this site is the same audit, done by hand for the six apps. An agent's report for one of them should agree with Gaps per app; where it doesn't, one of them is wrong.

Move an app to a newer edition

The version says how much work the move is (how editions are numbered): a newer patch asks nothing of the app, a newer minor asks it to add, a newer major asks it to undo or redo something. The Changelog says what each edition added and which rules became stronger. To see it rule by rule, compare two editions' `rules.json`:

```
old=2.0.0 new=2.0.2
diff <(curl -s https://pauta-project.dev/$old/rules.json | jq -r '.rules[] | "\(.id) \(.level)"' | sort) \
     <(curl -s https://pauta-project.dev/$new/rules.json | jq -r '.rules[] | "\(.id) \(.level)"' | sort)
```

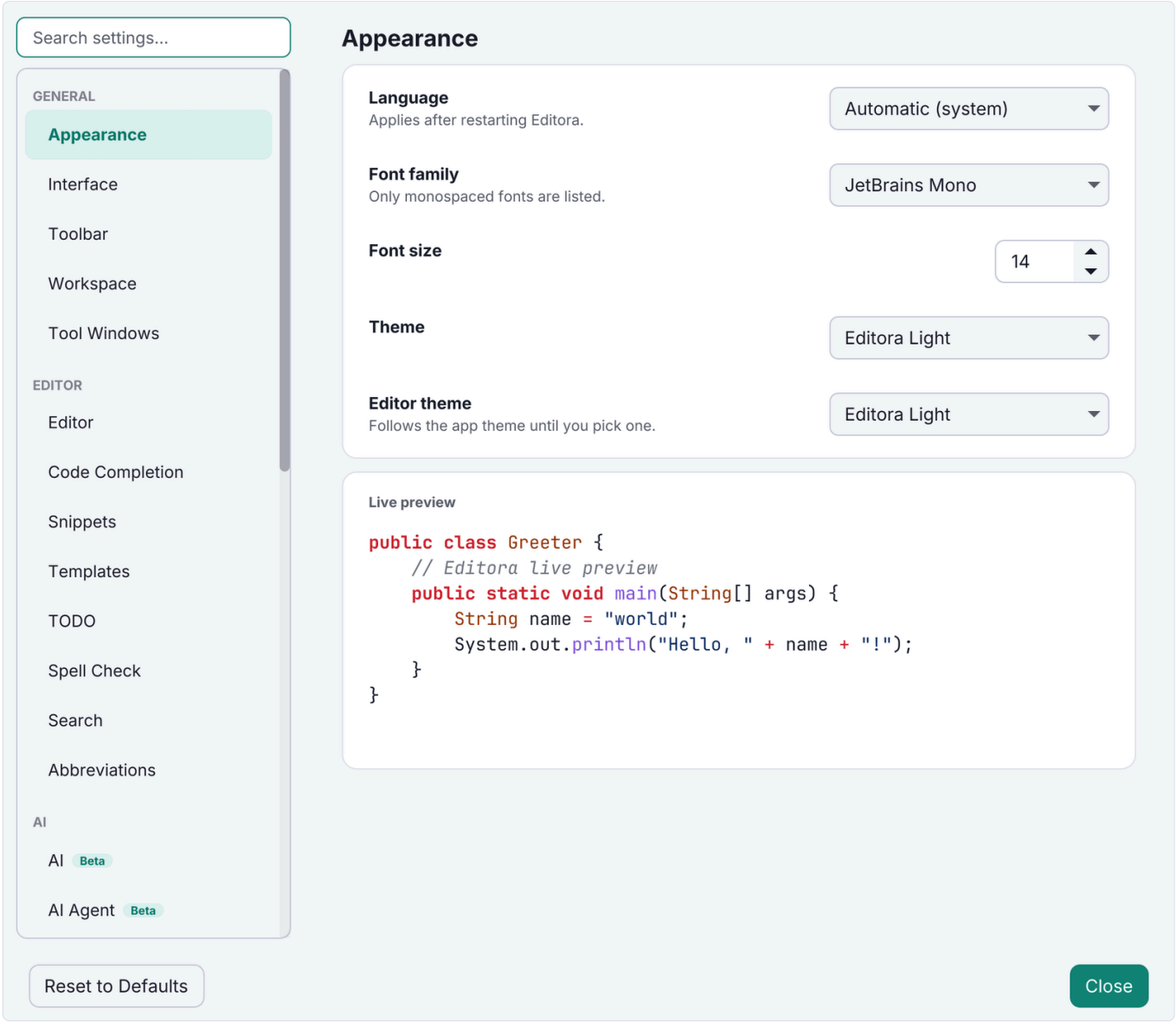
A line only in the new edition is a new rule or a raised level; one only in the old is a retired rule or a lowered level. Give the agent that list and the new edition's `llms-full.txt`, have it meet each new `MUST` or record it in `pauta.json`, replace `src/test/resources/pauta-rules.json` with the new edition's, then change `pauta.version`, `AGENTS.md` and the README together (PAUTA-11).

Working on Pauta itself

This repository has its own `AGENTS.md`: the rule syntax, each page's prefix, and how a change becomes an edition. `npm run check` fails on a malformed rule, and `rules.json`, `llms.txt` and `llms-full.txt` are built from the pages, so a rule written on a page reaches agents in the next edition with nothing else to update.

Platform baseline

- `PLAT-01` `MUST` The app builds on **JDK 25 or newer**. `maven.compiler.release` 25 is the floor, not the ceiling: build and bundle a current JDK.
  - *Check (code):* `pom.xml` sets `maven.compiler.release` to 25 and the enforcer's `requireJavaVersion` to the open-ended range `[25,)`.
  - *Reference:* Editora `pom.xml`.
- `PLAT-02` `MUST` The installers carry a jlinked runtime; users never install Java.
  - *Check (code):* the `dist` profile's jpackage step builds a jlinked app image (`jLinkOptions`) and wraps the installer around it.
  - *Reference:* Editora `pom.xml` (`dist` profile).
- `PLAT-03` `MUST` The app builds on **JavaFX 27 or newer**.
  - *Check (code):* `pom.xml` sets `javafx.version` to 27 or later.
  - *Reference:* Editora `pom.xml`.
- `PLAT-04` `MUST` The app is one named JPMS module, `com.<app>`, whose `module-info.java` opens only what reflection, FXML and Jackson need.
  - *Check (code):* `src/main/java/module-info.java` declares `module com.<app>`; each `opens` line serves FXML, Jackson or another reflective reader.
  - *Reference:* Editora `src/main/java/module-info.java`.
- `PLAT-05` `MUST` Controls use **AtlantaFX 3.x** (`atlantafx-base`), with light and dark themes.
  - *Check (code):* `pom.xml` depends on `atlantafx-base` 3.x; the theme list offers at least one light and one dark theme.
  - *Reference:* Editora `pom.xml`, `ui/Themes.java`.
- `PLAT-06` `MUST` App stylesheets take their colors from AtlantaFX `-color-*` tokens, or from state tokens built on them; no component rule carries a fixed hex. See Icons and color.
  - *Check (code):* `grep -nE '#[0-9a-fA-F]{3,8}\b' app.css` finds hexes only where a token is defined (a `ladder()` of the ground, a scoped `-color-*` block), not in a component's `-fx-*-color` or `-fx-fill`.
  - *Reference:* Editora `app.css`; `StyleContrastTest.severityAndDebugColoursComeFromTheStateTokens` holds the severity and debug colors. Editora's `app.css` still has fixed hexes in a few component rules (the breakpoint markers, the execution arrow, the editor area's defaults).
- `PLAT-07` `SHOULD` The app bundles **Inter** (Regular, Bold, Italic, BoldItalic and `OFL.txt`), registers it with `Font.loadFont` before any stylesheet, and attaches `ui-font.css` to every scene and dialog pane.
  - *Why:* the family has one look on every OS.
  - *Check (code):* the four Inter `.ttf` files and `OFL.txt` are in the resources; one place loads them before the first stylesheet, and one listener adds `ui-font.css` to every window's scene.
  - *Reference:* Editora `ui/Fonts.java`; `App.installUiFont`.
- `PLAT-08` `MUST` The build is **Maven** with the **Maven Wrapper** committed (`mvnw`, `mvnw.cmd`, `.mvn/wrapper/`), so `./mvnw` works on a clean machine.
  - *Check (code):* `mvnw`, `mvnw.cmd` and `.mvn/wrapper/maven-wrapper.properties` are in the repository.
  - *Reference:* Editora `mvnw`, `.mvn/wrapper/`.
- `PLAT-09` `MUST` Nullness is declared with **JSpecify** and checked by **NullAway** on **Error Prone** at compile time. Every package is `@NullMarked` in its own `package-info.java`.
  - *Check (code):* the compiler arguments include `-Xep:NullAway:ERROR`; every source package has a `package-info.java` with `@NullMarked`.
  - *Reference:* Editora `pom.xml`, `docs/conventions.md#null-annotations-jspecify-checked-by-nullaway`.
- `PLAT-10` `MUST` Code is formatted by **Spotless** with Palantir Java Format, and CI fails an unformatted change.
  - *Check (code):* the spotless plugin configures `palantirJavaFormat`, and a CI job runs `spotless:check`.
  - *Reference:* Editora `pom.xml` (spotless plugin), `.github/workflows/ci.yml` (*Format*).
- `PLAT-11` `MUST` Tests run on **JUnit** (Jupiter, 5 or newer) and **TestFX** over JavaFX's built-in **Headless** glass platform (`-Dglass.platform=Headless`), so FX tests need no display, no xvfb and no Monocle.
  - *Check (code):* surefire's `systemPropertyVariables` set `glass.platform` to `Headless`; no Monocle dependency and no xvfb step in CI.
  - *Reference:* Editora `pom.xml` (surefire), `docs/testing.md`, `FxTestSupport`.
- `PLAT-12` `MUST` A **JaCoCo** `check` runs at `mvn verify`: **≥ 0.90 lines, ≥ 0.90 methods**, ≥ 0.79 branches, plus per-package floors.
  - *Check (code):* the `jacoco-check` execution is bound to `verify` with `BUNDLE` limits of 0.90 `LINE` and 0.90 `METHOD`.
  - *Reference:* Editora `pom.xml` `jacoco-check`.
- `PLAT-13` `SHOULD` The app logs through `java.util.logging`, captured in-app by the Debug Log.
  - *Check (code):* the Debug Log adds a handler to the root `java.util.logging` logger; the app's own code imports no other logging API.
  - *Reference:* Editora `ui/DebugLog.java`.
- `PLAT-14` `SHOULD` JSON config files are Jackson POJOs, each with a `SCHEMA_VERSION`.
  - *Check (code):* every class written to the config folder declares `SCHEMA_VERSION` and a `schemaVersion` field.
  - *Reference:* Editora `config/Settings`, `config/migration/ConfigSchema.java`.



Settings → Appearance in Editora: language, fonts, app theme and editor theme, with a live preview.

Everyday commands (identical in every repo)

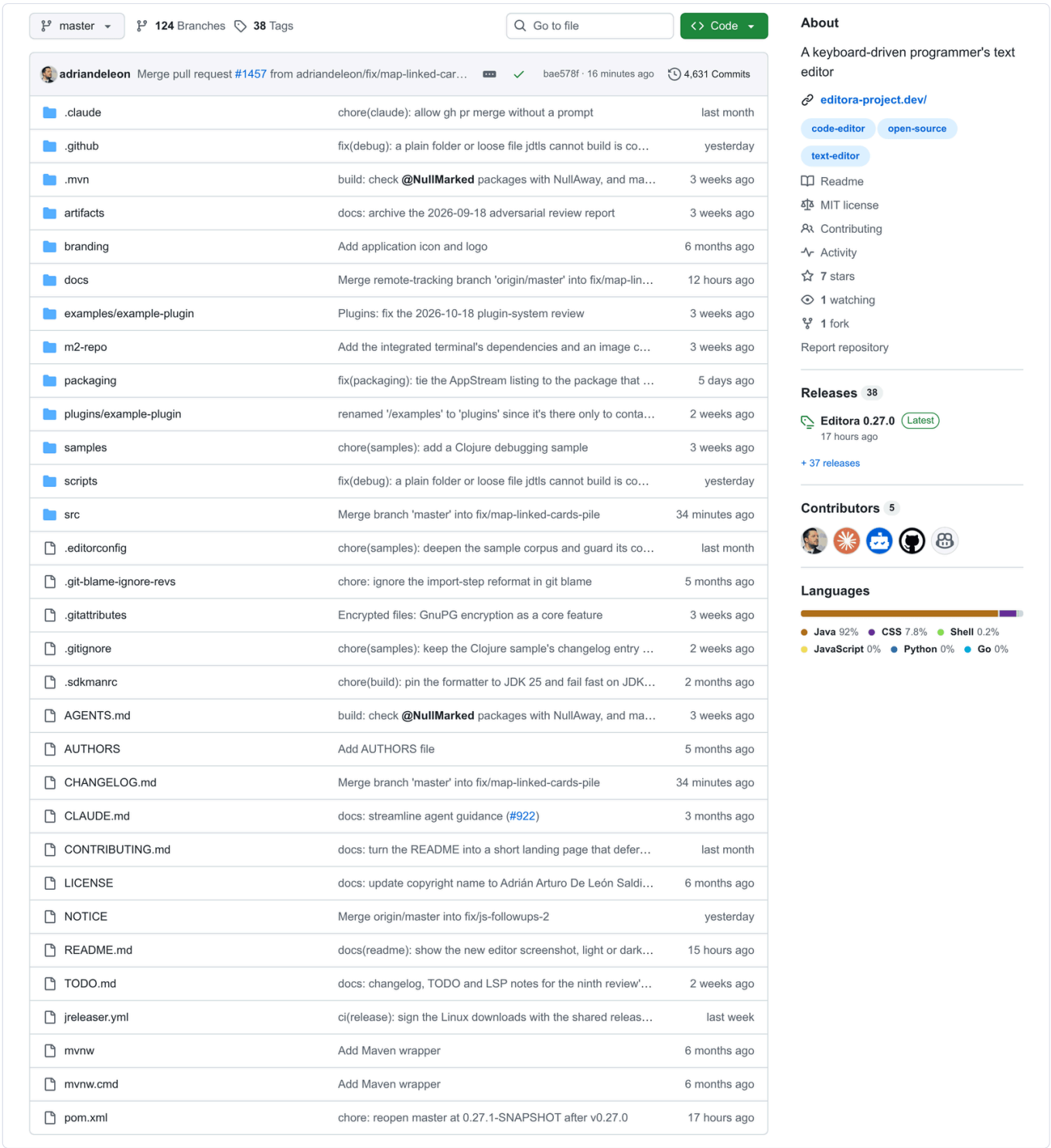
```
./mvnw javafx:run                # run the app
./mvnw test                      # unit + headless FX tests, writes the JaCoCo report
./mvnw spotless:apply            # format
./mvnw verify                    # full gate: tests, coverage check, NullAway
./mvnw -Pfatjar package          # runnable fat jar for this OS
./mvnw clean -Pdist package      # native installer for this OS (clean is mandatory)
```

The `clean` in every `-Pdist` build is mandatory: incremental compilation can leave synthetic enum-switch classes out of `target/classes`, and jlink then ships an app whose keyboard fails on the first keypress.

Repository documents

Start from the template. adl-desktop-app-template carries an example of every file on this page, filled in for a placeholder app, `ExampleApp`. `scripts/rename.sh <AppName> <package>` renames it to yours.

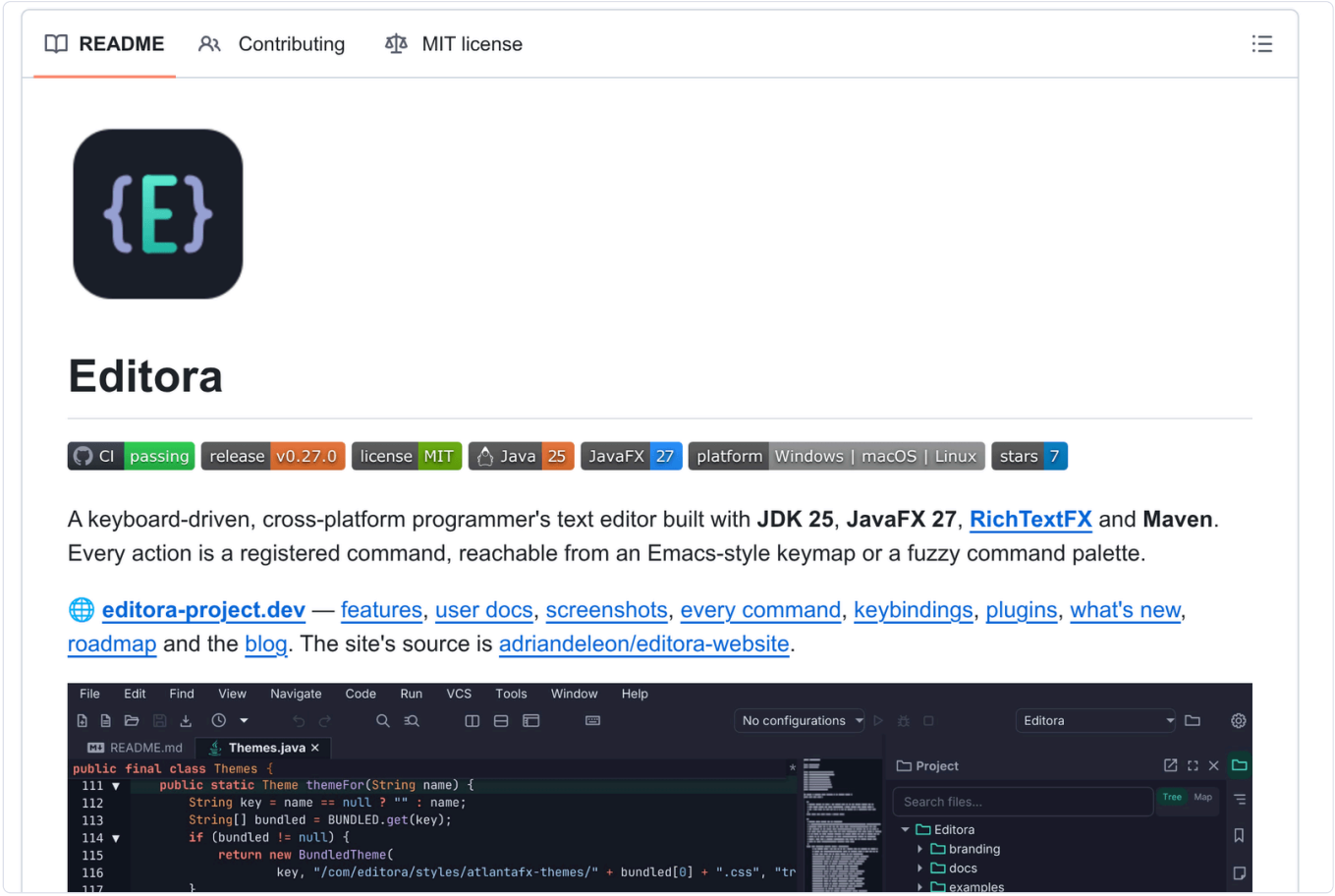
- **REPO-01** `MUST` `README.md` is a landing page of about 200 lines: logo, badges, a one-paragraph pitch, the hero screenshot, highlights, install, build, command line, config folder, license. Detail lives on the website.
  - *Check (code):* `README.md` is near 200 lines and has those parts, with links to the website for the rest.
  - *Reference:* Editora `README.md`.
- **REPO-02** `MUST` `CHANGELOG.md` follows Keep a Changelog 1.1 and SemVer: an `## [Unreleased]` section with Added / Changed / Fixed / Removed, and each entry opens with a bold one-line summary written for users.
  - *Check (code):* the file names Keep a Changelog and Semantic Versioning, starts with `## [Unreleased]`, and every entry starts with `- **`.
  - *Reference:* Editora `CHANGELOG.md`.
- **REPO-03** `MUST` `LICENSE` is MIT, "Copyright (c) 2026 Adrián Arturo De León Saldivar".
  - *Check (code):* `LICENSE` opens with `MIT License` and that copyright line.
  - *Reference:* Editora `LICENSE`.
- **REPO-04** `MUST` `NOTICE` gives the app name, the copyright, "licensed under the MIT License", then every bundled third-party component with its license (Inter is OFL, AtlantaFX is MIT, OpenJFX is GPLv2+CPE...). It ships inside the installers too.
  - *Check (test):* a test fails when a runtime dependency in `pom.xml` has no `NOTICE` entry, and another when `LICENSE` and `NOTICE` are not packaged.
  - *Reference:* Editora `NOTICE`, `NoticeCoverageTest`, `BuildHygieneTest`.
- **REPO-05** `MUST` `AGENTS.md` holds the operational rules for coding agents: essential commands, worktree-per-task, the change checklist. It is not a changelog. A short `CLAUDE.md` points to it.
  - *Check (code):* `AGENTS.md` has sections for commands, worktrees and the change checklist; `CLAUDE.md` is a few lines that send the reader to `AGENTS.md`.
  - *Reference:* Editora `AGENTS.md`, `CLAUDE.md`.
- **REPO-06** `MUST` `CONTRIBUTING.md` says how to build, test, format, and open a PR.
  - *Check (code):* `CONTRIBUTING.md` names the build, test and format commands and the PR step.
  - *Reference:* Editora `CONTRIBUTING.md`.
- **REPO-07** `MUST` `docs/` holds `architecture.md`, `conventions.md`, `testing.md`, `gotchas.md`, `release.md`, `building-and-packaging.md` and `decisions/` (ADRs).
  - *Check (code):* each of those paths exists under `docs/`.
  - *Reference:* Editora `docs/`.
- **REPO-08** `MAY` `TODO.md` holds the open list, kept in the same PR as the change.
  - *Check (code):* `TODO.md` exists at the root.
  - *Reference:* Editora `TODO.md`.
- **REPO-09** `MUST` `.github/ISSUE_TEMPLATE/bug_report.yml` is an issue form whose field ids (`environment`, `report`) the in-app Report a Problem fills.
  - *Check (code):* the form has fields with `id: environment` and `id: report`.
  - *Reference:* Editora `.github/ISSUE_TEMPLATE/bug_report.yml`.
- **REPO-10** `MUST` `branding/` holds the source SVG of the icon. The PNG, ICNS and ICO generated from it sit beside it or in `packaging/`.
  - *Check (code):* `branding/` has an `.svg`; the `jpackage` icon paths in `pom.xml` point at the generated files.
  - *Reference:* Editora `branding/editora-icon.svg`, with `editora.png`, `.icns` and `.ico` beside it; the template keeps them in `packaging/icons/`.



Editora's repository root: every file on this page, with the homepage, Readme, MIT license and Contributing links GitHub derives from them.

README badges (in this order)

```
[![CI](https://github.com/adriandeleon/<App>/actions/workflows/ci.yml/badge.svg)](…/actions/workflows/ci.yml)
[![Release](https://img.shields.io/github/v/release/adriandeleon/<App>)](…/releases/latest)
[![License: MIT](https://img.shields.io/github/license/adriandeleon/<App>)](LICENSE)
![Java](https://img.shields.io/badge/Java-25%2B-orange?logo=openjdk&logoColor=white)
![JavaFX](https://img.shields.io/badge/JavaFX-27%2B-1e90ff)
![Platforms](https://img.shields.io/badge/platform-Windows%20%7C%20macOS%20%7C%20Linux-lightgrey)
[![Pauta](https://img.shields.io/badge/Pauta-4.1.0%20%C2%B7%20Core-5b8def)](<homepage>/docs/pauta/)
[![Stars](https://img.shields.io/github/stars/adriandeleon/<App>?style=flat)](…/stargazers)
```



The top of Editora's README: logo, badges in this order, a two-sentence pitch, links to the website, then the hero screenshot.

The Pauta badge names the edition and the conformance and links to the app's *Pauta conformance* docs page; a test holds it to `pauta.version` and the computed conformance (Conformance).

The hero screenshot is a `<picture>` with a dark `srcset` under `prefers-color-scheme: dark`, so GitHub shows the screenshot that matches the reader's theme. Screenshots are taken at 2× by a headless harness so they can be retaken after any UI change. The README states that the app is built with the help of AI coding tools.

Documentation

An app ships with its documentation. There are three readers and three places: the person using the app reads the **website's docs**; the person changing the app reads `docs/`; a coding agent reads `AGENTS.md`. The README is the landing page that points to each. None of it is optional, and none of it is written after the fact: a change that affects what a reader would learn updates the documentation in the same pull request.

Reference: Editora's `AppInfo.docsUrl()`, `WindowCommandRegistrar` (`help.documentation`), `MenuBarModel` (the Help menu), `WelcomePane` (the Help row), `CommandPalette.docsUrl` and `SettingsFrameBits.helpUrl` (deep links); the repository's `docs/README.md`, `docs/conventions.md#docs-in-the-same-pr` and `CONTRIBUTING.md`; and the editora-website repository's `src/content/docs/` and `scripts/`.

User documentation

- `DOCS-01` `MUST` Every app has user documentation: a *Getting started* page (install, first launch, the first thing to do), one page per feature area, the keyboard reference, the Settings, and an FAQ. It is written for the person using the app, in their words, with screenshots from the real window. The README's 200 lines are not it.
  - *Check (code):* the website repository's `src/content/docs/` has a getting-started page, a keymaps page, a settings or configuration page, an FAQ and a page per feature area.
  - *Reference:* editora-website `src/content/docs/` (43 pages in nine categories, from *Getting started* to *Help*); torrenta-website (19), termina-website (16), mirada-website (11).
- `DOCS-02` `MUST` The user docs live on the app's website, under `/docs/`, in the app's `<app>-website` repository, beside the feature pages, the screenshots, the news posts and the blog. Each docs page is a Markdown file whose frontmatter gives its `category` and `order`; the sidebar, the sitemap and the RSS feed follow. The app's repository never carries a second copy.
  - *Check (code):* every file in the website's `src/content/docs/` has `category` and `order` in its frontmatter; the app's `docs/` holds no user guide.
  - *Reference:* editora-website `README.md` (*Content*); Editora's `AGENTS.md`: "User-facing feature documentation goes to the separate website repository".
- `DOCS-03` `SHOULD` Deep links from the app open the running version's docs. Pages live under `/docs/v-<version>/…`; bare `/docs` and every unversioned slug redirect to the current version, so a link in a page never carries a version and a bump just regenerates `src/lib/version.ts`. The app builds the base from its own version with `-SNAPSHOT` dropped, so a development build points at the release it is working toward instead of a path that would always 404.
  - *Why:* a release that pointed at another release's docs would claim what is not true (*Never claim what is not true*), so this becomes a must the day the app links to a specific page.
  - *Check (code):* the app's docs URL is `<homepage>/docs/v-` plus its version without `-SNAPSHOT`; the site's `astro.config.mjs` serves `/docs/v-<version>/` and redirects the unversioned paths.
  - *Reference:* Editora `AppInfo.docsUrl()`, `releaseVersion()`; editora-website `astro.config.mjs`, `README.md` (*Versioned docs*). The other three sites carry a comment to port these routes when their app first deep-links.
- `DOCS-04` `SHOULD` Docs are generated where the app is the source. The Commands and Keybindings pages are generated from the app's command registrations, `messages.properties` and keymaps; the Roadmap and What's new from `TODO.md` and `CHANGELOG.md`; the version from `pom.xml`. The output is committed, so the site builds without the app checked out. A check fails the site when a docs page quotes a command id that does not exist or a chord that is bound to a different command.
  - *Why:* one fact, one source (*One source of truth for each fact*).
  - *Check (code):* the website's `scripts/` holds the generators, their output under `src/lib/` is committed, and `npm run check:docs` runs the id and chord check.
  - *Reference:* editora-website `scripts/GenCommands.java` (every `Command.of` in Editora's `com.editora.ui`, `MainController` holding most), `GenRoadmap.java`, `GenFeatures.java`; `scripts/check-doc-commands.mjs` (`npm run check:docs`).
- `DOCS-05` `MUST` Release notes are written for users. Each `CHANGELOG.md` entry opens with a bold one-line summary (Repository documents); the GitHub release carries the section, the Welcome page's *What's New* opens this build's release, and the website's What's new is generated from the same file.
  - *Check (code):* the Welcome page's *What's New* opens `AppInfo.releaseNotesUrl()`; the website's What's new is generated from `CHANGELOG.md`.
  - *Reference:* Editora `AppInfo.releaseNotesUrl()`, `WelcomePane` (`welcome.whatsNew`); editora-website `GenRoadmap.java`.

Reachable from the app

The documentation is one command away, so nobody has to know the website's address.

- `DOCS-06` `MUST` **Help → Documentation** opens the user docs in the browser. It is a command (`help.documentation`), so it is in the palette, bindable and in the menu through the registry like every other row; it is the first row of the Help menu after the Command Palette, and the Welcome page's Help row repeats it.
  - *Check (code):* `help.documentation` is registered, follows `palette.show` in the Help `MenuSpec`, and the Welcome page's Help row runs it through the registry.
  - *Reference:* Editora `WindowCommandRegistrar` (`help.documentation` → `MainController.openDocumentation()`), `MenuBarModel` (`menubar.help`), `WelcomePane` (`welcome.documentation`).
- `DOCS-07` `SHOULD` The Help menu runs in this order: Command Palette · Documentation · Check for Updates · Message Log · Debug Log · Report a Problem · Pauta Conformance · About. Simple mode keeps Command Palette, Documentation and About. An app without a feature drops its row; nothing moves.
  - *Check (code):* both `menubar.help` menu specs list those command ids in that order.
  - *Reference:* Editora `MenuBarModel`, both `MenuSpec("menubar.help", …)`.
- `DOCS-08` `SHOULD` Help has a keyboard reference: a searchable window of every key, grouped as the menus are, or the keybinding editor.
  - *Check (by hand):* open it from Help, search for a command, and find its key.
  - *Reference:* Termina's and Mirada's `ShortcutsWindow`; Editora's keybinding editor (Keyboard first).
- `DOCS-09` `MUST` Every command has a one-line description (`command.<id>.desc`), shown under the title in the palette. It is the in-app documentation of the command.
  - *Check (test):* the parity test fails when a `command.<id>` title has no `.desc`.
  - *Reference:* Editora `messages.properties`, `MessagesTest.everyCommandTitleHasAMatchingDescription` (Code quality gates).
- `DOCS-10` `MAY` The palette opens per-command help. With a command highlighted, `C-h` opens its page, `/docs/v-<version>/commands/<id>`; where the keymap binds `C-h`, the first free of `F1` and `Shift+F1` does. Search Everywhere shares the URL builder so the two pickers can never disagree.
  - *Check (code):* the palette and Search Everywhere both build the URL with `CommandPalette.docsUrl`.

- *Reference:* Editora `CommandPalette.docsUrl`, `setDocsOpener`; `PickerKeys.freeChord`.

- **DOCS-11** `MAY` Each page of the Settings window has a *Help for <page>* link to its docs page, from one map of page to slug.
  - *Check (code):* one map from Settings page to docs slug feeds every page's help link.
  - *Reference:* Editora `SettingsWindow`, `SettingsFrameBits.helpUrl`, `settings.help`.

Strings

Key	English
`command.help.documentation`	Documentation
`command.help.documentation.desc`	Open the <App> documentation site for this version in your browser.
`welcome.documentation` / `welcome.whatsNew`	Documentation / What's New
`settings.help`	Help for {0}

Developer documentation

- **DOCS-12** `MUST` `docs/` has an index. `docs/README.md` says who the folder is for (contributors; user docs live on the website), states that the code wins when the two disagree, and lists every guide with one line. The set is the one Repository documents names: `architecture.md`, `conventions.md`, `testing.md`, `gotchas.md`, `release.md`, `building-and-packaging.md` and `decisions/`.
  - *Check (code):* `docs/README.md` exists, says those two things, and links each guide in `docs/`.
  - *Reference:* Editora's `docs/README.md`; the template's `docs/`.
- **DOCS-13** `SHOULD` Every choice a newcomer would question has a decision record: a fork, a rejected dependency, a storage format, an escape hatch. It is one numbered Markdown file, `docs/decisions/NNNN-slug.md`, with the context, the decision and what it costs.
  - *Check (code):* `docs/decisions/` holds `NNNN-slug.md` files, each with context, decision and consequences.
  - *Reference:* Editora's `docs/decisions/` (18 records, from `0001-richtextfx-fork.md`).
- **DOCS-14** `MUST` Three files have three jobs. `AGENTS.md` is the operational guide, kept short (commands, worktree-per-task, the change checklist). `CONTRIBUTING.md` is the short version for a person, pointing into `docs/`. `docs/` holds the durable explanations. Exhaustive history goes to `docs/reference/`, not into any of the three.
  - *Check (code):* `AGENTS.md` and `CONTRIBUTING.md` link into `docs/`; long catalogs sit under `docs/reference/`.
  - *Reference:* Editora's `AGENTS.md` (*Documentation map*), `CONTRIBUTING.md`, `docs/reference/README.md`.
- **DOCS-15** `SHOULD` Javadoc sits on its member: a doc comment above another doc comment documents nothing. Comments say why, not what.
  - *Check (test):* a test fails the build on a Javadoc comment followed by another.
  - *Reference:* Editora `StackedJavadocTest`.

Kept in step

- **DOCS-16** `MUST` Docs change in the same PR. A change that affects behavior updates `CHANGELOG.md`, `TODO.md`, the contributor guides it touches and the website's user docs in the same change, never as a follow-up. A reviewer treats a feature PR with no docs change as incomplete.
  - *Check (code):* `docs/conventions.md`, `CONTRIBUTING.md` and `AGENTS.md` state the rule; in review, a feature PR's diff touches `CHANGELOG.md`.
  - *Reference:* Editora `docs/conventions.md#docs-in-the-same-pr`, `CONTRIBUTING.md` (step 4), `AGENTS.md`.
- **DOCS-17** `MUST` When documentation and code disagree, the code wins, and the documentation is fixed in the same change. The same rule this guide applies to itself.
  - *Check (code):* `AGENTS.md` and `docs/README.md` say so.
  - *Reference:* Editora's `AGENTS.md`, `docs/README.md`.
- **DOCS-18** `SHOULD` Screenshots are retaken, not described. The capture hook renders the real window at 2× after any UI change, for the README (its hero in light and dark), the website and bug reports.
  - *Why:* a stale screenshot claims what is not true.
  - *Check (code):* the app has a capture hook or a headless harness that writes the real window to a PNG.
  - *Reference:* `DevCapture` (Shared features); Editora's headless harness.
- **DOCS-19** `MUST` The README defers. It is about 200 lines: pitch, hero, highlights, install, build, command line, config folder, license, and links into the website's docs. It carries no feature catalog, so it changes only when those facts do.
  - *Check (code):* `README.md` is near 200 lines and links to the website's docs instead of listing every feature.
  - *Reference:* Repository documents; Editora `docs/conventions.md#docs-in-the-same-pr`.

Code quality gates

- **QUAL-01** `MUST` NullAway is an error, not a warning. A new package starts with its own `package-info.java` carrying `@NullMarked`.
  - *Check (code):* the compiler arguments say `-Xep:NullAway:ERROR`; every source package has a `package-info.java` with `@NullMarked`.
  - *Reference:* Editora `pom.xml`, `docs/conventions.md` (*Null annotations*).
- **QUAL-02** `SHOULD` A NullAway report is fixed, not suppressed: put `@Nullable` on what may be null.
  - *Check (code):* `grep -rn '@SuppressWarnings("NullAway")' src/main` finds nothing.
  - *Reference:* Editora; one suppression is left, in `recovery/RecoveryCodec.java`.
- **QUAL-03** `SHOULD` Coverage floors are a regression net, not a target. Beyond the build-wide floor of 0.90 lines and methods, each package of 300+ lines gets its own floor about three points below its measured level; when a package's coverage rises, its floor is ratcheted up.
  - *Check (code):* the `jacoco-check` rules in `pom.xml` carry a `PACKAGE` line floor for each large package.
  - *Reference:* Editora `pom.xml` `jacoco-check`.
- **QUAL-04** `SHOULD` When a guide quotes the coverage floors, a test fails when the quoted table and the pom disagree.
  - *Check (test):* a build test reads the `jacoco-check` rules from `pom.xml` and compares them with the guide's table.
  - *Reference:* Editora `BuildHygieneTest.theCoverageFloorsQuotedInTheTestingGuideAreTheOnesTheBuildEnforces`, `docs/testing.md`.
- **QUAL-05** `MUST` Extract the decision, then test it (principle 12). Prefer a pure decision helper plus unit tests; use the headless FX harness only when controller or toolkit behavior is the point.
  - *Check (code):* enablement, visibility and navigation rules sit in classes that import no `javafx` type, each with a unit test not tagged `fx`.
  - *Reference:* Editora `docs/testing.md` (*Prefer pure logic*): `ui/Chrome`, `editops/TextNav`, `config/PathKeys`, each with a plain unit test.
- **QUAL-06** `MUST` FX tests are tagged `@Tag("fx")`, so the pure suite runs alone with `-DexcludedGroups=fx`.
  - *Check (code):* every test class that boots the toolkit (`FxTestSupport`, `FxToolkit`) carries `@Tag("fx")`.
  - *Reference:* Editora `docs/testing.md`. Editora does not hold it fully yet: about twenty `*FxTest` classes, most of them `ProjectMap*FxTest`, carry no tag.
- **QUAL-07** `SHOULD` Tests do not sleep. Workers and FX callbacks are owned by a scope (`AsyncTestScope`), and tests wait on latches.
  - *Check (code):* `grep -rn 'Thread.sleep' src/test` finds nothing.
  - *Reference:* Editora `AsyncTestScope`, `docs/testing.md`. Many older Editora tests still poll with `Thread.sleep`.
- **QUAL-08** `SHOULD` A test window writes only inside its temp folders, never in the real home directory.
  - *Check (test):* the command sweep checks the home directory after running every command.
  - *Reference:* Editora `CommandSweepFxTest.leftInTheHomeDirectory`.
- **QUAL-09** `MUST` UI work is verified in the running app (principle 13): a capture hook or headless screenshot of the real window, with real key and mouse events, before a UI change is called done.
  - *Check (code):* the app has a capture hook (`DevCapture`) or a headless fixture that builds the real window.
  - *Reference:* Editora `FxWindowFixture` (the real `MainController` window, headless); Torrenta and Termina `DevCapture.java`.
- **QUAL-10** `SHOULD` A guard that cannot fail is decoration. When you add a drift or parity test, break the thing it guards once and watch it fail.
  - *Check (by hand):* break what the new test guards, run it, and see it fail.
  - *Reference:* Modula's `DependenciesTest`.

Guard tests that pay for themselves across the family:

- **QUAL-11** `SHOULD` A command sweep runs every registered command in a real headless window.
  - *Check (test):* one FX test iterates the registry and runs each command.
  - *Reference:* Editora `CommandSweepFxTest`.
- **QUAL-12** `MUST` A message parity test holds that every locale has every key and every `command.<id>` has a `.desc`.
  - *Check (test):* the test fails on a key missing from a locale or a command title without `.desc`.
  - *Reference:* Editora `MessagesTest`.
- **QUAL-13** `SHOULD` A settings round trip shows that every switch, number and choice survives a restart.
  - *Check (test):* one FX test sets every control, reloads the settings and reads each back.
  - *Reference:* Editora `SettingsRoundTripFxTest`.
- **QUAL-14** `SHOULD` A contrast test holds that text tokens keep their ratios in every theme.
  - *Check (test):* a pure test reads the stylesheets and does the WCAG arithmetic.
  - *Reference:* Editora `StyleContrastTest`.
- **QUAL-15** `SHOULD` A release supply-chain test keeps the signing and attestation steps where they are.
  - *Check (test):* a test reads the workflows and fails when a signing, notarization or attestation step moves or goes.
  - *Reference:* Editora `ReleaseSupplyChainTest`.

Performance

A desktop app is judged by how it feels under the hand: a key that lags or a window that hangs while it reads a file is a defect, not a slow feature. These rules come from Editora's `docs/performance.md`, the contract every change on a hot path there must honor.

The UI thread

- **PERF-01** `MUST` Never block the JavaFX application thread. No file or network I/O, no process start or wait, no parsing, searching, hashing or JSON of any size on it, and no `Thread.sleep`, `Future.get()`, `join()` or `Process.waitFor()` there.
  - *Check (code):* event handlers, listeners and `Platform.runLater` bodies call no `Files.*`, `HttpClient.send`, `ProcessBuilder.start`, `waitFor`, `Future.get`, `join` or `Thread.sleep`; that work runs on an executor or a virtual thread.
  - *Reference:* Editora `docs/performance.md` §1.
- **PERF-02** `MUST` Touch the scene graph only on the FX thread. Background work hands its result back with `Platform.runLater`; nothing else creates, changes or reads live nodes.
  - *Check (code):* every executor task or thread body that ends in UI work wraps that work in `Platform.runLater`.
  - *Reference:* every Editora service, for example `git/GitService.java`.
- **PERF-03** `MUST` A background result is applied under a generation guard, so an answer to an old request never overwrites a newer one.
  - *Check (code):* each request takes a generation (`AtomicLong.incrementAndGet`) or goes through `LatestOnly`, and its result is applied only while that generation is still current, checked again inside the `runLater`.
  - *Reference:* Editora `git/GitService.java`, `search/SearchService.java`, `ui/LatestOnly.java` (`LatestOnlyTest`).
- **PERF-04** `SHOULD` Every thread is named and is a daemon.
  - *Why:* a thread dump then says what is running, and a stray worker never keeps the app alive after its last window closes.
  - *Check (code):* every `Executors.new…` and `new Thread` gets a name and `setDaemon(true)`, through a thread factory such as `daemon("my-feature")`; virtual threads are daemons already.
  - *Reference:* Editora `daemon("my-feature")` in the shape below, `terminal/TerminalExecutors.java`.
- **PERF-05** `MUST` Blocking work stays off the common `ForkJoinPool`, and a `Process` subclass overrides `onExit()`.
  - *Why:* the common pool has one worker fewer than the machine has cores, so a few blocked waits (`CompletableFuture.supplyAsync` of a `waitFor`) starve every other user of it.
  - *Check (code):* no `supplyAsync` or `runAsync` without an executor wraps a blocking call, and every class that extends `Process` overrides `onExit()`.
  - *Reference:* Editora `process/ExitWatch.java`, `docs/gotchas.md`. Editora's `DapManager.reportJavaExit` still waits up to 3 s for an exit code in a `supplyAsync` on the common pool.

The canonical shape, used by every Editora service:

```
private final ExecutorService exec = Executors.newSingleThreadExecutor(daemon("my-feature"));
private final AtomicLong gen = new AtomicLong();

void request(String input, Consumer<Result> onResult) {
    long mine = gen.incrementAndGet();
    exec.submit(() -> {
        Result r = computeOffThread(input);        // heavy work, off the FX thread
        if (mine == gen.get()) {                   // superseded? drop it
            Platform.runLater(() -> {
                if (mine == gen.get()) onResult.accept(r);
            });
        }
    });
}
```

For one-off blocking calls (reading a file, stat-ing a path, starting a process), a virtual thread (`Thread.ofVirtual().start(…)`) is enough; the result still comes back through `Platform.runLater`.

Hot paths

Each app names its hot paths: what runs on every keystroke, scroll pulse or progress tick. In Editora they are typing, scrolling, highlighting, the overlays and the gutter. In Torrenta they are the transfer table's refresh, in Termina the terminal grid, in Mirada stepping between images.

- **PERF-06** `MUST` Redraws are coalesced to at most one per pulse with a `pending` flag, and text-driven work waits for typing to pause.
  - *Check (code):* a redraw request sets a `pending` flag and schedules one pass; listeners on text go through a debounce.
  - *Reference:* Editora `ui/FxThrottle.java`.
- **PERF-07** `MUST` Streams (process output, logs, progress) reach the FX thread in batches at least 16 ms apart, never one `runLater` per line.
  - *Check (code):* a reader thread appends to a buffer that the FX thread drains, with a minimum interval between drains.
  - *Reference:* Editora `process/OutputPump.java` (`MIN_DRAIN_INTERVAL_MILLIS`), `ui/ConsoleAppender.java`.
- **PERF-08** `MUST` Work only on what is visible and only on what changed: no O(document) or O(list) work per keystroke, scroll or tick. Anything that can grow is shown in a virtualized control (`ListView`, `TableView`, a fixed cell size).
  - *Check (code):* a listener on text, scroll or a timer re-processes the edited range or the visible rows, never the whole document or list; growing lists are `ListView` or `TableView`, not a `VBox` of nodes.
  - *Reference:* Editora `editor/highlight/HighlightPass.java`, `editor/minimap/MinimapRows.java`.

- **PERF-09** `SHOULD` A hot path allocates little: it reuses buffers and arrays, and caches data a repaint only reads.
  - *Check (code):* the per-frame and per-keystroke code allocates no new arrays or images when the size has not changed.
  - *Reference:* Editora `editor/minimap/Minimap.java` (the snapshot image reused at the same size), `MinimapRows.java`.
- **PERF-10** `SHOULD` CSS stays cheap: no `applyCss()` on a large subtree, stable style classes, and spans of the same style merged before they are applied.
  - *Check (code):* `applyCss()` is called only on small nodes; style classes are not toggled per keystroke.
  - *Reference:* Editora `SpanMerger` in `editor/highlight/TextMateHighlighter.java`, `docs/gotchas.md`.

Memory and size

- **PERF-11** `MUST` Everything that grows is bounded: logs, histories, consoles, caches, undo. The newest is kept, with a note of what was cut.
  - *Check (test):* each such store has a test that fills it past its cap and finds the oldest dropped.
  - *Reference:* Editora `ui/MessageLog.java` (200, `MessageLogTest`), `ui/DebugLog.java` (`DebugLogTest`), `test/OutputCap.java`.
- **PERF-12** `MUST` Large inputs are guarded by size and shape before they load: expensive features turn off past one threshold, the input opens read-only or capped past another, and what will not fit the free heap is refused.
  - *Check (code):* the loader compares the size against named thresholds before reading the content.
  - *Reference:* Editora `EditorBuffer.LARGE_FILE_BYTES` (5 MB) and `HUGE_FILE_BYTES` (50 MB), `ui/LoadHeapGuard.java` (`LoadHeapGuardTest`).
- **PERF-13** `MUST` A closed window releases what it held: listeners, timers, executors, file watchers, native resources.
  - *Check (test):* a test opens and closes windows and checks nothing is left running.
  - *Reference:* Editora `FxWindowFixture.dispose` (`FxWindowFixtureLifecycleFxTest`), `ClosedLspManagers`.

Start-up

- **PERF-14** `MUST` The first frame comes first: only what the first window shows is built before it; tool windows, panels and pages build on first open.
  - *Check (test):* a test opens a window and finds no closed tool window's panel built before the first paint.
  - *Reference:* Editora's lazy `ToolWindow` suppliers, `LazyToolWindowsFxTest`.
- **PERF-15** `MUST` What nobody waits for starts after the first frame has painted: the update check, extra fonts, indexes and caches.
  - *Check (test):* a test finds the deferred work not started before the first paint.
  - *Reference:* Editora `ui/UpdateCheckScheduler.java`, `Fonts.loadRemainingAfterNextPaint` (`UpdateCheckSchedulerTest`, `FontsDeferredLoadFxTest`, `StartupDeferralFxTest`).
- **PERF-16** `SHOULD` Config is read off the FX thread, before the toolkit needs it.
  - *Check (code):* `main` starts the config read on its own thread before `launch`.
  - *Reference:* Editora's `config-prefetch` thread in `App.java`.
- **PERF-17** `SHOULD` The app ships precompiled stylesheets (`.bss`) and an AOT cache trained on a real session.
  - *Check (code):* the build converts the `.css` files to `.bss` and runs a training launch for the AOT cache.
  - *Reference:* Editora `scripts/css2bss.java`, `scripts/aot_build.java`; Termina's and Mirada's `scripts/aot-train.sh`.

Measure, don't guess

- **PERF-18** `SHOULD` Every change on a hot path says what it costs in its pull request: added FX-thread work, allocation per keystroke or scroll, extra layout passes, memory. "Negligible" is an answer; no answer is not.
  - *Check (by hand):* read the app's recent pull requests that touch a hot path.
  - *Reference:* Editora `docs/performance.md` (its opening paragraph).
- **PERF-19** `SHOULD` Start-up is measured by a script that takes the median of several cold launches, and by phase marks in the app.
  - *Check (code):* the repository has the script, and `main` and the first paint record phase marks.
  - *Reference:* `scripts/measure-startup.sh` in Editora and Mirada; Editora `perf/Startup.java`.
- **PERF-20** `SHOULD` The app can sample its FX thread when something stalls.
  - *Check (code):* an environment variable or property turns on a sampler of the FX thread's stack.
  - *Reference:* Editora `perf/Startup.java` (`EDITORA_PERF_SAMPLE=1`).
- **PERF-21** `SHOULD` Each hot path that once regressed keeps a headless test.
  - *Check (test):* the fix for a hot-path regression adds a headless test that fails on the old code.
  - *Reference:* Editora `MinimapDeferredRenderFxTest`.
- **PERF-22** `SHOULD` A suspected regression gets a number before and after, from temporary `System.nanoTime` instrumentation if nothing better exists.
  - *Check (by hand):* the pull request that fixes it states both numbers.
  - *Reference:* Editora `docs/performance.md`.

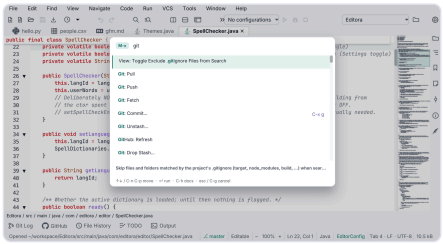
Keyboard first

Commands

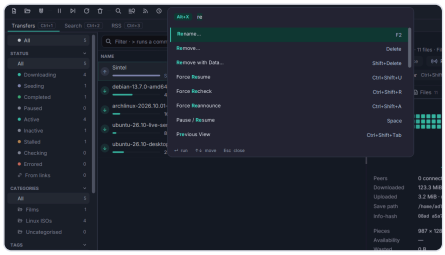
- KBD-01 MUST Every user-facing action is a `Command` (id, title, runnable) registered in one `CommandRegistry`.
  - *Check (code):* menu items, toolbar buttons, palette rows and key bindings run a command id through the registry; no control's handler does work no command can reach.
  - *Reference:* Editora `command/Command.java`, `command/CommandRegistry.java`.
- KBD-02 SHOULD Command ids are dotted and grouped by area: `file.open`, `view.toggleStatusBar`, `help.reportProblem`.
  - *Check (code):* every registered id matches `<area>.<camelCaseName>`.
  - *Reference:* Editora `ui/MenuBarModel.java`.
- KBD-03 MUST A command's title comes from `command.<id>` and its palette description from `command.<id>.desc` in the message catalogs.
  - *Why:* a registered command is then localized and discoverable without extra work.
  - *Check (test):* a catalog parity test fails when a registered command has no `command.<id>` or `command.<id>.desc` key.
  - *Reference:* Editora `i18n/messages.properties`, `MessagesTest`.
- KBD-04 SHOULD A command that cannot run in the current context is shown grayed, with a tooltip that says why, not hidden.
  - *Check (code):* the palette and menus list a disabled command with a reason, rather than filtering it out.
  - *Reference:* Editora `ui/Chrome.java` (`paletteEnabled`, `disabledReason`).

Command Palette

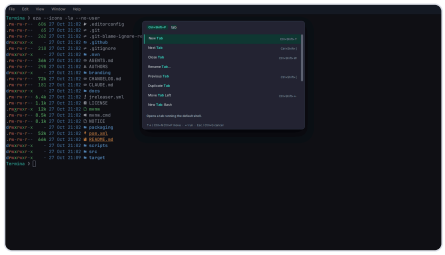
- KBD-05 MUST The Command Palette opens with `M-x` in the Emacs keymap and `Ctrl+Shift+P` (`Cmd+Shift+P` on macOS) in the others.
  - *Check (code):* each bundled keymap binds that chord to the palette command.
  - *Reference:* Editora `keymaps/emacs.json` (`M-x`), `cua.json` and `vscode.json` (`C-S-p`), `cua.mac.json` (`Cmd-S-p`), all to `palette.show`.
- KBD-06 SHOULD The palette matches fuzzily over title, id and description; a query that names a command's title outranks a fuzzy hit. Each row shows the command's binding in the active keymap. See the Command Palette template.
  - *Check (test):* a ranking test puts the title match first.
  - *Reference:* Editora `ui/CommandPalette.java`, `search/FuzzyMatch.java`.
- KBD-07 MUST The palette is an in-scene overlay in the window's root `StackPane`, not a `javafx.stage.Popup`.
  - *Why:* on Windows a Popup does not reliably take OS keyboard focus, and the whole app can stop receiving keystrokes.
  - *Check (code):* the palette and every picker add their node to the root `StackPane`; none creates a `Popup`.
  - *Reference:* Editora `ui/CommandPalette.java`.
- KBD-08 SHOULD The same picker serves every other list choice: recent files, themes, keymaps, a setting's values.
  - *Check (code):* no second list-picker class or `ChoiceDialog` for those choices.
  - *Reference:* Editora `ui/QuickOpen.java`.



Editora: M-x over the editor; typing git lists every Git command.



Torrenta: the same palette, filtered to torrent actions with their shortcuts.



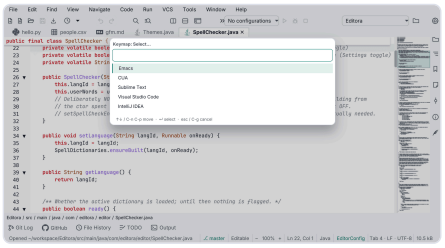
Termina: the palette over a shell session, filtered to tab commands.

Keymaps

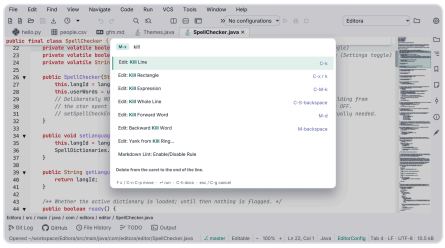
- KBD-09 MUST Emacs and CUA keymaps ship as JSON resources, CUA with a `.mac.json` variant where Cmd replaces Ctrl.
  - *Check (code):* `emacs.json`, `cua.json` and `cua.mac.json` are in the app's resources.
  - *Reference:* Editora `src/main/resources/com/editora/keymaps/`.
- KBD-10 SHOULD IntelliJ, VS Code and Sublime keymaps ship the same way, each with its `.mac.json` variant.
  - *Check (code):* `intellij`, `vscode` and `sublime` `.json` and `.mac.json` files are in the resources.
  - *Reference:* Editora `src/main/resources/com/editora/keymaps/`.
- KBD-11 MUST Keymaps are applied in layers: base keymap → user overrides → plugin keymaps → the user's own bindings again. An app without plugins has the first two.
  - *Why:* the user always has the last word, and a plugin can still use a chord the user freed.
  - *Check (code):* one place applies the layers in that order.
  - *Reference:* Editora `ui/KeymapLayers.java`.
- KBD-12 SHOULD A keymap editor in Settings lists every command with its chord, is searchable, and warns of conflicts.
  - *Check (by hand):* open Settings → Keymap, search, and bind a chord that is taken.
  - *Reference:* Editora `ui/SettingsKeymapsPage.java`; conflicts from `KeybindingEdits.takenDefaults`.
- KBD-13 SHOULD No command's chord is written into translated text. Every label shows the binding from the active keymap. Fixed keys of a control (Enter, Tab, Esc, arrows) and keys meant for another program may be written.

- *Check (code):* no message catalog value names a command's chord (`Ctrl+` , `Cmd+` , `C-x` , `M-x`); chords are rendered by one formatter.
- *Reference:* Editora `command/ChordFormat.java` , the one place a binding is rendered. Editora's `command.edit.universalArgument` title still carries `C-u` .

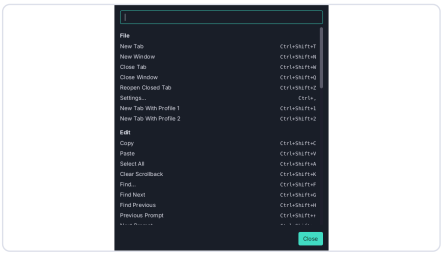
- **KBD-14** `SHOULD` A multi-key prefix (`C-x …`) shows a which-key style hint listing the possible next keys.
  - *Check (by hand):* press `C-x` in the Emacs keymap and wait.
  - *Reference:* no app lists the next keys yet. Editora echoes only the pending prefix (`C-x -`) in the status bar (`command/KeyDispatcher.java`).



Keymap: Select... offers Emacs, CUA, Sublime Text, Visual Studio Code and IntelliJ IDEA.



M-x kill: each command shows its chord from the active keymap.



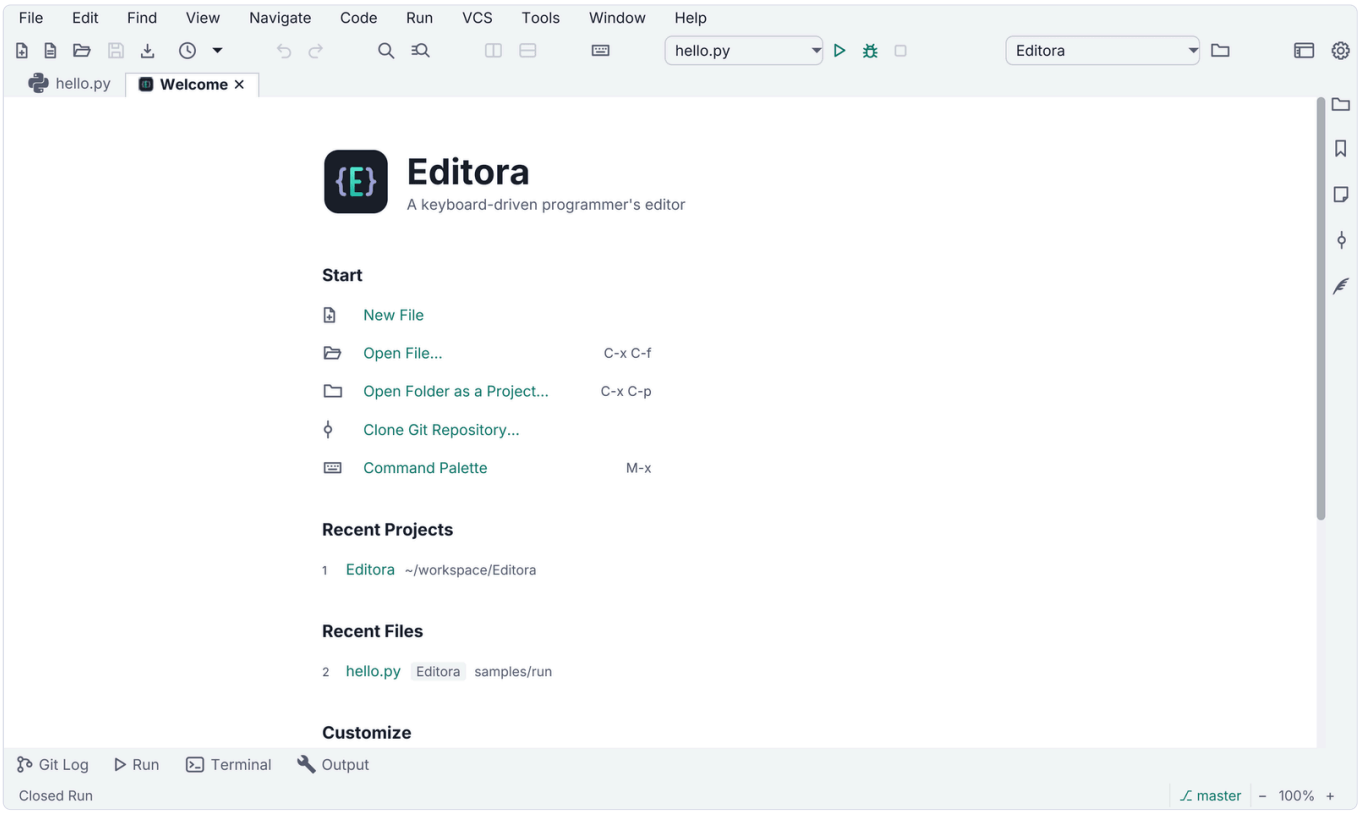
Termina's shortcuts reference: every key, grouped by menu.

Focus and accessibility

- **KBD-15** `SHOULD` Every interactive control is reachable with Tab and operable with Space or Enter. Toolbar buttons are the exception: they stay out of the Tab order (see Toolbars and tooltips).
  - *Check (by hand):* Tab through every window and dialog without the mouse.
  - *Reference:* Editora; `FocusRingFxTest` holds that every control family draws the focus ring when Tab reaches it.
- **KBD-16** `SHOULD` The keyboard focus ring is one layer of `:focus-visible` rules at the end of `app.css` .
  - *Check (test):* an FX test tabs to each control family and asserts the ring.
  - *Reference:* Editora `app.css` (*Keyboard focus*, one layer for every control), `FocusRingFxTest` .
- **KBD-17** `MUST` Icon-only buttons are built by one helper that sets both the tooltip and the accessible text.
  - *Check (code):* no icon-only `Button` is built outside the helper.
  - *Reference:* Editora `ui/Icons.java` (`Icons.button`). Editora's main toolbar doesn't yet: `WindowToolbar.setupButton` and `ToolbarCoordinator.buildExtraButton` set the tooltip but no accessible text.
- **KBD-18** `SHOULD` On/off rows use a keyboard-operable switch with the `TOGGLE_BUTTON` role.
  - *Why:* a bare AtlantaFX `ToggleSwitch` is mouse-only.
  - *Check (code):* no `new ToggleSwitch` outside the switch class.
  - *Reference:* Editora `ui/SettingSwitch.java` .
- **KBD-20** `SHOULD` Rows wrap instead of truncating to "...".
  - *Check (by hand):* run the app in German at 1024 px wide; no test catches a cut row.
  - *Reference:* Editora `messages_de.properties` .

Readable text and its contrast are one rule, ICON-09 on Icons and color.

Window shell



Editora's Welcome page: Start actions with their chords, Recent Projects, Recent Files and Customize.

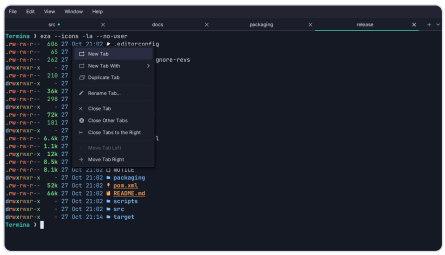
- `WIN-01` `MUST` A status bar is always present (it can be toggled). Its left side is the *echo area*: one line of feedback for the last action, errors in red and warnings in amber. Clicking it opens the Message Log.
  - *Check (code):* the echo label switches between an error and a warning style class, and its click handler runs the Message Log command.
  - *Reference:* Editora `ui/StatusBar.java` (`status-echo-error` , `status-echo-warn` ; a click runs `view.messageLog`) and the Message Log template.
- `WIN-02` `MUST` Light and dark AtlantaFX themes and a "follow the system" choice, switchable at run time without a restart. Semantic colors: amber is not durable yet (unsaved, read-only), red is broken, green is verified or added. See Icons and color.
  - *Check (code):* one place applies the app theme to every open window, and the theme list has a system choice that follows the OS color scheme as it changes.
  - *Reference:* Editora `ui/Themes.java` (`applyAppTheme`) switches at run time and `ui/WindowColorScheme.java` tells every window's frame which scheme it is in; Mirada `ui/Theme.java` (`ThemeChoice.SYSTEM`) and nux `ui/Theme.java` (`THEME_SYSTEM` , the default) follow the system. Editora, Torrenta, Termina and Modula switch live but have no system choice yet.
- `WIN-03` `SHOULD` The title bar uses JavaFX 27's `HeaderBar` with `StageStyle.EXTENDED` to carry the menu or tabs, with a decorated-window fallback, and `useSystemMenuBar` on macOS.
  - *Check (code):* the main stage is created `StageStyle.EXTENDED` with a `HeaderBar` , a setting falls back to a decorated window, and the menu bar sets `useSystemMenuBar` on macOS.
  - *Reference:* Editora `ui/ExtendedWindow.java` , `ui/MainMenuBar.java` .
- `WIN-04` `SHOULD` Native dialogs go through one helper that attaches the app stylesheet and the UI font.
  - *Why:* a dialog is its own scene and inherits neither.
  - *Check (code):* every `Alert` and `Dialog` the app builds passes through the helper.
  - *Reference:* Editora `ui/Dialogs.java` (`Dialogs.styled`).
- `WIN-05` `MUST` A destructive action asks first, and says what it does. Removing, deleting, discarding or overwriting something the user made goes through one confirm helper: the question names the thing ("Remove 3 torrents?"), the confirm button names the action (Remove, Discard, Overwrite), never OK or Yes, and Escape cancels. Where the action can be undone instead, the undo is better than the question. See the Confirm dialog template.
  - *Check (code):* each destructive action calls the one confirm helper before it changes anything, or records an undo.
  - *Reference:* Torrenta `RemoveConfirm` and `UndoStack` ; Editora's save decisions (`ui/CloseCoordinator.java`); Termina's and Editora's reopen-closed-tab.
- `WIN-06` `SHOULD` A Welcome page when there is nothing to restore: Start actions (which run commands), Recent items, a Customize block (keymap and theme) and Help links. See the Welcome template.
  - *Check (by hand):* start the app with no session to restore.
  - *Reference:* Editora `ui/WelcomePane.java` .
- `WIN-07` `MUST` About shows the version, the build, the Pauta edition the app implements and its conformance, the runtime (Java and JavaFX versions), the license, links, and the result of the last update check. See the About template.
  - *Check (code):* the About window reads the version, build and edition from `AppInfo` , the conformance from the conformance helper (Conformance), and shows the update checker's last answer.
  - *Reference:* Editora `ui/AboutDialog.java` . No app names its Pauta edition yet.
- `WIN-08` `SHOULD` Window size, position, maximized state and open items are restored on the next launch; `--no-session` skips the restore.
  - *Check (test):* a round-trip test of the saved workspace state, and a test that `--no-session` is honored.

- *Reference:* Editora `config/WorkspaceState.java` (`windowX` … `windowMaximized`), `ui/WindowSessionCoordinator.java`, `App.noSessionFlag`; tests `WorkspaceStateV29Test`, `AppArgsTest`, `NoSessionStartupFxTest`.

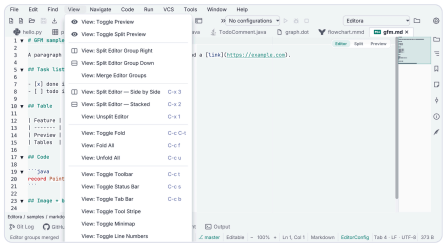
- **WIN-09** `SHOULD` UI and content zoom from the keyboard (`Ctrl/Cmd +`, `-`, `0`), kept across launches.
  - *Check (code):* zoom in, out and reset are commands bound in every keymap, and the zoom level is a saved setting.
  - *Reference:* Editora zooms content only: `view.textZoomIn`, `view.textZoomOut` and `view.textZoomReset`, kept in `Settings.fontZoom`.
- **WIN-10** `MAY` Modes: Zen (no chrome), Simple (fewer menus for new users), Expert.
  - *Check (code):* each mode is a command, and one truth table decides which chrome each mode shows.
  - *Reference:* Editora `ui/WindowChromeCoordinator.java` (`toggleZen`, `applySimpleMode`) over the truth table in `ui/Chrome.java`.



Mirada's HeaderBar title bar carries the file name and folder; the info panel and thumbnails are commands too.



Termina puts its tabs in the title bar with JavaFX 27's HeaderBar.



Every menu item is a command and shows its chord.

Icons and color

One typeface pair, one palette, one color vocabulary and one way of drawing icons, so a tab, a status segment, a tree row and a Settings row say the same thing in the same way in every app. Reference: Editora's `styles/ui-font.css` and `ui/Fonts.java` (type), `styles/atlantafx-themes/editora-light.css` and `editora-dark.css` (the Caret & Ink palette, which Torrenta and Termina vendor), the `.root` block at the top of `styles/app.css` (the state vocabulary), `ui/Themes.java` and `ui/EditorThemes.java` (themes), `ui/Icons.java`, `ui/FileIcons.java`, `ui/MenuBarIcons.java` and `editor/MenuIcons.java` (icons), and `StyleContrastTest`, which pins the color rules.

Type

Inter speaks, JetBrains Mono works. Inter is the chrome: menus, buttons, settings, pickers, status bar. The monospace face is for text that is code or output: the editor, a terminal, the environment block in About, log timestamps.

Property	Value	Reference
UI family	Inter, in four faces, bundled in the app's resources and registered before any CSS is applied	`Fonts.UI`, `Fonts.loadForFirstPaint`, `resources/com/editora/fonts/inter/`
Root size	14 px, the AtlantaFX theme's root size; popups pin absolute pixels where an owner's font would leak (see Menus)	`MainMenuBar.UI_FONT_SIZE`
How it is attached	A scene stylesheet (`ui-font.css`, one `.root` rule whose `-fx-font-family` names Inter) on every window's scene, dialogs and popups included	`App.installUiFont`
Monospace	JetBrains Mono by default; Cascadia Code, Fira Code, IBM Plex Mono and Source Code Pro are bundled as editor choices	`Fonts.DEFAULT`, `Fonts.BUNDLED`

- **`ICON-01`** `MUST` Inter is attached by a *scene* stylesheet on every window's scene, dialogs, popups and tooltips included, not by a rule in the app stylesheet.
  - *Why:* dialogs, the Settings window, the palette, context menus and tooltips each live in a scene of their own that does not share `app.css`, and a scene sheet is re-applied cleanly when the theme switches at run time (`setUserAgentStylesheet`). AtlantaFX neither bundles nor names Inter, and JavaFX does not fall through a font-family list, so without the sheet the whole interface renders in the platform's "System" face.
  - *Check (code):* a listener on `Window.getWindows()` adds `ui-font.css` to each window's scene, and to a new scene when the window's scene changes; no `.root` rule in `app.css` sets `-fx-font-family`.
  - *Reference:* Editora `App.installUiFont`, `styles/ui-font.css`. Termina and Torrenta carry the same sheet.
- **`ICON-02`** `MUST` The app bundles Inter's four faces and registers them before any CSS is applied; it never maps the platform's own UI font.
  - *Why:* an earlier macOS-only sheet used San Francisco, which JavaFX faux-bolds; Inter has a real bold on every platform.
  - *Check (code):* `Inter-Regular`, `Inter-Bold`, `Inter-Italic` and `Inter-BoldItalic` `.ttf` are under the app's `fonts/inter/` with their `OFL.txt`, and `Font.loadFont` runs on them before the first window is shown.
  - *Reference:* Editora `src/main/resources/com/editora/fonts/inter/`, `ui/Fonts.java` (`loadForFirstPaint`, called from `App`). Termina and Torrenta bundle the same four faces.

Modula is the stated exception to ICON-01 and ICON-02: IBM Plex Mono everywhere, because every reading on a radio is a number (see *Before adding a setting* in Settings).

The palette: Caret & Ink

The flagship theme pair, "Editora Light" and "Editora Dark", is authored (not vendored) and derived from Editora's icon: a **caret** teal accent, **ink** navy as the text in light and the surface in dark, **porcelain** and **abyss** grounds, and **brace** periwinkle reserved for the keyboard. Amber, red and green are semantic and never decorate.

	LIGHT	DARK	TOKEN
Ground	#ffffff	#1b1e2a	-color-bg-default
Porcelain / Abyss	#f4f5f9	#12141d	-color-bg-subtle / -color-dark
Ink	#1b1e2a	#e8eaf3	-color-fg-default
Muted ink	#575d6e	#9ba1b5	-color-fg-muted
Subtle ink	#8b91a3	#6c7288	-color-fg-subtle
Caret	#0e8577	#43dec5	-color-accent-emphasis
Caret, as text	#0d786b	#43dec5	-color-accent-fg
Amber	#8d6309	#e3a93c	-color-warning-fg
Red	#c03434	#ee7a6e	-color-danger-fg
Green	#2c784c	#5bc98a	-color-success-fg
Brace	#3c4380	#b8bee5	-state-key (a ladder of the ground)

- **ICON-04** `SHOULD` A theme file follows the vendored AtlantaFX community themes: a `.root` block of `-color-*` tokens, then component rules that reference only those tokens.
  - *Check (code):* the file's first rule is `.root`, and no rule after it holds a `#` hex.
  - *Reference:* Editora `styles/atlantafx-themes/editora-light.css`, `editora-dark.css`.
- **ICON-05** `SHOULD` A new app ships the Caret & Ink pair as its default theme, with `editora-light.css` and `editora-dark.css` copied unchanged into its own `styles/`.
  - *Why:* the palette is the family's, not the editor's.
  - *Check (code):* the app's two files are identical to Editora's.
  - *Reference:* Torrenta and Termina `styles/`. Both copies of `editora-dark.css` match Editora's; both copies of `editora-light.css` still have the earlier `-color-accent-fg`, `-color-warning-fg` and `-color-success-fg`, which Editora has since darkened.
- **ICON-06** `SHOULD` Brace (`-state-key`) is a `ladder()` over `-color-bg-default` that picks a deep ink on a light ground and a lighter periwinkle the darker the ground gets, and reads at 4.5:1 or more on every bundled theme's default and subtle surfaces.
  - *Why:* Brace has no AtlantaFX role, so it has to be a literal.
  - *Check (test):* a contrast test resolves the ladder against every bundled theme's grounds.
  - *Reference:* Editora `app.css` (`.root`), `StyleContrastTest.theKeyboardPeriwinkleReadsOnEveryBundledAppTheme`.

State colors

A color is a meaning, named once on `.root` and referenced everywhere. The same state wears the same color in a tab, a status segment, a tree row, a picker and a Settings row.

Token	Chains to	Meaning	Where it appears
`-state-amber`	`-color-warning-fg`	Not durable yet	An unsaved tab, a read-only file, a narrowed view, a snapshot build
`-state-red`	`-color-danger-fg`	Broken	Errors in the echo area and the Message Log, breakpoints, a recording
`-state-green`	`-color-success-fg`	Verified	Found, added, passing, running
`-state-accent`	`-color-accent-fg`	The app is offering something	An update, a link, the current branch, a selected row's bar
`-state-olive`, `-state-violet`	A ladder of the ground	Git only: untracked, renamed	The Project tree and tabs, the Map's structure columns
`-state-key`	A ladder of the ground	The keyboard, and nothing else	Chord chips, the menu chord column, brace matching
`-state-sel`	`-color-accent-subtle`	The selected overlay row	The palette, pickers, the Message Log (about a 13 % accent wash)

Each of the first four also has a `-bg` (the role's `-subtle`) and a `-hover` (the role's `-muted`) variant, so a pill, a row tint and a hover state come from the same place. `.state-pill` is the shape (bold, radius 999, padding 1 × 8); the color variant beside it carries the meaning.

- **ICON-07** `MUST` Each state color is named once, as a `-state-*` token on `.root` that chains to one of the theme's roles (or a `ladder()` of its ground), and component rules use the token.
  - *Check (code):* the `-state-*` tokens are declared only in the `.root` block at the top of `app.css`, each chaining to a `-color-*` role or a `ladder()`.

- *Reference:* Editora `styles/app.css` (`.root`).

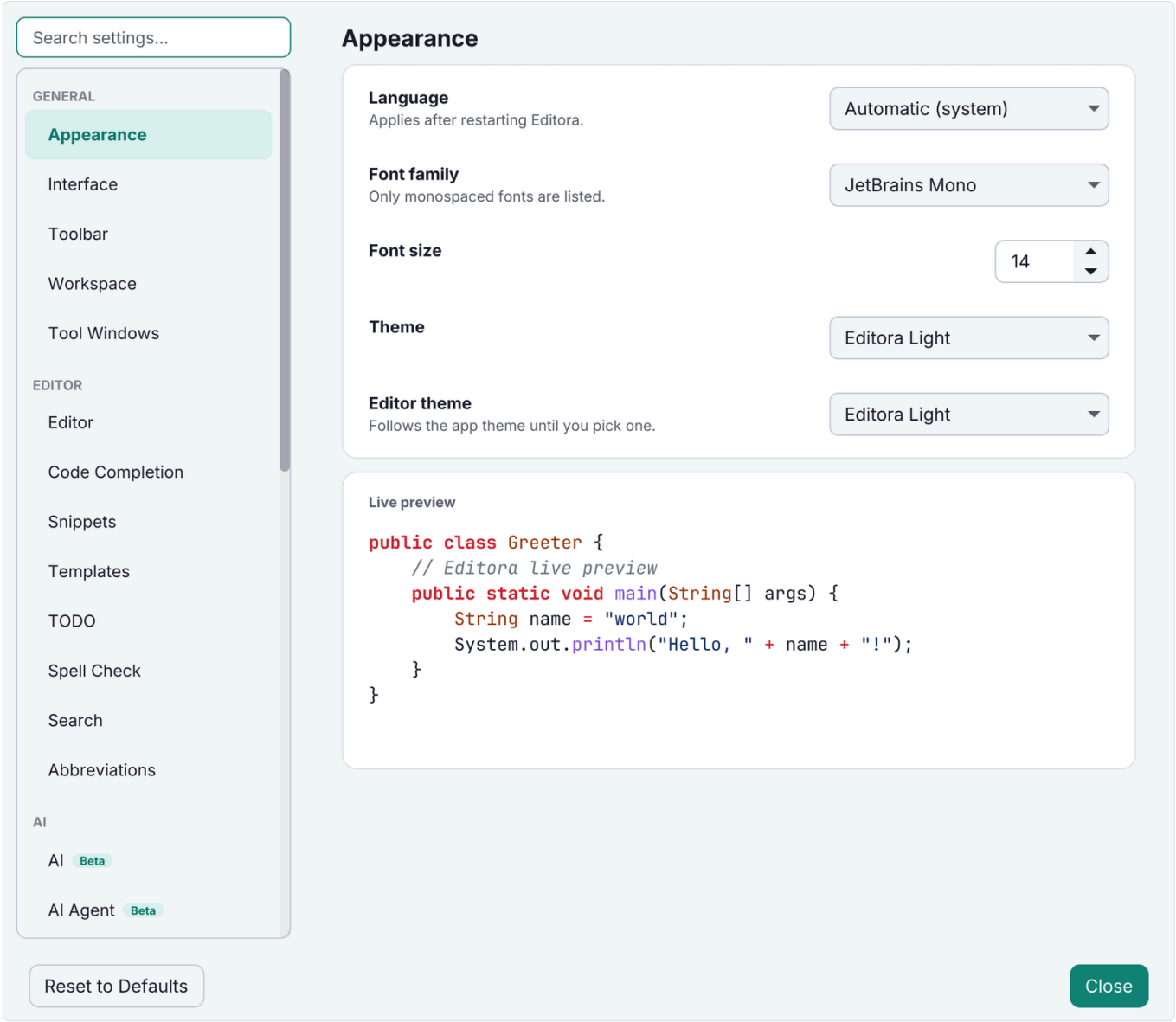
- **ICON-08** `MUST` No fixed hex in a component rule.
  - *Why:* a hex suits one theme; the roles track every theme for free.
  - *Check (code):* outside the `.root` token block, `app.css` holds no `#` hex. A test fails a severity or debug color that is not a state token.
  - *Reference:* Editora `StyleContrastTest.severityAndDebugColoursComeFromTheStateTokens` . Editora doesn't yet pass the whole check: `.breakpoint-marker` , `.exec-arrow` , `.view-mode-bar` , `.recent-remove` , `.personal-note-tooltip-content` , the fold-gutter markers and the default editor and Markdown preview palettes in `app.css` still carry fixed hexes.
- **ICON-09** `SHOULD` Readable text uses `-color-fg-muted` or stronger, which is 4.5:1 on the default, subtle and overlay grounds in both flagship themes: a line of UI text is `-color-fg-default` and secondary text `-color-fg-muted` . `-color-fg-subtle` (about 3:1) is for disabled or decorative text only.
  - *Check (test):* a contrast test computes muted ink on each ground, and fails a readable selector set in `-color-fg-subtle` .
  - *Reference:* Editora `StyleContrastTest.textThatMustBeReadUsesTheMutedRoleNotSubtle` .
- **ICON-10** `MUST` Tinted text (`-color-<role>-fg`) reaches 4.5:1 on its own subtle fill and on both grounds.
  - *Check (test):* a contrast test computes each role's `-fg` on its `-subtle` , on `-color-bg-subtle` and on `-color-bg-default` , in both flagship themes.
  - *Reference:* Editora `StyleContrastTest.lightTintedTextReachesAaOnItsOwnGroundAndOnTheSubtleSurface` , `darkTintedTextStaysAa` .
- **ICON-11** `MUST` Ink on an emphasis fill is `-color-fg-emphasis` , never white.
  - *Why:* white on the dark accent is 1.68:1.
  - *Check (test):* a contrast test computes `-color-fg-emphasis` on each role's `-emphasis` , and the rules that paint on an emphasis fill use it.
  - *Reference:* Editora `StyleContrastTest.inkOnAnEmphasisFillIsTheThemesOnEmphasisColour` .
- **ICON-12** `MUST` One focus ring, `-focus-ring: -color-accent-emphasis` , drawn by the `:focus-visible` block at the end of `app.css` , at 3:1 or better on every surface (see Keyboard first).
  - *Check (test):* a test reads `-focus-ring` from `.root` and computes it on the default, subtle, inset and overlay grounds.
  - *Reference:* Editora `app.css` (*Keyboard focus*), `StyleContrastTest.theFocusRingIsAtLeastThreeToOneOnEverySurface` .
- **ICON-13** `SHOULD` A brand hue the theme has no role for (periwinkle, olive, violet) is a `ladder()` of `-color-bg-default` , never a fixed pair.
  - *Why:* the old fixed light hexes read at 3.2:1 on Editora Dark.
  - *Check (test):* a contrast test resolves each ladder on every bundled theme's ground.
  - *Reference:* Editora `app.css` (`-state-olive` , `-state-violet` , `-state-key`), `StyleContrastTest.theGitOliveAndVioletFollowTheAppThemesGround` .
- **ICON-14** `SHOULD` A pure test reads the stylesheets and does the WCAG arithmetic.
  - *Why:* a contrast regression is invisible in a code diff.
  - *Check (code):* a test class reads the app's `.css` resources and computes contrast ratios without starting the toolkit.
  - *Reference:* Editora `StyleContrastTest` . Mirada's `AccentTest` checks that the accent reads as text on both grounds; Termina's `ThemeTest` keeps every chromatic color of its terminal schemes legible.

Themes

- **ICON-15** `MUST` The app has light and dark themes and switches between them at run time, without a restart (see Window shell).
  - *Check (code):* picking a theme calls `Application.setUserAgentStylesheet` in the running app.
  - *Reference:* Editora `ui/Themes.java` (`applyAppTheme`).
- **ICON-16** `MUST` A "follow the system" choice applies the light or dark theme to match the system's color scheme, and changes with it.
  - *Check (code):* the theme setting has a system value, and a listener on `Platform.getPreferences().colorSchemeProperty()` re-applies the theme.
  - *Reference:* Mirada `settings/Settings.java` (`ThemeChoice.SYSTEM`), `ui/Theme.java` ; nux `config/Advanced.java` (`THEME_SYSTEM`), `ui/Theme.java` . Editora, Torrenta, Termina and Modula don't yet: their pickers list named themes only.
- **ICON-17** `MUST` A theme name the app does not know falls back to the first-run default theme.
  - *Check (code):* the lookup from a saved name to a theme ends in the default, not an exception.
  - *Reference:* Editora `Themes.themeFor` , falling back to `Themes.DEFAULT` (Editora Light).
- **ICON-18** `SHOULD` Semantic colors never decorate: a theme may change the teal, the ground and the ink, but amber still means "not durable yet", red "broken" and green "verified".
  - *Check (code):* every rule that uses `-state-amber` , `-state-red` or `-state-green` marks one of the meanings in the State colors table.
  - *Reference:* Editora `app.css` .

Editora lists 28 app themes: the Caret & Ink pair, AtlantaFX's Primer, Nord, Cupertino and Dracula, and 19 community themes (Army, Autumn, Blacky, Blue, Browny, Fall, Navy, News, Spring, Summer, Winter, Yacht, several with light and dark variants) vendored unmodified from the MIT-licensed dlsc `atlantafx-themes` collection. User themes under `<config>/themes/` join the list (`Themes.names()`).

Editor themes are separate (`EditorThemes`). The flagship pair carries its own editor stylesheet; every community theme is served by one `adaptive` editor theme whose syntax palette is expressed in the theme's own `-color-*` tokens, so it follows whatever ground the theme has.



Settings: the theme picker with the Caret & Ink pair first, and the UI and editor fonts.

Icons

Every icon is an SVG path drawn in code, colored by the stylesheet so it follows the theme, and returned as a fresh node per call (a JavaFX node has one parent). There are two families, mid-migration to the UI Kit, and they are not interchangeable.



Property	Line family	Filled family	Reference
Source	The UI Kit's 16-unit outline glyphs, drawn for the kit	Material Design 24 dp single-path glyphs (Apache-2.0), plus Simple Icons brand marks (CC0)	`Icons.line`, `Icons.of`, `NOTICE`
Style class	`icon-line`	`toolbar-icon`	`app.css`
Color	`-fx-stroke: -color-fg-muted`, stroke width 1.3, round caps and joins, fill transparent	`-fx-fill: -color-fg-muted`	`.icon-line`, `.toolbar-icon`

Property	Line family	Filled family	Reference
Hover, disabled	`-color-fg-default` on hover, `-color-fg-subtle` disabled; every context mirrors the matching rule of the other family	The same	`.button:hover .icon-line`, `.button:hover .toolbar-icon`
Drawn size	16 × 1.2 = 19.2 px (`LINE_SCALE` = `ICON_SCALE` × 24 ÷ 16)	24 × 0.8 = 19.2 px (`ICON_SCALE`)	`Icons`
Used for	The chrome the kit draws: toolbar, tool stripe, tabs, and every action that has a kit glyph	Everything the kit specifies no glyph for: file types, context-menu rows, build-tool and browser brand marks	`Icons` , `FileIcons`

- **ICON-19** `MUST` A glyph node carries one family's style class, `icon-line` or `toolbar-icon` , never both.
  - *Why:* a line glyph is colored by its stroke, and any `X .toolbar-icon` fill rule (there are dozens) would paint its outline solid. A glyph that moves family moves its CSS with it.
  - *Check (test):* a test builds every line glyph and fails when one also carries `toolbar-icon` .
  - *Reference:* Editora `ui/Icons.java` (`line` , `of`), `MenuIconFamilyFxTest.aLineGlyphIsNeverAlsoTaggedAsAFilledOne` .
- **ICON-20** `MUST` The same action wears the same glyph wherever it is reached: Save is one floppy on the toolbar, in the File menu and in a tab's menu.
  - *Check (test):* a test compares the path data and style class of each action's glyph across the icon classes.
  - *Reference:* Editora `ui/MenuBarIcons.java` reuses `Icons` ; `editor/MenuIcons.java` (the editor package may not depend on `ui`) copies each twin's family and path data verbatim, held by `MenuIconFamilyFxTest.everySharedActionUsesTheSameGlyphInBothIconSets` .
- **ICON-21** `MUST` An icon-only control has a name: it is built through `Icons.button` or `Icons.toolbarButton` , which set the tooltip and the accessible text together (see Keyboard first).
  - *Check (code):* no icon-only `Button` is given a graphic outside those helpers without `Icons.name` .
  - *Reference:* Editora `ui/Icons.java` . Editora doesn't yet pass the check on its main toolbar: `WindowToolbar.setupButton` and `ToolbarCoordinator.buildExtraButton` set only the tooltip (see Toolbars and tooltips).
- **ICON-23** `SHOULD` A file's icon comes from a pure function from file name to icon key, unit-tested without the toolkit, and any type it does not know gets the generic document glyph, so a new type never regresses the look.
  - *Check (test):* a unit test resolves names to keys, an unknown one to the generic key.
  - *Reference:* Editora `ui/FileIcons.java` : `iconKeyFor(name)` resolves the key, `forFileName` turns it into a node; brand logos from Simple Icons, category glyphs (image, archive, PDF, table, lock, JSON, settings) from Material. `FileIconsTest.unknownAndNullFallBackToGeneric` .
- **ICON-24** `MUST` Every glyph set and font the app ships is credited in `NOTICE` with its license.
  - *Check (code):* each icon source named in the icon classes and each family under `fonts/` has an entry in `NOTICE` .
  - *Reference:* Editora `NOTICE` : Material Design Icons (Apache-2.0), Simple Icons (CC0; the logos stay their owners' trademarks), Inter and the monospace faces (OFL), and the glyph sets inside the bundled Symbols Nerd Font, Octicons and Seti UI (MIT) among them.

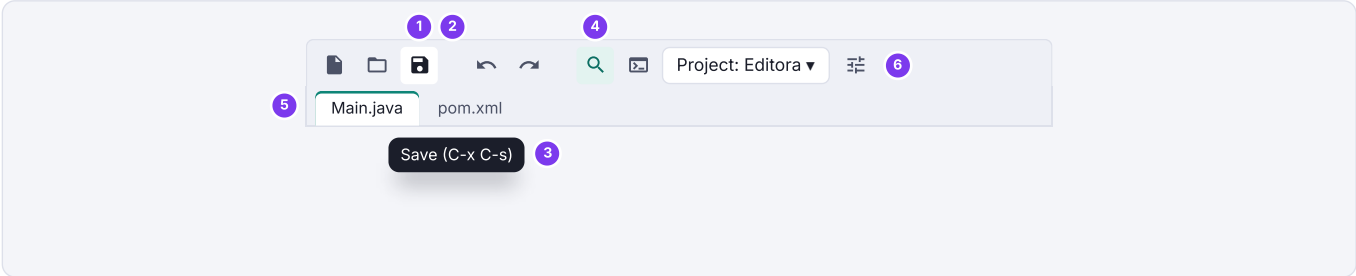
In the apps

App	Icons	Size
Editora	Both families, `ui/Icons` , `FileIcons` , `MenuBarIcons` , `editor/MenuIcons`	19.2 px on the toolbar and in menus
Torrenta	Both families, copied from Editora as `Icons.Glyph` , which knows its box and resizes with `at(px)` ; the toolbar is all line glyphs (stroke 1.5, `-torrenta-icon`)	17 px toolbar, 16 px menu and tab (`MENU_SIZE`), 13 px sidebar
Termina	Material filled, `ui/MenuIcons` , colored by `.menu-icon` (`-color-fg-muted` ; `-color-fg-default` focused; subtle and half opacity disabled)	24 × 0.7 = 16.8 px, menus only
Mirada	Hand-drawn stroked paths on a 24-unit grid, two-unit strokes, round caps, in GNOME's symbolic style, so nothing carries a license; `.icon` strokes `-color-fg-default`	16 px, centered in a 16 px box
Modula	`Glyphs` : the transport symbols (tune, seek, star) as shapes, because the characters are missing from most faces and render differently on every platform	11 px
nux	No in-window icons; the tray icon is rendered as a bitmap by severity (`TrayIconRenderer`)	Tray only

Toolbars and tooltips

A toolbar is the fast path to commands that are also in the menus and the Command Palette: nothing lives only on it. Every button is a command, so its icon, title and chord come from the registry and the active keymap. Reference: Editora's `ui/Icons.java` (`button` , `toolbarButton` , `name`), `ui/ToolbarCoordinator.java` (the customizable bar), `ui/WindowChromeCoordinator.java` (`collapseToolbarSeparators`), `ui/ToolWindowManager.java` (the tool window stripes) and the `.tool-bar` , `.toolbar-button` , `.tool-stripe-button` and `.tooltip` rules in `app.css` .

Anatomy



1. **Icon button**, 28 px: padding 3 × 5, radius 4, no border, the glyph in `-color-fg-muted` and `-color-fg-default` under the pointer. It has a name even though it shows none: the same string is its tooltip and its accessible text.
2. **Group gap.** A vertical `Separator` node whose line is transparent: its padding is what separates the groups. No drawn rule.
3. **Tooltip.** The command's title and its chord in the active keymap, "Save (C-x C-s)". A popup window with the menus' radius 8.
4. **Open toggle.** A button whose overlay is showing (Find, the palette, a split) wears `:open` : an accent-subtle fill and the glyph in the accent color. Simple mode's own button does the same while the mode is on.
5. **One band.** The menu bar, the toolbar and the tab strip sit on `-color-bg-subtle` with no hairline between them; spacing does the grouping.
6. **Pinned tail.** The project picker, About and Quit live in their own container beside the `ToolBar` , so a narrow window overflows the icons (which are all in the menus too) and never the tail.

Spec

Property	Value	Reference
Button	28 px high: padding 3 × 5, radius 4, style classes `button-icon flat toolbar-button` ; every control on the bar is pinned to 28 px, combos and menu buttons included	`.toolbar-button` , `.tool-bar .combo-box-base`
Bar	Padding 5 1 2 6 (asymmetric: the tab strip below adds its own inset); 18 px font for the bar's own labels, controls inside it at 14 px; `-color-bg-subtle` , no rule above or below	`.tool-bar` , `.menu-bar, .tool-bar`
Icon	The command's glyph, 17 px drawn (Torrenta) or a kit glyph at the toolbar scale (Editora); `-color-fg-muted` , `-color-fg-default` on hover, `-color-accent-fg` while `:open`	`Icons` , `.toolbar-icon` , `.icon-line` , `.button:open`
Group gap	A vertical `Separator` with its line transparent; its 0.75 em padding is the gap	`.tool-bar .separator:vertical > .line`
Tail	Its own container (`.toolbar-tail`), same insets and font as the bar, never overflowed	`ToolbarOverflowFxTest`
Tooltip	`<title> (<chord>)` , the chord from `KeymapManager.displayChord` ; only the title when unbound; radius 8 like the menus	`ToolbarCoordinator.buildExtraButton` , `ToolWindowManager.tooltipFor` , `.tooltip`
Stripe button	A toolbar button on the tool window rail: min 24 px, padding 9 × 5, radius 6, 8 px apart on a 30 px rail; the same muted, hover and `:open` colors; the bottom stripe adds a 14 px label	`.tool-stripe-button` , `ToolWindowManager.register`
Customize mode	The bar tinted `-color-accent-subtle` , a move cursor, the dragged button at 0.4 opacity, a 2 px accent insertion line on the drop target's edge	`.tool-bar.toolbar-customizing` , `.toolbar-drop-before`

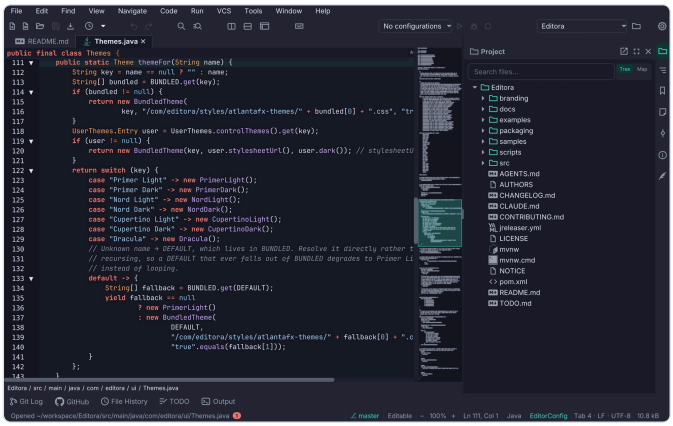
Rules

- **TBAR-01** `MUST` Every icon-only button has a name: one helper sets its tooltip and its accessible text from the same string.
  - *Why:* a hand-rolled button gets the tooltip and forgets the name, which is how Editora once had seven copies of the helper drifting apart, and how a screen reader comes to announce "button".
  - *Check (code):* every icon-only `Button` is built by `Icons.button` or `Icons.toolbarButton` , or passed to `Icons.name` ; none calls `setTooltip` without `setAccessibleText` .
  - *Reference:* Editora `ui/Icons.java` (`button` , `toolbarButton` , `name`); Torrenta's copy of all three. nux sets both by hand on its "?" button for the same reason. Editora doesn't yet on its main toolbar: `WindowToolbar.setupButton` and `ToolbarCoordinator.buildExtraButton` set the tooltip only.

- **TBAR-02** `MUST` Toolbar buttons stay out of the Tab order: the command, the palette and the menus reach the action, and Tab is for the content. The keyboard page has the general focus rules.
  - *Check (code):* every button on a toolbar is built by `Icons.toolbarButton` or calls `setFocusTraversable(false)`.
  - *Reference:* Editora `Icons.toolbarButton`, used by the tool window toolbars (Git, Diff, Markdown lint); Termina's search bar buttons; Mirada's header buttons. Editora doesn't yet on its main toolbar: the buttons built by `WindowToolbar.setupButton` and `ToolbarCoordinator.buildExtraButton`, and the stripe buttons, take focus.
- **TBAR-03** `MUST` The tooltip names the command and its chord, read from the active keymap and never typed into a string: "Save (C-x C-s)". An unbound command shows its title alone.
  - *Check (code):* the tooltip is built from the command's title and the keymap's display chord for its id, and is rebuilt when the keymap changes.
  - *Reference:* Editora `WindowToolbar.setupButton` (`refreshToolbarTooltips`), `ToolbarCoordinator.buildExtraButton` and `ToolWindowManager.tooltipFor`; Torrenta's `toolbar.withChord`. Mirada's header buttons use the same "label (key)" form with the key written into the string.
- **TBAR-04** `SHOULD` Groups are separated by spacing, not rules: keep the `Separator` nodes (their padding is the gap) and make the line transparent.
  - *Check (code):* the stylesheet sets `.tool-bar .separator:vertical > .line` to a transparent border.
  - *Reference:* Editora `app.css`. Torrenta's separators still draw a line.
- **TBAR-05** `SHOULD` Hiding a button group (Simple mode, a build tool that is not detected) hides the leading, trailing and doubled separators it leaves, so no gap stands on its own, and shows them all again when nothing is hidden.
  - *Check (code):* every change to a toolbar item's visibility ends in one pass over the bar's separators.
  - *Reference:* Editora `WindowChromeCoordinator.collapseToolbarSeparators`.
- **TBAR-06** `SHOULD` One chrome band: the menu bar, the toolbar and the tab strip share `-color-bg-subtle` and draw no hairline between them; the breadcrumb, the bottom stripe and the status bar do the same at the bottom.
  - *Why:* three rules within forty pixels read as clutter, not structure.
  - *Check (by hand):* look at the top and the bottom of a window in a light and a dark theme for a line between the bands.
  - *Reference:* Editora `app.css` (`.menu-bar, .tool-bar`).
- **TBAR-07** `SHOULD` A toggle looks on while it is on: a button that opens an overlay (Find, the palette, a split) and the Simple mode button carry the `:open` pseudo-class while their state holds, so the way back out does not look like the way in.
  - *Check (code):* each such button sets a pseudo-class from its state, and the stylesheet gives that state a fill.
  - *Reference:* Editora `MainController.OPEN`, `WindowChromeCoordinator` (Simple mode), `.button:open` in `app.css`. Torrenta uses `:on` for Simple mode and `:active` for the alternative speed limit.
- **TBAR-08** `MUST` The toolbar never holds the only way to an action: every icon in the customizable cluster is also in the menus and the palette.
  - *Why:* that is what makes it safe to hide icons in a narrow window and to let the user remove them.
  - *Check (code):* every toolbar catalog item runs a registered command that a menu also lists.
  - *Reference:* Editora `toolbar/ToolbarCatalog.java`: each item carries the `commandId` it runs; the only items without one are declared special widgets, such as the Recent menu button (`ToolbarCatalogTest`).
- **TBAR-09** `MAY` The toolbar is customizable: the user reorders, removes and restores its buttons, and the layout is saved in Settings and shared by every window.
  - *Check (test):* FX tests drive customize mode, reorder and removal.
  - *Reference:* Editora `ui/ToolbarCoordinator.java`, `ToolbarCustomizeModeFxTest`, `ToolbarCustomizationFxTest`. The cluster is a list of catalog ids in Settings (`toolbarLayout`), sanitized on read. The bar's right-click menu offers *Customize Toolbar...* (or *Done*), *Configure...* (the Settings page) and *Restore Default*. In customize mode the bar tints, buttons drag to reorder with an accent insertion line and drag off the bar to remove; Escape leaves the mode.
- **TBAR-10** `MUST` The bar's right-click menu is one menu, reused across right-clicks, so showing it again moves it, and it closes on a press anywhere, the toolbar included.
  - *Check (test):* an FX test right-clicks the bar twice and finds one menu, then presses on the bar and elsewhere in the window and finds it closed.
  - *Reference:* Editora `ToolbarCoordinator`, `ContextMenuDismissal`, `ToolbarContextMenuDismissFxTest`. See Menus, whose rules on icons, chords and the 14 px font (MENU-16) hold for this menu too.
- **TBAR-12** `SHOULD` Tool window stripes are toolbars: a stripe button is built like a toolbar button, with the chord in its tooltip, accessible text bound to the window's title, and a right-click menu with *Open in Split* and *Hide*.
  - *Check (code):* the stripe button's tooltip comes from the same title-and-chord format, and its `accessibleTextProperty` is bound to the tool window's title.
  - *Reference:* Editora `ToolWindowManager.register`, `tooltipFor`.
- **TBAR-13** `SHOULD` The right stripe's first icon lines up with the editor's first line of text, not with the tab strip.
  - *Check (test):* an FX test compares the first stripe button's top with the editor's first line.
  - *Reference:* Editora `ToolWindowStripeAlignmentFxTest`.

In the apps

- **Editora**: the customizable cluster, the pinned tail, the stripes.
- **Torrenta**: the same `Icons.toolbarButton` and `Icons.name`, 32 px buttons, Simple mode as `:on`; its separators still draw a line.
- **Mirada**: a `HeaderBar` title bar with flat icon buttons around the file name, stroked symbolic icons in the text color, tooltips of the form "label (key)"; each button's accessible text is set by hand, and the buttons are out of the Tab order.
- **Termina**: no main toolbar (its tabs are in the title bar); the search bar's buttons are 24 × 22 flat icons with tooltips, out of the Tab order.



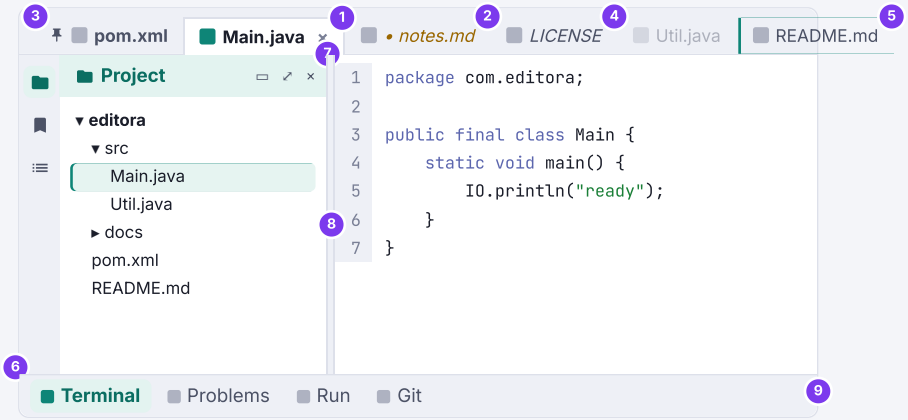
Editora's toolbar: grouped icons on the chrome band, the project picker and Quit pinned at the right.



Mirada's HeaderBar: flat icon buttons around the file name.

Tabs and tool panels

A window shows its documents in a **tab strip** and its tools in **panels** docked beside or under them. Both are driven by commands, both remember their state, and both draw their chrome as one band with the toolbar, not as boxes inside boxes. Reference: Editora's `ui/TabHeaders.java` (the tab header), `ui/ClosedTabs.java`, `ui/Switcher.java`, `ui/ToolWindowManager.java`, `ui/ToolWindowPanel.java` and the `.tab-pane`, `.tool-stripe` and `.tool-window` rules in `app.css`.

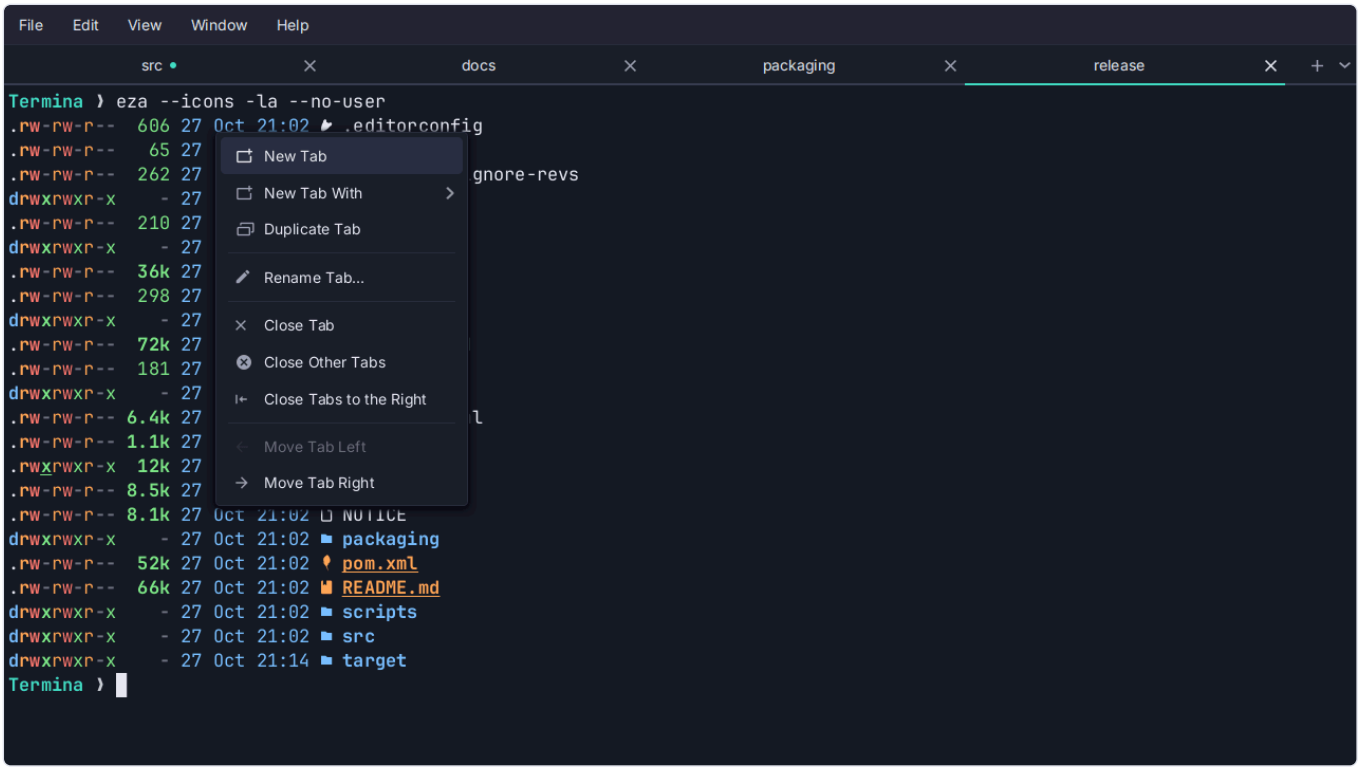


1. **Active tab.** Its fill is the editor's own surface (`-editor-surface`) with a border on three sides and none at the bottom, so the tab opens into the editor instead of sitting on it. Title bold in `-color-fg-default`; the file-type glyph tints accent.
2. **Unsaved tab.** A `•` before the name, amber (`-state-amber`, "not durable yet") and italic. Amber outranks the Git-status color: "you have not saved this" matters more than "this differs from HEAD".
3. **Pinned tab.** A pin glyph and a bold title, kept at the front of the strip in pin order. Every bulk close (Others, All, Unmodified, Left, Right) skips it.
4. **Read-only tab.** Muted and italic, echoing the status bar's Read-Only segment.
5. **Drag to reorder.** The header is the drag handle and a snapshot of the whole tab follows the pointer; the dragged tab dims to 40 % and a 2 px accent line marks the edge it would land on (left half of a tab: before it; right half: after).
6. **Tool stripe.** A 30 px rail of icon buttons, one per tool window, 8 px apart, on the subtle ground with no drawn edge. The open window's button is tinted; the one holding keyboard focus is tinted stronger. The first icon lines up with the editor's first line of text, not with the tab strip.
7. **Panel header.** The window's glyph and title (14 px bold, muted) with float, maximize and close; the same size and glyph as its stripe button, so the two name the same thing. On the subtle ground, with no rule under it; when the panel holds focus the band tints accent-subtle and the title turns accent.
8. **The divider is the edge.** The panel draws no frame: the SplitPane divider between it and the editor is the real, grabbable edge, and a hairline beside it would only double it. Its position is remembered.
9. **Bottom stripe.** Its buttons carry a 14 px label beside the glyph, side by side; the open one is accent and bold.

Tabs

- `TABS-01` `MUST` Every tab operation is a command, so it can be bound to a chord, is in the palette and has a menu row: close, close others, close all, close unmodified, close to the left and right, pin, reopen closed, next and previous. The tab's right-click menu lists the ones that act on a tab.
  - *Check (code):* each operation has a registered command id, and `TabContextMenu` lists the per-tab ones.
  - *Reference:* Editora `ui/WindowCommandRegistrar.java` (`buffer.close`, `buffer.closeOthers`, `buffer.closeAll`, `buffer.closeUnmodified`, `buffer.closeLeft`, `buffer.closeRight`, `buffer.togglePin`, `buffer.reopenClosed`, `buffer.next`), `ui/TabContextMenu.java`. Editora has no previous-tab command yet.
- `TABS-02` `MUST` The header is the title: the Tab's own text stays empty, and a graphic header carries the file-type glyph, the markers and the title in a `.tab-title` label. The tooltip is the full path.
  - *Why:* the graphic is what makes the header a drag handle and lets each state set its own color.
  - *Check (code):* every editor tab gets `setText("")` and a graphic with a `.tab-title` label; its tooltip is the file's path.
  - *Reference:* Editora `ui/TabHeaders.java`.
- `TABS-03` `MUST` The one header carries the states: a `•` prefix in amber italic for unsaved, a pin and bold for pinned, muted italic for read-only.
  - *Check (code):* `app.css` styles `.tab-header.dirty`, `.tab-header.pinned` and `.tab-header.read-only` `.tab-title`, and the header sets those classes.
  - *Reference:* Editora `ui/TabHeaders.java`, `app.css`; the Unsaved, Pinned and Read-only rows of the Spec table below.
- `TABS-04` `MUST` Bulk close skips pinned tabs: close others, close all, close unmodified and close to the left or right never close a pinned tab.
  - *Check (code):* `closeOtherTabs`, `closeAllTabs`, `closeUnmodifiedTabs` and `eligibleToLeft` / `eligibleToRight` all filter on the pinned set.
  - *Reference:* Editora `ui/MainController.java`.
- `TABS-05` `MUST` Pinning moves the tab to the end of the pinned group, unpinning to just after it.
  - *Check (code):* `togglePin` calls `moveToPinBoundary`.
  - *Reference:* Editora `ui/MainController.java` (`togglePin`), `ui/EditorArea.java` (`moveToPinBoundary`).
- `TABS-06` `MUST` Reopen Closed Tab brings back the files whose tabs this window closed, newest first, bounded to twenty, a file listed once, with the caret where it was. In a terminal app the *tab* comes back (its shells, their directories, the pane arrangement, the names the user gave them, its place in the strip) and

not the scrollback.
  - *Why:* the process is gone, and pretending otherwise would be worse than saying so.
  - *Check (test):* a unit test over the closed-tab list holds the order, the bound, the single entry per file and the caret.
  - *Reference:* Editora `ui/ClosedTabs.java`, `ClosedTabsTest`; `C-S-t` in the CUA keymap. Termina `ui/ClosedTabs.java`, on `Ctrl+Shift+Z` (`Cmd+Z` on macOS).
- **TABS-07** `SHOULD` Tabs reorder by dragging, with the feedback shown above: the dragged header dimmed and an accent line on the edge where it will land.
  - *Check (by hand):* drag a tab along the strip and watch the landing edge.
  - *Reference:* Editora `TabHeaders.enableTabDrag`; Termina `installTabDrag` and the pure `TabReorder.insertIndex`. In Termina a double-click on a tab renames it, and a pane dragged onto a tab joins it (the whole header outlined, `tab-drop-into`).
- **TABS-08** `SHOULD` A tab shows its file's Git status in the Project tree's colors: modified accent, added green, untracked olive, renamed violet, deleted muted, conflict red. A read-only tab still shows it; an unsaved one shows amber.
  - *Check (test):* a test gives a changed file the same style class the Project tree uses.
  - *Reference:* Editora `ui/TabGitStatus.java`, `TabGitStatusTest`.
- **TABS-09** `SHOULD` A switcher lists the open files in tab order: `C-x C-b` (Emacs) opens it, `C-n` / `C-p` or the arrows move, Enter or releasing Ctrl activates, `C-g` or Esc cancels, `C-d` or Backspace closes the highlighted file. Its width is fixed to the longest path on each open, so it never resizes while you move.
  - *Check (by hand):* open it with several tabs and move through it; the list keeps its width.
  - *Reference:* Editora `ui/Switcher.java`, `SwitcherFxTest`.
- **TABS-10** `SHOULD` The window title names the file, the project and the app, with a `●` prefix for an unsaved file.
  - *Check (code):* the code that sets the stage title adds all three and the marker.
  - *Reference:* Editora `MainController.updateWindowTitle`.
- **TABS-11** `SHOULD` Tabs sit in the title bar through JavaFX 27's `HeaderBar`; see Window shell.
  - *Check (by hand):* the tab strip shares the title bar's row.
  - *Reference:* Termina `ui/TerminalWindow.java`.
- **TABS-12** `SHOULD` A command hides the tab bar.
  - *Check (code):* a registered command toggles the tab header.
  - *Reference:* Editora `view.toggleTabBar`, `.tab-pane.no-tab-header` in `app.css`.



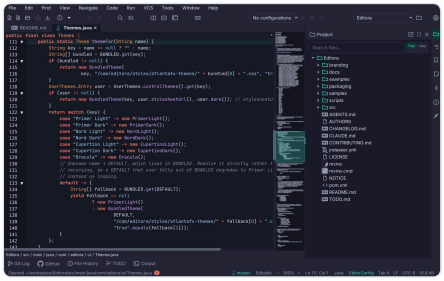
Termina's tabs in the title bar, with a tab's right-click menu.

Tool panels

- **TABS-13** `MUST` A command toggles each panel (`tool.<id>`: `M-1` … `M-0` in the Emacs keymap for ten of them), and the stripe button is the same command with a glyph.
  - *Check (code):* every tool window registers a `tool.<id>` command, and its stripe button runs it.
  - *Reference:* Editora `ui/ToolWindowManager.java`, `keymaps/emacs.json`.
- **TABS-14** `MUST` A panel's content is built on first use.
  - *Check (test):* an FX test holds that no closed tool window builds its panel before the first paint.
  - *Reference:* Editora `LazyToolWindowsFxTest`.
- **TABS-15** `SHOULD` Panel state is restored: which windows are open, on which side, in what order, hidden or shown, floating, and every divider position go to `workspace-state.json` and come back on the next launch.
  - *Check (code):* the workspace state holds each of those, and the manager reads them back at startup.
  - *Reference:* Editora `ui/ToolWindowManager.java`, `config/WorkspaceState.java`.

- **TABS-16** `SHOULD` Panels dock on three sides, left, right and bottom, and a side holds at most two windows stacked (Project over Structure, two consoles along the bottom).
  - *Why:* past that a side stops being a dock and becomes a list of slivers.
  - *Check (code):* the sides are an enum of three and the per-side limit is two.
  - *Reference:* Editora `ToolWindow.Side`, `ToolWindowManager.MAX_PER_SIDE`.
- **TABS-17** `SHOULD` A panel re-docks by dragging its stripe button to another stripe, with the same dim-and-accent-line feedback as a tab; an empty stripe appears as a dashed accent drop zone for the duration of the drag.
  - *Check (by hand):* drag a stripe button to the opposite side.
  - *Reference:* Editora `ui/ToolWindowDock.java`, `ToolWindowVisibilityTest` (an empty stripe shows while dragging).
- **TABS-18** `SHOULD` Panels float, maximize, resize and close from the keyboard, and at most one window is maximized at a time.
  - *Check (code):* the commands are registered and bound in the Emacs keymap.
  - *Reference:* Editora `view.floatToolWindow`, `view.maximizeToolWindow` (`C-x f`), `view.closeFocusedToolWindow` (`C-x 0`), the resize commands (`C-x S-.`, `C-x S-,`, `C-x S-6`).
- **TABS-19** `SHOULD` Closing the panel that holds keyboard focus returns focus to where it was in the editor.
  - *Check (code):* the manager keeps the last editor focus owner and refocuses it on close.
  - *Reference:* Editora `ToolWindowManager.focusEditor`, a weak reference to the last editor focus owner.
- **TABS-20** `SHOULD` Hidden is not unavailable: the user's show/hide choice for a stripe button is kept apart from a context-driven hide (the Commit window outside a Git repository), so the second never clobbers the first.
  - *Check (test):* a unit test holds that a button shows only when visible and available.
  - *Reference:* Editora `ui/ToolWindowVisibility.java`, `ToolWindowVisibilityTest`.
- **TABS-21** `SHOULD` A bottom window that is unusable at the default height asks for more on its first open; the user's own size wins afterwards.
  - *Check (test):* a test opens an output window taller while the others keep the side's size.
  - *Reference:* Editora `ToolWindow.setFirstOpenShare`, `ToolWindowFirstOpenShareTest`.
- **TABS-22** `SHOULD` A stripe button badges a spinner while its window has something under way.
  - *Check (test):* an FX test badges a busy window's stripe button without changing its size.
  - *Reference:* Editora `ToolWindow.busy`, `ToolWindowManagerFxTest`.
- **TABS-23** `SHOULD` Panels follow the active document where they should: the Git UI follows the active tab's repository, the status bar's environment segment the active `.http` file, Local History the file whose tab menu opened it.
  - *Check (by hand):* switch between tabs from two repositories with the Git window open.
  - *Reference:* Editora `ui/GitPathScope.java`.

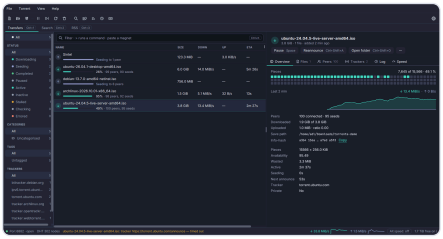
The other apps dock their panels the same way without a stripe. Torrenta's filter **sidebar** sits down the left or the right of the list or nowhere, kept as two settings keys so hiding it keeps its side (`SidebarPlace`); its inner edge drags to resize within readable bounds and a double-click resets it (`SidebarWidth`); the **detail pane** docks under the list in two columns or beside it in one, from the View menu (`DetailPane`). Mirada's info panel opens on the right and its thumbnail strip along the bottom or down the left side, each closed until its command asks for it (`ThumbnailStrip.place`). Termina splits a tab into **panes** instead: a pane is picked up with a drag while `Ctrl+Alt+Shift` (`Cmd+Option+Shift` on macOS) is held, since every smaller chord is spoken for (`PaneDrag`), and a maximized pane wears a "Pane 2 of 3" badge so the others are known to still be there.



Editora: the Project tool window docked left of the editor, its stripe on the window's edge.



Mirada: the info panel on the right and the thumbnail strip along the bottom, both commands.



Torrenta: the filter sidebar beside the list and the detail pane under it.

Spec

Property	Value	Reference
Tab strip	`-color-bg-subtle`, padding 3 10 0 10, tabs bottom-aligned so the active one touches the editor; no bottom rule	`.tab-pane > .tab-header-area`
Tab	Transparent, square corners, padding 2 11 3 11; hover `-color-bg-inset`	`.headers-region > .tab`
Active tab	Two background layers: `-color-border-default` outside, `-editor-surface` inset 1 1 0 1; title bold `-color-fg-default`; glyph `-color-accent-fg`	`.tab:selected`
Title	Inter 14 px, `-color-fg-muted`, in a graphic label; the full path as tooltip	`.tab .tab-title`, `TabHeaders.tooltip`
Unsaved	`•` prefix, `-state-amber`, italic	`.tab-header.dirty .tab-title`
Pinned	`Icons.pin()` in the header, bold; first in the strip	`.tab-header.pinned`, `togglePin`
Read-only	`-color-fg-muted`, italic	`.tab-header.read-only`

Property	Value	Reference
Drag	Dragged header at 0.4 opacity; 2 px `-color-accent-emphasis` on the landing edge	`.tab-dragging`, `.tab-drop-before`, `.tab-drop-after`
Closed tabs	Twenty, newest first, one entry per file, with the caret	`ClosedTabs.DEFAULT_CAPACITY`
Switcher	26 px rows, 12 visible, width fixed per open	`Switcher.CELL_HEIGHT`, `MAX_VISIBLE`
Tool stripe	30 px wide, `-color-bg-subtle`, 8 px spacing, no edge; buttons min 24, padding 9 5, radius 6	`.tool-stripe`, `.tool-stripe-button`
Stripe states	Open: `-color-accent-subtle` fill, accent glyph. Focused window: `-state-accent-hover`	`:open`, `:tw-active`
Bottom stripe	Label 14 px beside the glyph, padding 3 8; open = accent, bold	`.tool-stripe-horizontal .tool-stripe-button`
Panel	`-color-bg-subtle` ground, radius 0 0 8 8, no border	`.tool-window`
Panel header	Padding 6 6 6 12, radius 7 7 0 0, no rule under it; title 14 px bold muted with the window's glyph; active: `-color-accent-subtle` band, accent title	`.tool-window-header`, `.tool-window:active`
Panel content	`-color-bg-default`, padding 8; its tree or list draws no frame of its own	`.tool-window-content`
Per side	Two windows at most, stacked in an inner SplitPane	`MAX_PER_SIDE`
Persisted	Open state, sides, order, visibility, floating windows, divider positions	`WorkspaceState` in `workspace-state.json`

Menus

Two surfaces show the same rows: the **menu bar**, a browsable map of every command, and **right-click menus**, what can be done with the thing under the pointer. Both are built from the command registry, so a row's title, icon, chord and enabled state come from the command and never drift from the palette or the toolbar. Reference: Editora's `ui/MainMenuBar.java` (menu bar), `ui/ContextMenuDismissal.java`, `ui/LazyContextMenu.java`, `ui/RowContextMenu.java` and `ui/TabContextMenu.java` (right-click menus), and the `.menu-item` and `.context-menu` rules in `app.css`.

Look and feel

Both surfaces draw their rows the same way. Absolute pixels are deliberate: a popup inherits the font of the node it was opened from, and `em` would resolve against a file tree's or an 18 px toolbar's size (Editora's toolbar menu once rendered a third larger than every other menu in the app).

Property	Value	Reference
Font	Inter 14 px, regular, in every row and label	`.context-menu`, `.context-menu .menu-item .label` in `app.css`; `MainMenuBar.UI_FONT_SIZE`
Row	34 px high: padding 8 × 12, the same in every theme	`.menu-item { -fx-padding: 8px 12px }`
Icon column	22 px, filled on every row: a command row's glyph, or a check row's tick; 8 px to the title	`MainMenuBar.ICON_COLUMN`, `rowContent`
Icon	The command's glyph, the same one as on the toolbar and in the palette (Save is the same floppy everywhere), filled `-color-fg-muted`	`Icons`, `MenuBarIcons`, `.toolbar-icon`
Title	Inter 14 px, `-color-fg-default`	`.menu-item-title`
Chord	Right-aligned in its own column, Inter 11 px, in the keyboard color (`-state-key`, brace periwinkle); at least 18 px from the title column; in every menu, right-click menus included	`.menu-item-chord`, `alignChordColumns`
Submenu row	Laid out like a command row, with an invisible chord as wide as the menu's widest, so the arrow lands at the right edge	`nestedMenu`, `.menu-submenu`
Card	Radius 8; shadow `rgba(27,30,42,0.28)`, radius 16, **offset Y 16**	`.context-menu`
Danger item	Text in `-color-danger-fg`	`.menu-item.danger`
Disabled item	The theme's dimmed label, and the icon dimmed with it	Termina's `.menu-item:disabled .menu-icon`
Separators	Never two in a row and none at either end: when items hide, their orphaned separators hide too	`LazyContextMenu.collapseSeparators`, Torrenta's `MenuSeparators`

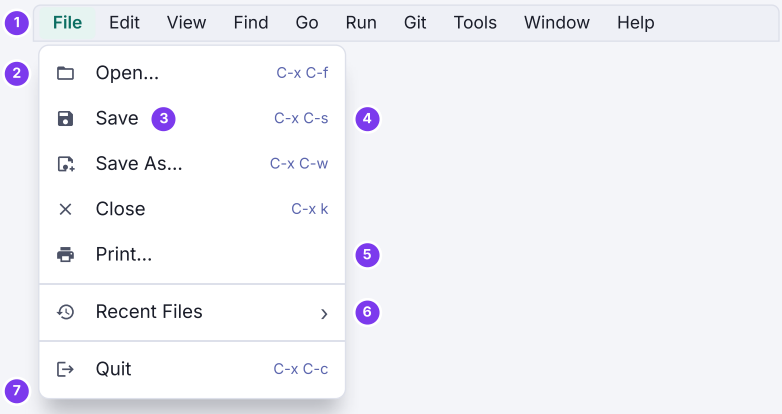
The shadow's offset is not cosmetic. A menu is a native window sized to its root *including the effect*, so a shadow that reaches above the menu is invisible window over the bar the menu hangs from, and the pointer lands on it instead of the next menu button. The offset must be at least the radius; Editora pins it with `MenuPopupBoundsFxTest`.

The three rules below hold for every menu the app draws itself: the menu bar, right-click menus, the drop-down of a menu button or a split button, and the toolbar's overflow. The menus a platform draws (the macOS system menu bar, a tray icon's menu) show text only; there the chord is appended to the title (MENU-02) and the tray shows neither.

- **MENU-14** `MUST` Every row of every menu the app draws carries an icon that depicts it. A command row wears its command's glyph; a row that names a thing rather than an action (a recent file, a branch, a project) wears that thing's icon; a submenu wears its family's. Only a check or radio row goes without, because its mark fills the icon column. Where the icon set has no glyph that depicts a command, the app adds one (a Material 24 dp path, or a line glyph in the kit's style) instead of leaving the row blank or borrowing another command's.
  - *Why:* a menu is scanned before it is read, and the glyph is what the eye finds first; the same glyph on the toolbar, in the palette and in the menu (ICON-20) is learned once. A blank cell breaks the column, and a borrowed glyph says something else.
  - *Check (test):* an FX test opens the menu bar's menus and the right-click menus of a tab, an editor, a list row and the toolbar, and fails on any visible row that is not a check or radio item and has no graphic.
  - *Reference:* Editora `ui/MenuBarIcons.java`, `editor/MenuIcons.java`, `ui/TabContextMenu.java`; Termina's tab menu has an icon on every row. Editora doesn't yet pass: about half its menu-bar rows have no glyph, and `MenuBarIconsTest` holds that a command without a fitting glyph gets none, which this rule reverses; many right-click and drop-down rows have none either (Run All Requests in a `.http` file, the CSV grid's menu, the Markdown preview's).
- **MENU-15** `MUST` Every menu row whose command has a chord in the active keymap shows it, in right-click menus and drop-downs as in the menu bar: right-aligned in the menu's chord column, Inter 11 px in `-state-key`. An unbound command leaves the cell empty, never a wrong chord, and a keymap switch rewrites every cell.
  - *Why:* the menu is where a user learns the keys. A right-click menu that hides them teaches the mouse, while the palette and the toolbar's tooltips already show the same chord.
  - *Check (test):* an FX test binds a command, opens a right-click menu that lists it and finds its chord in a `menu-item-chord` label; after a keymap switch it finds the new chord.
  - *Reference:* Editora `MainMenuBar.rowContent` and `alignChordColumns`, `.menu-item-chord` in `app.css`. Editora's right-click menus show no chords yet: a row there is a plain `MenuItem`, not the menu bar's row.
- **MENU-16** `MUST` Every menu draws its titles in Inter 14 px regular and its chords in Inter 11 px, whatever node it was opened from: the font is pinned in absolute pixels on `.context-menu` and its rows' labels.
  - *Why:* a popup inherits the font of the node it was shown from, and `em` resolves against it: Editora's toolbar menu once rendered at the bar's 18 px, a third larger than every other menu, and a file tree's or a monospace editor's font leaks the same way.

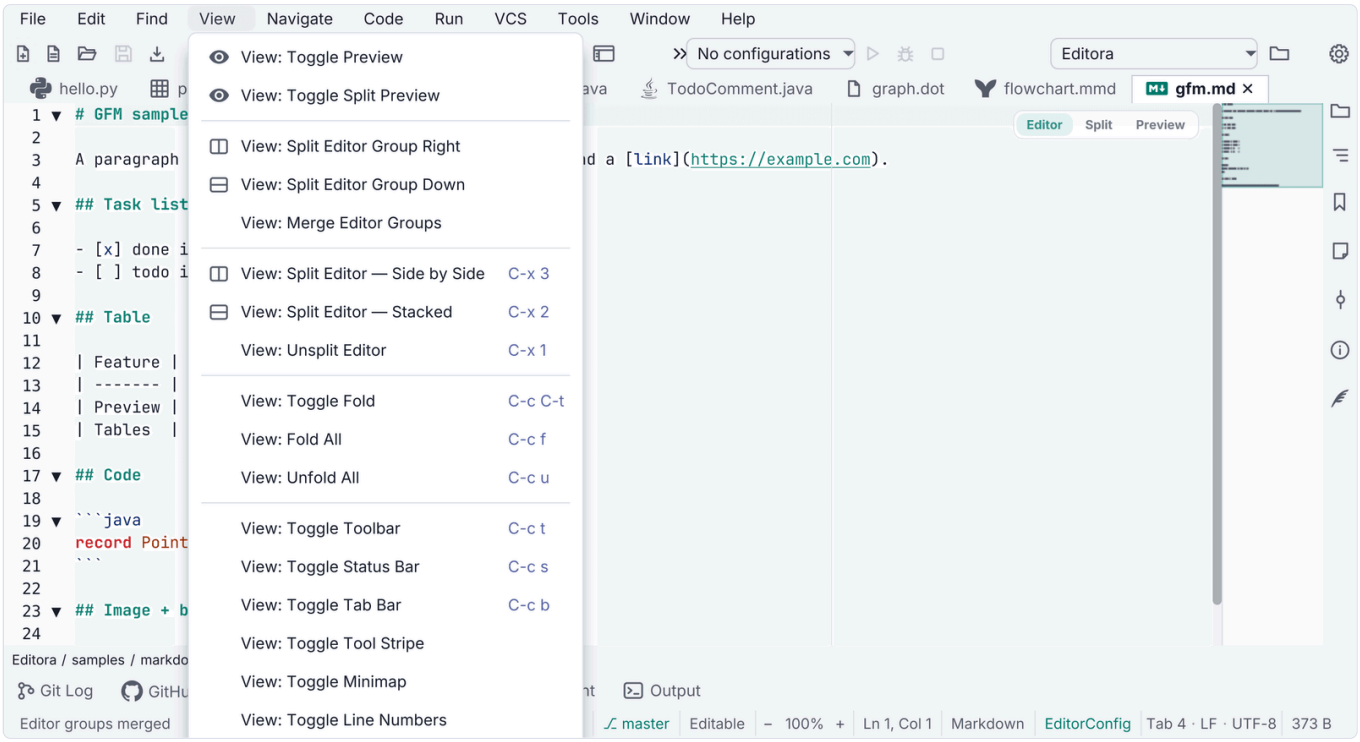
- *Check (test):* an FX test opens a menu from the toolbar, an editor and a tree row and reads Inter 14 px from each row's title.
- *Reference:* Editora `.context-menu` and `.menu-item-chord` in `app.css`, `MainMenuBar.UI_FONT_SIZE`, `ToolbarMenuFontFxTest`.

The menu bar



1. **Menu button.** Padding 3 × 10 on a bar with 4 px side insets; the bar and the toolbar share one `-color-bg-subtle` band with no rule under it.
2. **Icon column**, 22 px. Every row has a glyph that depicts its command; where the icon set has none that fits, the app adds one rather than leave the cell blank.
3. **Title**, 14 px Inter, the command's title with the menu's own name dropped ("Save", not "File: Save") unless two rows would then read the same.
4. **Chord column.** Inter 11 px, periwinkle, right-aligned into one column per menu (not app-wide), read from the active keymap. Drawn by the app, not a JavaFX accelerator: Emacs chords are multi-key, which a `KeyCombination` cannot express, and on macOS an accelerator would bypass the key dispatcher.
5. **Unbound command.** An empty chord cell, never a wrong chord. A live keymap switch rewrites every cell.
6. **Submenu row.** Icon, title and an invisible chord as wide as the menu's widest, so its arrow sits at the right edge instead of halfway across.
7. **Card.** Radius 8; the shadow falls below the menu (offset ≥ radius), so nothing invisible covers the bar.

- `MENU-01` `MUST` Every menu-bar item is a command from the registry, with its title from `command.<id>`, its chord from the active keymap and its enabled state from the command's gate. The bar assembles and refreshes; it holds no behavior of its own.
  - *Check (code):* the bar's model lists command ids only, and every row runs its id through the registry.
  - *Reference:* Editora `ui/MenuBarModel.java` over `command/CommandRegistry.java`, `ui/MainMenuBar.java`.
- `MENU-02` `MUST` Menu-bar rows use no `KeyCombination` accelerators. In the in-window bar each row is a `CustomMenuItem` laying out icon, title, spacer and chord. On the macOS system menu bar (`useSystemMenuBar`) the chord is appended to the title and rows carry no icon.
  - *Why:* Emacs chords are multi-key, which a `KeyCombination` cannot express, and on macOS an accelerator would bypass the key dispatcher. The native macOS menu draws text only and no graphics, which is also the platform's convention.
  - *Check (code):* no `setAccelerator` in the menu-bar classes.
  - *Reference:* Editora `ui/MainMenuBar.java` (`rowContent`, `refresh`).
- `MENU-03` `SHOULD` Enablement is re-read every time a menu opens, and titles and chords on a keymap or locale change.
  - *Why:* a menu built before Git finished detecting a repository must not stay grayed out.
  - *Check (test):* an FX test detects a repository after the window is built and finds the menu enabled.
  - *Reference:* Editora `MainMenuBar.refreshEnablement`, `MenuBarEnablementFxTest`.
- `MENU-04` `SHOULD` Simple mode shows a shorter menu, rebuilt only when the mode actually changes.
  - *Check (code):* the bar rebuilds only when the simple flag differs from the current one.
  - *Reference:* Editora `MainMenuBar.setSimple`, `MenuBarModel.menus(simple)`.

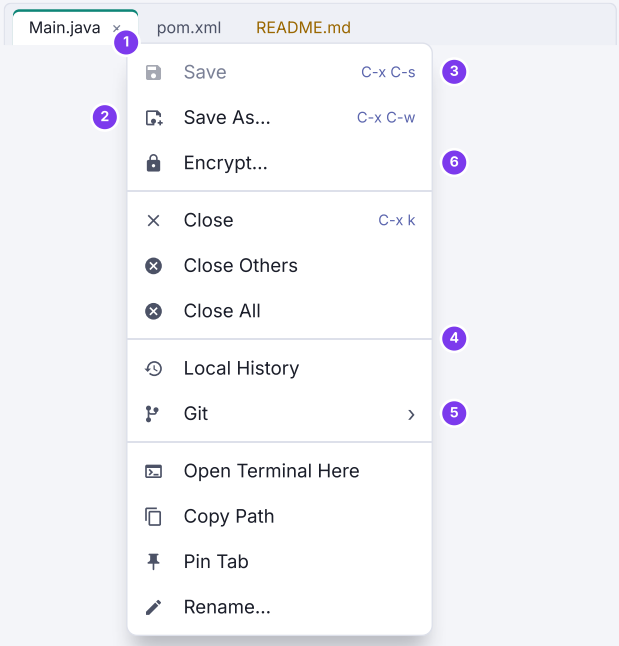


Editora's View menu in an earlier build: chords in one periwinkle column, but half the rows have no icon and every title repeats the menu's name.

Right-click menus

- `MENU-05` `MUST` Every list row, tree row, tab, editor and read-only console has a right-click menu with the actions for *that* thing, each one a command.
  - *Check (by hand):* right-click each kind of thing; every row's action is also in the palette.
  - *Reference:* Editora `ui/TabContextMenu.java`, `ui/RowContextMenu.java`, `ui/ConsoleContextMenu.java`.

The mockup is an editor tab's menu, shortened; Editora's has twenty-five rows.



1. **Anchored at the pointer.** The menu's top-left corner is the click. From the keyboard (Menu key or `Shift+F10`) it opens under the selected row, a quarter of the way in and never off a narrow one, or just below the caret in an editor; never in the middle of the control.
2. **Every row has an icon**, the command's own, and its chord when it has one, in the same periwinkle column as the menu bar's. The one row without an icon is a `CheckMenuItem`, whose tick lives in the graphic column.
3. **Grayed, not hidden.** An action that does not apply right now (Save on an unchanged file, Reveal on an unsaved one) stays in place, dimmed. Only a row that cannot apply to this thing at all is hidden (the Git submenu on an untitled buffer).
4. **Separators** group the rows; one that hiding has left alone hides too.

5. **Submenu** for a family of actions (Git: stage, unstage, revert, compare, history), with its own icon.
6. **Labels follow state.** Encrypt… becomes Lock File on an encrypted buffer, Pin Tab becomes Unpin Tab. Decided on every showing, not when the menu was built.

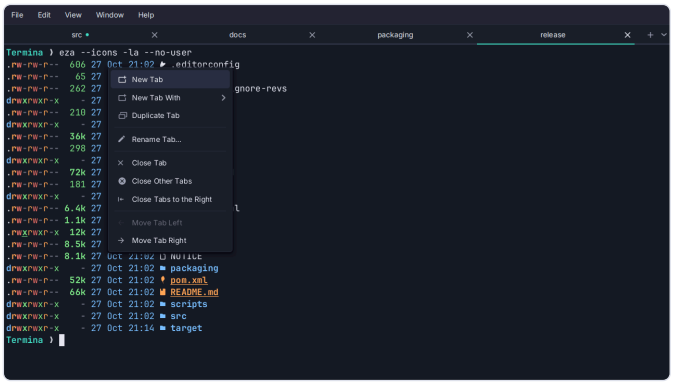
Behavior

- `MENU-06` `MUST` One right-click menu at a time: a right-click while a menu is open replaces it, and the first one closes before the second shows.
  - *Why:* JavaFX stacks a second menu beside the first when each row owns its own `ContextMenu`, and the stale one is then no longer dismissible.
  - *Check (test):* an FX test right-clicks another row with a menu open and finds one menu showing.
  - *Reference:* Editora `ui/ContextMenuDismissal.java` closes every open right-click menu on the press that opens the next (`ContextMenuDismissalFxTest`); Modula's `RadioPane` hides the previous menu before showing a new one; Termina keeps one menu per view and shows it again.
- `MENU-07` `MUST` A press anywhere outside a right-click menu closes it, including inside the control the menu was opened on. The press is not consumed: a click in a list still selects the row, and a right-click on another row opens that row's menu straight away. Drop-downs opened by a plain click (menu buttons, choice boxes, the menu bar) keep the stock behavior, and a menu that turned auto-hide off is left alone.
  - *Why:* JavaFX's auto-hide deliberately ignores a press on the menu's *owner node*, so a stock menu on a tree or a text area stays open when the user clicks another row or the text.
  - *Check (test):* an FX test clicks inside the owner control and in another window, and the menu closes.
  - *Reference:* Editora `ContextMenuDismissal.install()` , called once in `App` at startup for every current and future window; `ContextMenuDismissalFxTest` .
- `MENU-08` `MUST` Escape closes a right-click menu and focus goes back where it was. In a terminal the Escape does not reach the shell.
  - *Check (by hand):* open a menu, press Escape, and type: the keys reach the control that had focus.
  - *Reference:* the stock `ContextMenu` hide-on-Escape (Spec below); Termina's `onKeyPressed` filter hides the menu and consumes the key (`TerminalView`).
- `MENU-09` `MUST` The Menu key and `Shift+F10` open the selected row's menu, scrolled into view first, so every row action a mouse can reach the keyboard can too.
  - *Why:* those keys arrive at the focused control, not at a row's cell, so a menu installed on cells never saw them.
  - *Check (test):* an FX test sends a keyboard request to a tree and a list and finds the selected row's menu, anchored to that row.
  - *Reference:* Editora `RowContextMenu.install(tree)` , `RowContextMenuFxTest` .
- `MENU-10` `SHOULD` A right-click selects first. In an editor, a right-click outside every selection moves the caret to the click (so Paste lands there); inside a selection it keeps carets and selections (so Cut and Copy act on what was clicked). From the keyboard the target is the caret. In a list, the menu is the clicked row's.
  - *Check (by hand):* select a word, right-click elsewhere and paste; then right-click inside a selection and copy.
  - *Reference:* Editora `editor/ContextMenuTarget.java` .
- `MENU-11` `SHOULD` A menu attached to something that exists once per open file is built when first shown, and set up for each showing: an `onShowing` handler sets the labels, enablement and visibility for *this* showing, then collapses the separators.
  - *Why:* such a menu is built for every file and opened for almost none.
  - *Check (code):* per-file menus are created through `LazyContextMenu.of(build)` .
  - *Reference:* Editora `ui/LazyContextMenu.java` , `LazyContextMenuFxTest` . A read-only console's menu is just Select All and Copy, both disabled while it is empty (`ConsoleContextMenu`).
- `MENU-12` `SHOULD` A destructive row carries the `danger` class. The action follows the window shell rule: it asks first with a button that names it, or is undoable.
  - *Check (code):* rows that delete, discard or reset add `danger` to their style classes.
  - *Reference:* Editora `.menu-item.danger` in `app.css` , `ui/BranchPopup.java` .
- `MENU-13` `MAY` In a terminal view, a program that grabbed the mouse owns the right-click; `Shift`+right-click always opens the menu, and the keyboard request is never contested.
  - *Check (test):* a pure test of when the menu shows, per mouse mode, Shift and trigger.
  - *Reference:* Termina's `TerminalView.shouldShowMenu` , `ContextMenuContentionTest` .

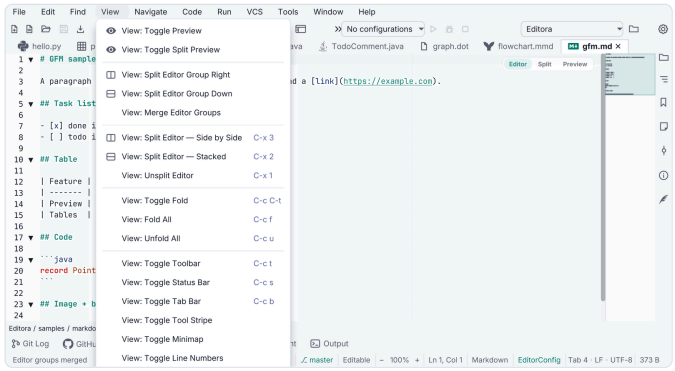
Spec

Property	Value	Reference
Kind	`javafx.scene.control.ContextMenu` , auto-hide on, hide on Escape on: a popup window, unlike the palette	`TabContextMenu` , `ConsoleContextMenu`
Opens at	The pointer; from the keyboard, the selected row's bottom edge at min(width ÷ 4, 48) px in, or under the caret	`RowContextMenu.anchorX` , `ContextMenuTarget`
Font	Inter 14 px, pinned on `.context-menu` , whatever the owner's font	`app.css`
Rows	34 px (padding 8 × 12); icon column 22 px; glyph scaled 0.8 from a 24 dp Material path, or a 16-unit line glyph	`Icons.ICON_SCALE` , `Icons.line`
Card	Radius 8, shadow radius 16 at offset Y 16	`.context-menu`
Dismissal	Any press outside the menu, in any window; the next right-click; Escape	`ContextMenuDismissal` , `ContextMenuDismissalFxTest`
Marker	A showing menu opened by a right-click or the Menu key carries the window property `editora.rightClickMenu`	`ContextMenuDismissal.RIGHT_CLICK_MENU`

In the apps



Termina's tab menu: an icon on every row, following Editora's convention.

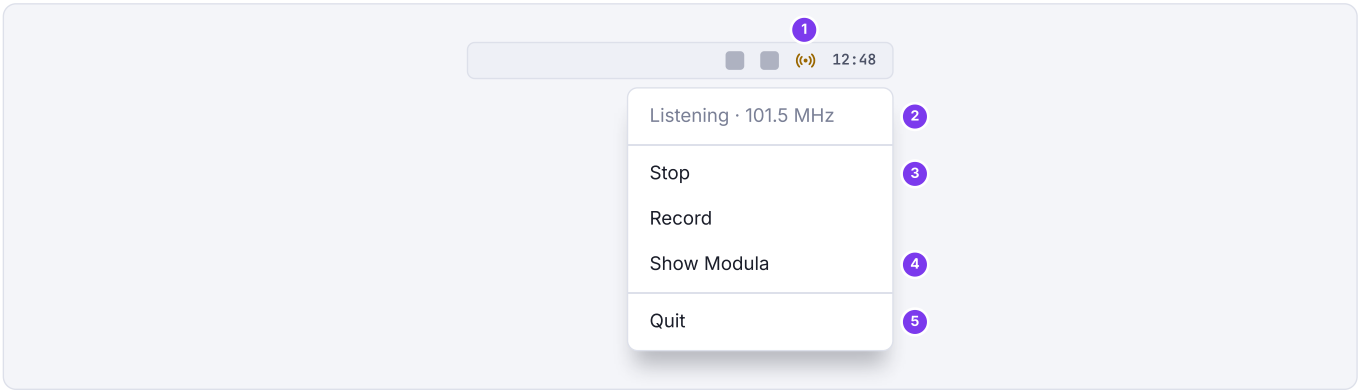


Editora's in-window menu bar with the chord column, in an earlier build whose rows still lack icons and repeat the menu's name.

Tray and background

An app whose work goes on without its window (Torrenta's transfers, Modula's radio, nux's monitoring) lives behind a tray icon. The rules below are for such an app; for the others (Editora, Termina, Mirada) a tray is `MAY`, and none has one. Reference: Torrenta's `tray/` package (`Trays`, `TorrentaTray`, `SniTray`, `AwtTray`, `TrayIconRenderer`, `SniMenuModel`), `os/Autostart` with `os/AutostartSpec`, and `automation/Notifiers`. Modula's and nux's `tray/` packages are the same code with their own menus; nux's `TrayController` holds the close policy.

The icon and its menu



1. **The icon** is the app's own mark, recolored by state, not a badge and never the number it stands for ("101.5" at 22 px is unreadable and lies by rounding). Torrenta's arrow is teal while something downloads, muted when nothing does, red on an error; Modula's broadcast glyph amber while listening, ink when stopped, coral on a fault; nux shows the charge number colored by severity, gray when disconnected.
2. **Status line**, disabled, first. It is also the tooltip, because GNOME's AppIndicator extension renders a StatusNotifierItem's menu but not its tooltip, so the menu is the one place the live figures are sure to appear.
3. **The main actions** the window would offer, so the common jobs need no window: Listen becomes Stop, Record is grayed (not hidden) until the receiver runs; Torrenta's menu has none; nux offers Connect, Switch UPS and the instant commands, which still go through the app's confirmation.
4. **Show *App*** brings the window back, as a left click on the icon does.
5. **Quit**, after a separator. The only way to end the program apart from File → Quit and the system's Quit.

Tray menus are text only: AWT's `PopupMenu` and dbusmenu show no item icons, so the menu rules about icons and chords do not apply here. The order does.

- `TRAY-01` `MUST` The tray comes from the shared `tray/` package: a StatusNotifierItem on Linux, AWT's `SystemTray` on Windows and macOS, created off the FX thread after the first frame.
  - *Why:* D-Bus and AWT can block, and neither is worth a stalled first frame.
  - *Check (code):* `Trays.create` picks `SniTray` on Linux and `AwtTray` elsewhere, and the window calls it from a background thread.
  - *Reference:* Torrenta `tray/Trays.java`, `MainWindow.installTray`; the Spec table below.

Spec

Property	Value	Reference
Linux backend	A StatusNotifierItem on the session bus, found through `org.kde.StatusNotifierWatcher`: KDE natively, GNOME with the AppIndicator extension, X11 and Wayland alike. No AWT fallback: the XEmbed tray GNOME dropped in 3.26 and Wayland never had	`SniTray`, `StatusNotifierWatcher`, `Trays.create`
Windows and macOS	AWT `SystemTray`, every mutation on the EDT, a multi-resolution image so a HiDPI tray picks a crisp variant	`AwtTray`
Created	Off the FX thread, after the first frame (D-Bus and AWT can block); the outcome is a tray or the one-line reason there is none	`Trays.Attempt`, `MainWindow.installTray`
Icon	Pure Java2D from the branding SVG's geometry, at each size the backend asks for, so the panel entry and the dock entry are one mark	`TrayIconRenderer`, `TrayDisplay`
Attention	A state that needs a person sets the SNI status `NeedsAttention`; otherwise `Active`	`SniTray.update`
Left click	Shows the window: SNI `Activate` and `SecondaryActivate`, a single primary click on the AWT icon	`SniTray.Item`, `AwtTray`
Right click	The menu; the host pulls it from the item's `Menu` property	`SniMenu`, `SniMenuModel`
Updates	Only when the value-equal snapshot changed: a repaint crosses a process boundary on Linux. Torrenta polls once a second, Modula nine times	`TrayDisplay`, `MainWindow.pushTray`
Package	Free of JavaFX: it takes callbacks, and the window marshals them onto the FX thread	`TorrentaTray`

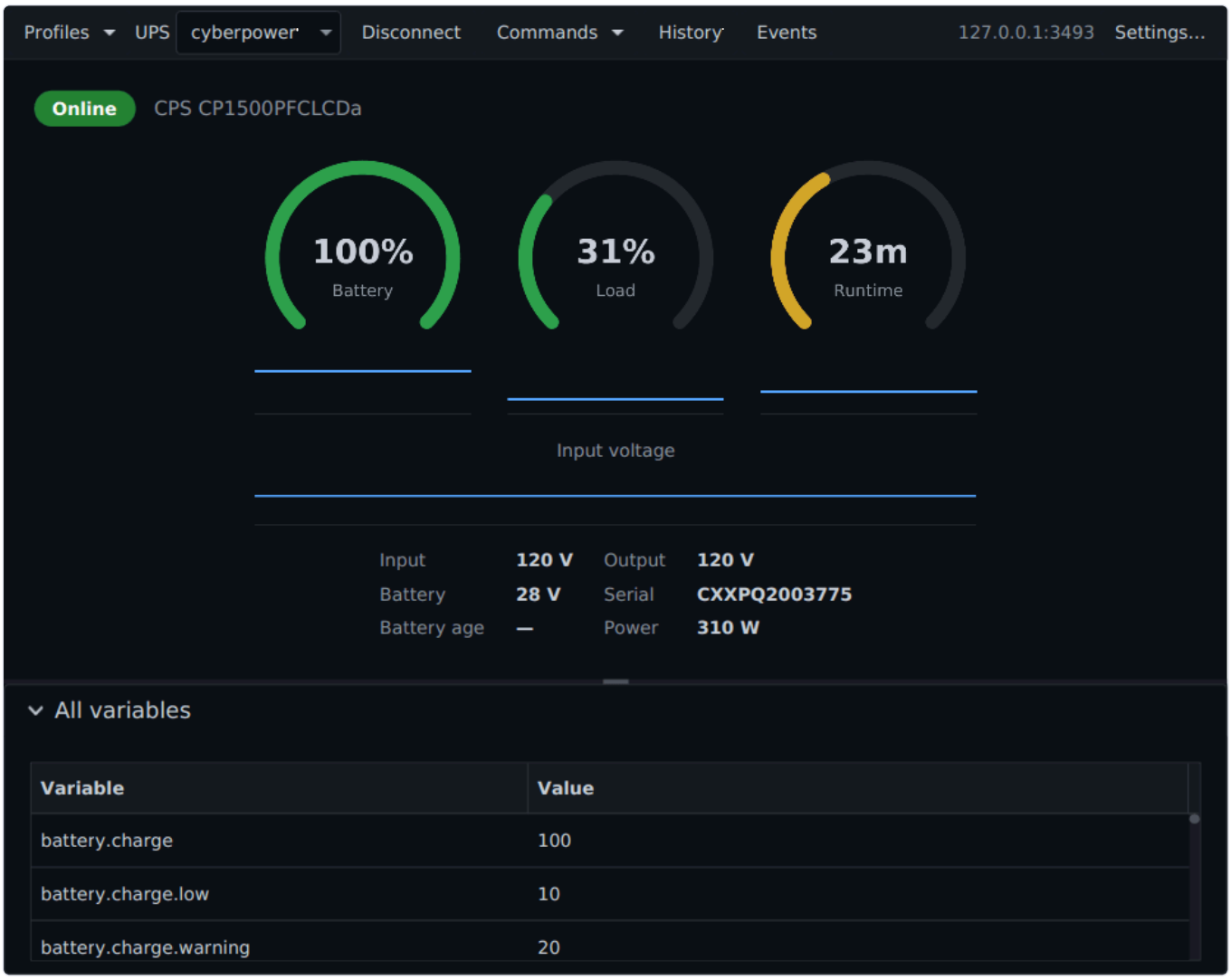
Behavior

- **TRAY-02** `MUST` The app is fully usable without a tray. No icon is a normal outcome (stock GNOME has no StatusNotifier host), not an error, and the reason is said where the person will look.
  - *Why:* the bug they actually hit is silence: the box says close to tray, and closing quits.
  - *Check (code):* when the tray attempt returns no tray, the status bar says why and that closing quits.
  - *Reference:* Torrenta `MainWindow.installTray` ("No tray icon (…); closing the window quits Torrenta", `status.noTray`); Modula's status says closing the window will quit it (`RadioPane.reportNoTray`).
- **TRAY-03** `MUST` Closing the window hides it only while an icon is actually up and the setting says so, both read at close time; otherwise a close quits. `Platform.setImplicitExit(false)` is set only once a tray exists, and turning the tray off brings a hidden window back.
  - *Why:* the checkbox can change while the window is open, and hiding a window that leaves no icon behind is how a program becomes unreachable.
  - *Check (code):* the close handler tests that a tray exists and reads the live setting; `setImplicitExit(false)` sits after the tray is created.
  - *Reference:* Torrenta `MainWindow.setOnCloseRequest`, Modula's `installCloseBehaviour`, nux's `TrayController.closeHidesWindow` and `updateImplicitExit`.
- **TRAY-04** `MUST` Quit is explicit and one path: the tray's Quit, File → Quit and the system's Quit all run the same method, which closes open dialogs first and saves the window geometry.
  - *Why:* quitting from inside a nested event loop aborted the JVM.
  - *Check (code):* every quit entry point calls the one quit method.
  - *Reference:* Torrenta `MainWindow.quit`.
- **TRAY-05** `SHOULD` `--minimized` comes up behind the icon. The window is never shown until the tray attempt has answered; if no icon comes, the window is shown after all. With the tray setting off, Torrenta starts minimized instead and nux ignores the flag.
  - *Why:* a start at login can then never leave a program with no way back.
  - *Check (code):* a `--minimized` start shows the window when the tray attempt returns none.
  - *Reference:* Torrenta `MainWindow.show`, nux's `App`.
- **TRAY-06** `SHOULD` Start at login writes the OS's own entry, and nothing else: an XDG autostart entry in `~/.config/autostart` (quoted `Exec`, `StartupWMClass` matching the installed launcher, `X-KDE-autostart-after=panel` so a tray host is up first), a LaunchAgent in `~/Library/LaunchAgents`, the `HKCU\…\Run` value on Windows (read together with `StartupApproved`, where Task Manager keeps the person's "off"). The entry is the one truth: turned off in GNOME Tweaks, KDE's Autostart page or Task Manager, the box shows unticked. The one setting beside it is *start behind the tray icon*, written into the entry as `--minimized`.
  - *Check (test):* a pure test of the exact bytes of each platform's entry.
  - *Reference:* Torrenta `os/Autostart.java`, `os/AutostartSpec.java`, `AutostartSpecTest`; nux's design.
- **TRAY-07** `SHOULD` Only the app writes the login entry, and only from an installed package: the installers never do it, and a run from source says it has nothing to start.
  - *Why:* an installer runs for every account.
  - *Check (code):* the entry's target comes from `jpackage.app-path`, and is absent when it is unset.
  - *Reference:* Torrenta `os/Autostart.java`.
- **TRAY-08** `SHOULD` Every start-at-login call runs off the FX thread.
  - *Why:* it may block on the file system or `reg.exe`.
  - *Check (code):* every `Autostart` read and write starts on a worker thread.
  - *Reference:* Torrenta `SettingsWindow` (`autostart-read`, `autostart-write`) and `MainWindow` (`autostart-refresh`); nux's `App` (`nux-autostart`).
- **TRAY-09** `MUST` Desktop notifications go through one `Notifier`: `notify-send` on Linux (nux and Termina fall back to `gdbus` against the freedesktop service), `osascript` with `display notification` on macOS, a PowerShell toast on Windows. Each is a short program run with the title and message as arguments, never as part of a script, off the FX thread and bounded to a few seconds; a failure is logged and never breaks the app.
  - *Why:* text from a torrent name or a remote shell then cannot run anything.
  - *Check (test):* a test holds that each platform's command line passes the text as arguments and that a missing program is a problem, not an exception.
  - *Reference:* Torrenta `automation/Notifiers.java`, `ProcessNotifier`, `NotifierTest` (it ships `toast.ps1`); nux's `service/Notifier` raises it through the tray icon, which it keeps as the last resort on every platform; Termina's `DesktopNotifier`.
- **TRAY-10** `MUST` Notifications are for what happened while the window was away, with one switch per event and few on by default.
  - *Check (code):* each notified event has its own setting, and most default to off.
  - *Reference:* Torrenta notifies on added, finished, seeding done, error and updated, with only *finished* on at first, and a Test button in Settings shows one now (`automation/AutomationSettings.java`); nux notifies on power events with a rule per UPS; Termina and Editora relay a program's own OSC 9 and OSC 777 notifications.
- **TRAY-11** `SHOULD` A notification is never the only place a state is said: the in-app status already shows it.
  - *Check (by hand):* trigger each notified event with notifications off; the window still shows it.
  - *Reference:* Torrenta's status bar and transfer list.
- **TRAY-12** `MAY` A dock badge: Torrenta puts the download rate on the macOS dock icon through AWT's `Taskbar` and a count on a Linux launcher that honors Unity's `LauncherEntry` signal (KDE, Dash to Dock); nothing on Windows, whose taskbar badges only AWT's own windows. It is cleared when nothing downloads and has its own switch.
  - *Check (test):* a pure test of the badge state.
  - *Reference:* Torrenta `os/Badges.java`, `BadgeState`, `BadgeStateTest`.
- **TRAY-13** `SHOULD` A second launch brings the hidden window forward: the single-instance hand-off shows a window hidden to the tray before it opens what it was given.
  - *Check (by hand):* close to the tray, then launch the app again with a file.
  - *Reference:* Torrenta `MainWindow.showFromTray`.
- **TRAY-14** `SHOULD` AWT stays headless unless the tray needs it. Linux never needs it; Windows and macOS lift the guard only when the tray setting is on, decided in `main`. A tray turned on later says "restart to show the tray icon" rather than "unsupported".
  - *Check (test):* a test holds that AWT is headful only where the tray is AWT's and wanted.
  - *Reference:* Torrenta `Trays.usesAwt`, `os.AwtUse`, `TraysTest`.

Settings copy

Key	English
`ui.tray`	A tray (menu-bar) icon with the rates and a menu; on by default
`ui.closeToTray`	Closing the window keeps Torrenta in the tray. *The torrents go on; the tray icon's Show Torrenta brings the window back, its Quit ends the program, as does Quit in the File menu. Without a tray, closing quits.* On by default; nux grays it while the tray is off
(the OS entry)	Start Torrenta when you log in. *Adds Torrenta to your login items: an entry in ~/.config/autostart on Linux, a LaunchAgent on macOS, the Run key on Windows. Turned off in the system's own settings, it shows unticked here. Only for an installed Torrenta, not a run from source.*
`os.autostartInTray`	Start behind the tray icon. *At login Torrenta comes up behind its tray icon instead of as a window, and minimized where there is no tray.* `torrenta --minimized` *does the same by hand.*
`notify.<event>`	One switch per event; `notify.finished` on by default
`status.noTray`	No tray icon ({reason}); closing the window quits {app}

In the apps

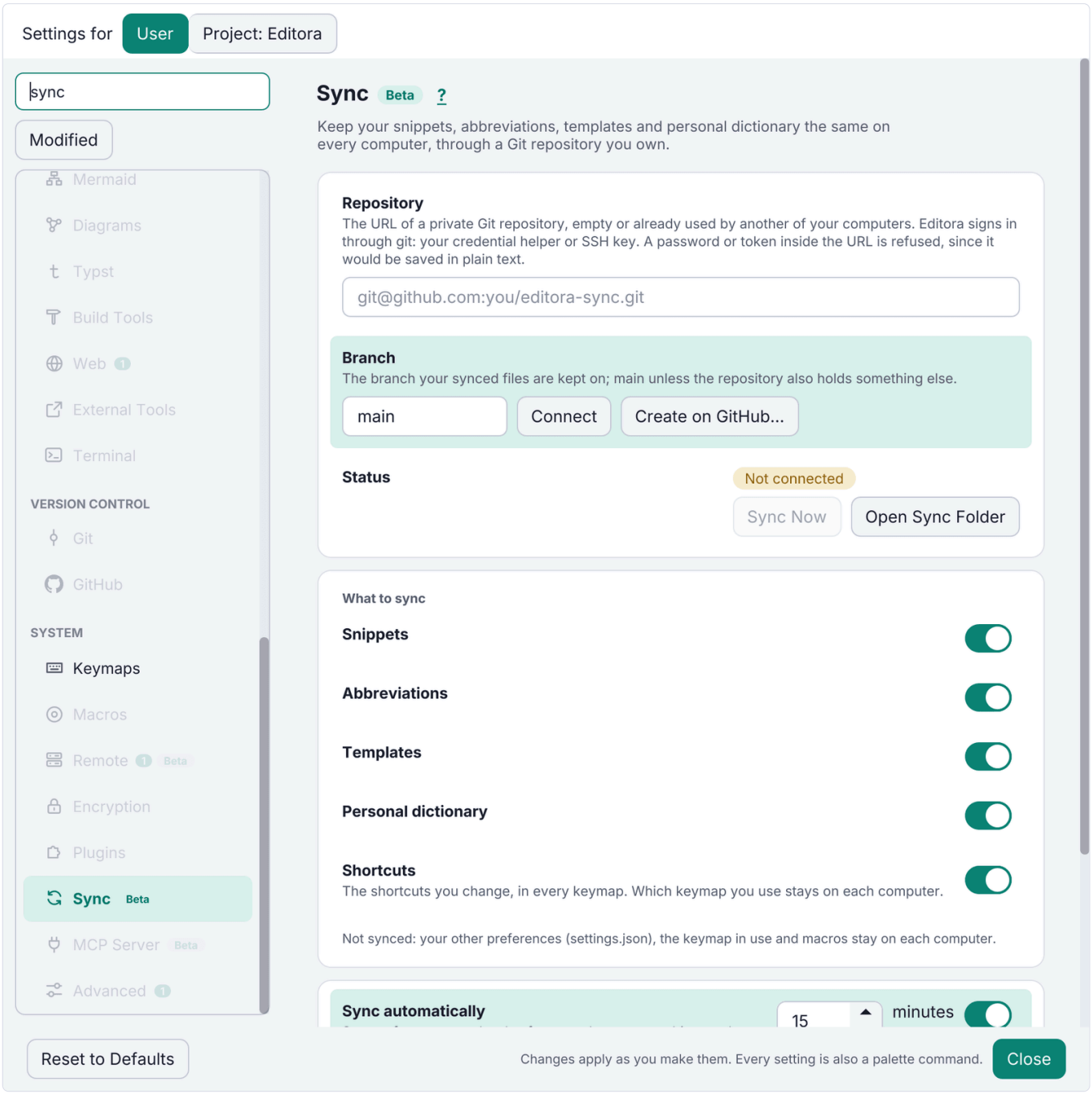


nux lives in the tray: its icon carries the charge number, and this dashboard is one left click away.

Settings

The Settings window

- **SET-01** `MUST` The Settings window opens from a palette command, bound to `Ctrl+,` (`Cmd+,` on macOS) in the CUA keymap.
  - *Check (code):* a registered command opens Settings, and `cua.json` and `cua.mac.json` bind it.
  - *Reference:* Editora `view.settings`, bound in `keymaps/cua.json` and `cua.mac.json` (and in the VS Code and Sublime keymaps; IntelliJ's uses `C-M-s`).
- **SET-02** `MUST` The window has a sidebar of pages with icons on the left, the page on the right, and a search field on top.
  - *Check (by hand):* open Settings; the search field sits above the sidebar and the page.
  - *Reference:* Editora `ui/SettingsWindow.java`, `ui/SettingsSidebarNav.java`, `ui/SettingsIcons.java`.
- **SET-03** `MUST` Search finds a row by its title, description, visible text, its card or section heading, its page name and English keywords. Title hits rank above description hits; whole words above word starts. The page with the best match opens, and every non-matching row, heading and card is hidden.
  - *Check (test):* a unit test ranks a title hit above a keyword hit and a whole word above part of one.
  - *Reference:* Editora `ui/SettingsSearch.java`, `SettingsSearchTest`.
- **SET-04** `MUST` Settings apply live, with no Save button. Text fields commit on Enter or focus loss; invalid values are marked and never committed.
  - *Check (code):* no Save or Apply button in the window; text fields commit through `commitOnEnterOrBlur`.
  - *Reference:* Editora `SettingsWindow.commitOnEnterOrBlur`; `SettingsSyncPage`'s branch field marks an invalid branch.
- **SET-05** `SHOULD` A change made in one window reaches every other open window.
  - *Check (test):* an FX test changes a setting in one window and finds it applied in the other.
  - *Reference:* Editora: every window holds the same `config/SharedConfig.java`, which announces each change (`onSettingsChanged`); `CrossWindowSettingsFxTest`.



Typing sync in Settings: pages without a match gray out and the best page opens.

Before adding a setting

A setting is a question the user has to answer. When the question has one right answer, choose it, make it the behavior, and say why in the code and the docs; do not ship a knob for it. Modula's test: *would a car radio have it?* A preset button, yes; a "decimation stages" spinner, no. So Modula has no gain, bandwidth or squelch control: each has one right value for a listener, and the app supplies it. Add a setting when people really differ (a keymap, a theme, a language), not when the developer could not decide.

Adding a setting touches five places

1. **Model** field, default and accessors, with a schema bump and migration.
2. **Control**, built once, writing the field and applying it.
3. **Row** on its page with search keywords.
4. **Load** line that sets the control from the model, safe to run at any time.
5. **Palette command** that flips or prompts the same field, plus strings in every catalog.

- `SET-06` `MUST` Every setting has a palette command that flips the field or prompts for its value.
  - *Check (by hand):* for each row in Settings, find a palette command that changes it.
  - *Reference:* Editora's `view.toggle…` and picker commands (`ui/WindowCommandRegistrar.java`). Editora does not have one for every setting yet: `terminalNotifications` , for one, is only a Settings row.

Config folder and schema

- `SET-07` `MUST` Config lives in `~/.<app>`: `~/.<app>-dev` for `--dev`, any folder via `--config-dir` or `<APP>_CONFIG_DIR`.
  - *Check (code):* one resolver picks the folder, in the order `--config-dir`, the environment variable, the default.
  - *Reference:* Editora `App` and `ConfigManager.configDirFor`, `ConfigDirTest`.
- `SET-08` `SHOULD` `settings.json` holds preferences, `workspace-state.json` session state, and other stores their own JSON files.
  - *Check (code):* the config folder's file names are constants in the config package.
  - *Reference:* Editora `config/ConfigManager.java`, `config/WorkspaceState.java`.
- `SET-09` `MUST` Every versioned file has a `SCHEMA_VERSION` and a `v → v+1` migration step. A purely additive field uses an identity step.
  - *Check (code):* each store declares `SCHEMA_VERSION`, and the migrations have one step per version.
  - *Reference:* Editora `config/migration/ConfigMigrations.java`, `ConfigSchema.java`, `ConfigMigrationsTest`.
- `SET-10` `MUST` A file newer than the running build is backed up to `<name>.v<n>.bak` and defaults load.
  - *Why:* an older build never clobbers a newer config.
  - *Check (test):* a test loads a too-new `settings.json` and finds the backup and the defaults.
  - *Reference:* Editora `ConfigVersioningTest` (`tooNewSettingsAreBackedUpAndDefaultsLoaded`).
- `SET-11` `MUST` Never trust a config file. Clamp every value to its valid range when it is **read**, not only when it is written; an unknown value falls back to its default; a file that cannot be decoded is kept as a backup (Editora names it `<name>.corrupt.bak`) and defaults load. A bad file never stops the app from starting.
  - *Why:* config is plain JSON (or properties) and people edit it by hand. A hand-edited two-million-line scrollback must not exhaust memory at the next launch (Termina).
  - *Check (test):* a test loads out-of-range numbers and an unparseable file, including one that makes the parser throw something other than an `IOException` (a malformed `\uXXXX` escape in a `.properties` file).
  - *Reference:* Editora `config/migration/ConfigMigrations.java` (`keepCorrupt`), `ConfigLoadProblemsTest` (`outOfRangeNumbersAreClampedOnLoad`, `anUnparseableFileIsReportedWithItsBackup`); Termina `config/Settings.java`, `SettingsTest` (`aMalformedEscapeNoLongerStopsTheAppStarting`). Torrenta, Modula, Mirada and nux keep no backup, and in Torrenta and nux a malformed escape stops the app from starting.
- `SET-12` `MUST` Export configuration writes a timestamped `.zip` of the config folder; Import restores from one.
  - *Check (test):* tests over the zip's name and contents, and an import that replaces only the ticked parts.
  - *Reference:* Editora `config/ConfigExporter.java`, `config/ConfigImport.java`, `ConfigExporterTest`, `ConfigImportTest`.

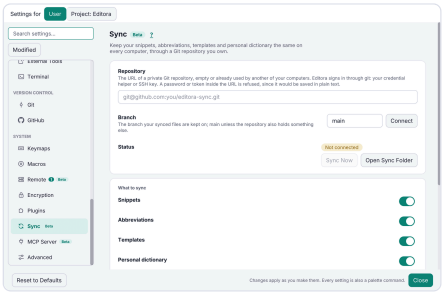
Secrets stay out of files

- `SET-13` `MUST` No secret goes in a config file. API keys and tokens go to the operating system's keychain: macOS Keychain, Windows Credential Manager, the Secret Service on Linux. Calls may block on a prompt, so they run off the FX thread.
  - *Check (code):* no key or token field in the settings model; reads and writes go through the secret store.
  - *Reference:* Editora `secrets/SecretStore.java`, `config/ApiKeyVault.java`. Editora doesn't yet: `Settings` keeps the AI keys as a fallback for when no keychain answers, and saving a key from Settings writes the keychain on the FX thread. Torrenta keeps its proxy password, Torznab key and remote-profile passwords in plain text.
- `SET-14` `MUST` A credential the app can ask for again lives in memory for the session only and is never written. Work that needs it waits until the user has signed in this session; do not "fix" that by storing the password.
  - *Check (code):* the credential is a field of a runtime object, absent from every config class.
  - *Reference:* nux's UPS login (`app/CommandRunner.java`).
- `SET-15` `MUST` A secret in the wrong place is refused. A repository URL with a password or token in it is refused, with the fix (Settings Sync); a problem report redacts tokens, credentials, e-mail addresses, user names and home paths before anyone sees it (Diagnostics).
  - *Check (by hand):* paste `https://user:token@host/repo.git` as the sync repository.
  - *Reference:* Editora `ui/SettingsSyncPage.java` (`settings.sync.repo.credentials`), `sync/SyncEngine.java` (`hasCredentials`, checked again at sync time), `report/Redactor.java`.
- `SET-16` `MUST` An option that weakens security has no UI: it exists only as a key in the config file, so nobody turns it on by clicking.
  - *Check (code):* no Settings row or command writes such a key.
  - *Reference:* nux's trust-all TLS (`tlsTrustAll` in `config/Config.java`). Editora doesn't yet: *Require registry signature* for plugins is a Settings checkbox and the `plugins.toggleRequireSignature` command.

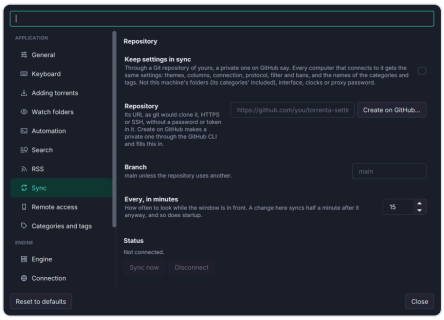
Settings Sync (Git / GitHub)

- `SET-17` `SHOULD` Settings, keybindings and user data can sync, optionally, through a Git repository the user owns.
  - *Check (by hand):* set up Sync on two machines and change a setting on one.
  - *Reference:* Editora `sync/SyncEngine.java`, `docs/subsystems/settings-sync.md`. Editora syncs key bindings and user data (snippets, abbreviations, templates, the dictionary), not the rest of `settings.json`; Torrenta syncs its settings file (`sync/SyncEngine.java`).
- `SET-18` `SHOULD` Setup is by pasting a repo URL, or **Create on GitHub** through the `gh` CLI. The account that can read the repo is pinned per clone, so machines with two GitHub accounts work.
  - *Check (test):* a test remembers the first account that can read the repository and uses it without asking again.
  - *Reference:* Editora `sync/SyncAccount.java`, `SyncAccountTest`, `ui/SettingsSyncPage.java`.
- `SET-19` `SHOULD` Sync runs automatically (on change and at start) and on demand. Machine-local keys (window size, paths) never sync.
  - *Check (code):* one pure class decides which keys or files travel, and machine-local ones are not among them.
  - *Reference:* Editora `ui/SettingsSync.java` (after startup, after a local change, on an interval, and `sync.now`), `sync/SyncCategory.java`; Torrenta `sync/SyncKeys.java`.
- `SET-20` `SHOULD` On a same-entry conflict *this machine wins*, and a backup is taken before any sync that replaces local files.
  - *Check (test):* a merge test where both sides changed one entry keeps this machine's.
  - *Reference:* Editora `sync/SyncMerge.java`, `SyncMergeTest`.
- `SET-21` `SHOULD` Sync status shows as a pill in Settings and as echo messages; failures go to the Message Log.
  - *Check (by hand):* point Sync at a repository that does not exist and read the pill and the log.

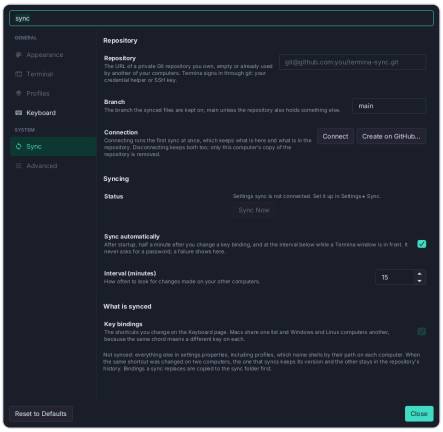
- *Reference:* Editora `ui/SettingsSyncPage.java`, `SyncStatus`.



Editora's Sync page: repository, branch, Create on GitHub... and what to sync.



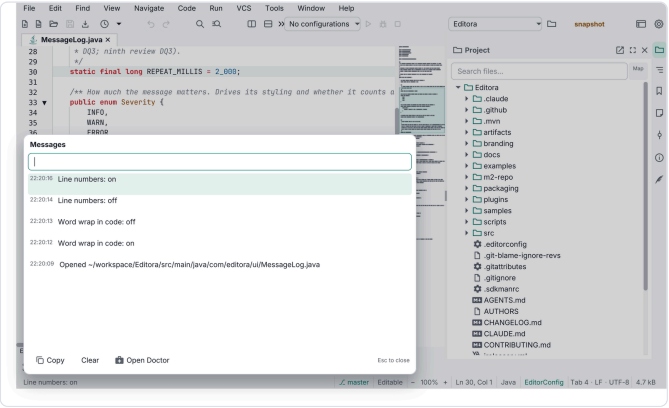
Torrenta's Sync page.



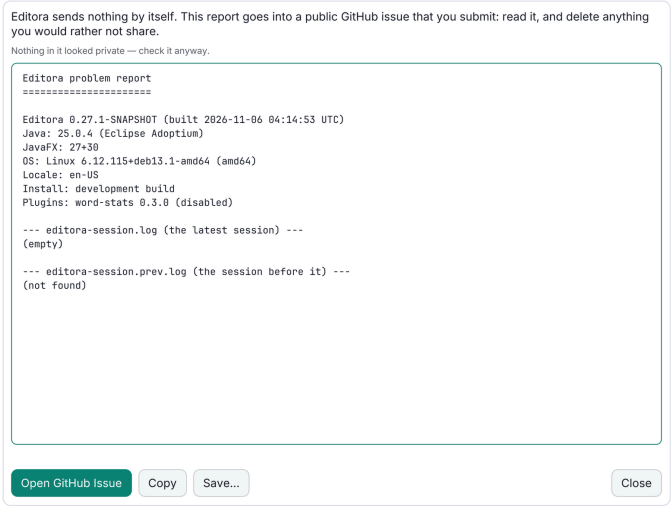
Termina's Sync page.

Diagnostics and support

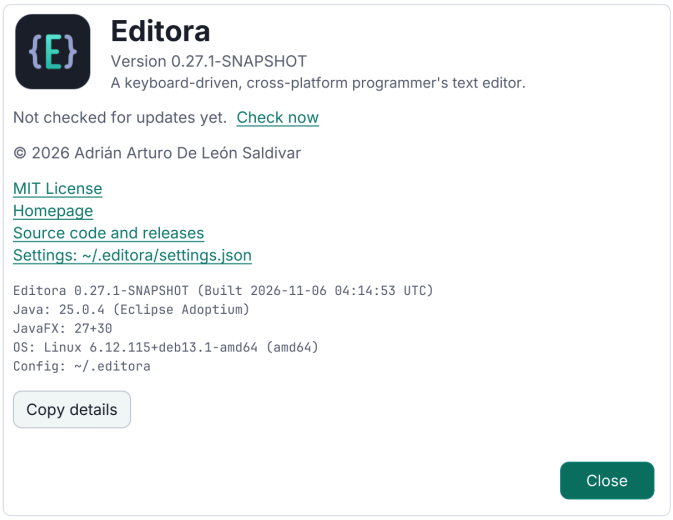
- `DIAG-01` `MUST` The **Message Log** is a session-only, bounded history of every echo-area message with `HH:mm:ss` times, newest first. Clicking the status message opens it; Esc closes it; Clear empties it.
  - *Check (test):* a test records messages newest first, evicts the oldest past the cap, and clears.
  - *Reference:* Editora `ui/MessageLog.java` (`MessageLogTest`), `ui/MessageLogPopup.java` .
- `DIAG-02` `MUST` The **Debug Log** captures all `java.util.logging` records and uncaught exceptions in memory (bounded) **and** mirrors them to `<app>-session.log` in the config folder, keeping the previous one as `<app>-session.prev.log` so a crash is still visible next launch.
  - *Check (test):* tests bound the buffer and find the session file written and the previous one kept.
  - *Reference:* Editora `ui/DebugLog.java` (`DebugLogTest` , `DebugLogSessionFileTest`).
- `DIAG-03` `MUST` The Debug Log opens in a modeless window with Refresh, Copy, Clear, Export and Report a Problem.
  - *Why:* it is the only way to see logs in a packaged build.
  - *Check (by hand):* open the Debug Log and keep working in the main window while it is open.
  - *Reference:* Editora `ui/DebugLogWindow.java` .
- `DIAG-04` `MUST` An **uncaught exception handler** is installed at the start of `main` , before the toolkit starts, and routes to the Debug Log and an echo message. Never a stack trace dialog.
  - *Check (code):* `main` calls the install method before any argument handling or `launch` , and the handler shows no dialog.
  - *Reference:* Editora `DebugLog.install()` . Editora, Torrenta and Termina log the exception but do not yet post an echo message.
- `DIAG-05` `MUST` The **update notification** reads GitHub `/releases/latest` once per app session (not per window), at most once a day, a few seconds after the first frame. A newer version shows a status-bar notice and a line in About; "Check for Updates" runs it by hand; Settings has an off switch. It never downloads or installs by itself.
  - *Check (test):* a test waits for the first paint and the delay, then checks once a session and not when off or not due.
  - *Reference:* Editora `update/UpdateCheck.java` , `ui/UpdateCheckScheduler.java` (`UpdateCheckSchedulerTest`), `ui/UpdateCoordinator.java` .
- `DIAG-06` `MUST` Help → **Report a Problem** builds a report (version, OS, runtime, settings summary, recent Debug Log), **redacts** it (home paths, user names, e-mail addresses, credentials, tokens), shows it editable, copies it to the clipboard and opens a prefilled GitHub issue from `bug_report.yml` . Over about 7,500 characters the link falls back to "paste from the clipboard". `<app> --report` does the same from the command line for when the app cannot start. Nothing is uploaded.
  - *Check (test):* redaction tests cover home paths, user names, e-mail addresses, credentials in URLs and tokens; a report test puts a short report in the link and leaves the paste hint for a long one.
  - *Reference:* Editora `report/ProblemReport.java` , `report/Redactor.java` , `report/ReportCommand.java` , `ui/ReportProblemWindow.java` (`RedactorTest` , `ProblemReportTest`).
- `DIAG-07` `SHOULD` A **Doctor** page checks the environment (external tools, versions, permissions, paths), with a status per row and a fix or link for each failure.
  - *Check (by hand):* open the Doctor with a tool missing and find its row failed, with what to install.
  - *Reference:* Editora `doctor/*` , `ui/DoctorPane.java` .
- `DIAG-08` `SHOULD` An app with user documents snapshots unsaved work off the FX thread and offers it back after a crash.
  - *Check (test):* a test finds an abandoned session's snapshots and offers them to the first window only.
  - *Reference:* Editora `recovery/RecoveryService.java` (`RecoveryServiceTest` , `CrashRecoveryFxTest`).



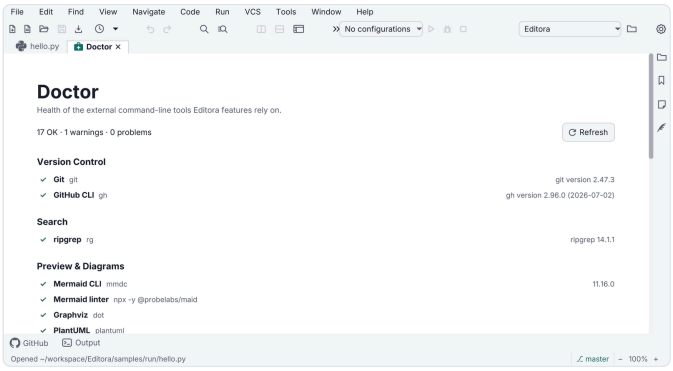
Clicking the status message opens the Message Log: this session's messages, newest first.



Report a Problem: a redacted, editable report that opens as a prefilled GitHub issue.



About: version, runtime, license, links and the update status.



Doctor: each external tool with its version, or what to install.

Secrets never reach a log, a report or a config file: see Secrets stay out of files.

Command line and launch

Flags

- **CLI-01** `MUST` `--help` and `--version` print before the GUI starts, in plain English, not translated.
  - *Check (code):* `main` handles both and exits before `launch`, printing fixed English text rather than catalog messages.
  - *Reference:* Editora `App.main`, `App.helpText`; `AppArgsTest`.
- **CLI-02** `MUST` `--config-dir <dir>` uses another config folder, and `--dev` uses `~/.<app>-dev`.
  - *Check (code):* the config folder is resolved from both flags before anything reads `~/.<app>`.
  - *Reference:* Editora `App.main` (`ConfigManager.configDirFor`, `App.devFlag`).
- **CLI-03** `SHOULD` `--new-instance` starts a second process instead of handing off to the running one.
  - *Check (code):* the flag skips the single-instance hand-off.
  - *Reference:* Editora `App.newInstanceFlag`.
- **CLI-04** `SHOULD` `--no-session` does not restore the last session.
  - *Check (by hand):* launch with `--no-session` and find only the items given on the command line.
  - *Reference:* Editora `App.helpText`.
- **CLI-05** `MUST` `--report` writes a redacted problem report and opens the GitHub issue, without starting the GUI.
  - *Check (test):* a test runs the report command and finds the redacted report written.
  - *Reference:* Editora `report/ReportCommand.java` (`ReportCommandTest`).
- **CLI-06** `SHOULD` `[files…]` opens these items.
  - *Check (by hand):* launch with two files and find both open.
  - *Reference:* Editora `App.fileTargets`.

Launch

- **CLI-07** `MUST` One instance runs per config folder. A second launch hands its arguments to the running app over a local endpoint and exits; on macOS, `openFiles` Apple Events reach the running app.
  - *Check (test):* a test makes the first launch primary and delivers a second launch's arguments to it.
  - *Reference:* Editora `ipc/SingleInstance.java` (`SingleInstanceTest`), `MacOpenFiles.java`.
- **CLI-08** `SHOULD` An advisory lock tells two processes on one config folder about each other.
  - *Check (test):* a test takes the lock from two processes.
  - *Reference:* Editora `config/InstanceLock.java` (`InstanceLockTest`).
- **CLI-09** `SHOULD` `main` sets `java.awt.headless=true` before any AWT class loads.
  - *Why:* on macOS, AWT's native toolkit and JavaFX contend for the one AppKit run loop and can hang; headless Java2D still draws to images.
  - *Check (code):* `System.setProperty("java.awt.headless", "true")` is among the first statements of `main`.
  - *Reference:* Editora `App.main`.
- **CLI-10** `SHOULD` External tools are resolved through the login shell's `PATH`.
  - *Why:* a `.app` launched from Finder inherits a stripped `PATH`.
  - *Check (code):* every process the app starts gets a `PATH` that includes the login shell's.
  - *Reference:* Editora `process/ProcessRunner.java` (`augmentedPath`).

Internationalization

Reference: Editora's `i18n/Messages.java` (`tr` , and `available()` behind the Settings language picker), the catalogs in `src/main/resources/com/editora/i18n/` , and `i18n/MessagesTest.java` , which holds them in step.

Languages

Every app ships **at least these six**. They are a floor, not the set: an app `SHOULD` add languages as people ask for them, and a new app can start with more.

Language	Catalog
English (base)	`messages.properties`
Spanish	`messages_es.properties`
Portuguese	`messages_pt.properties`
French	`messages_fr.properties`
Italian	`messages_it.properties`
German	`messages_de.properties`

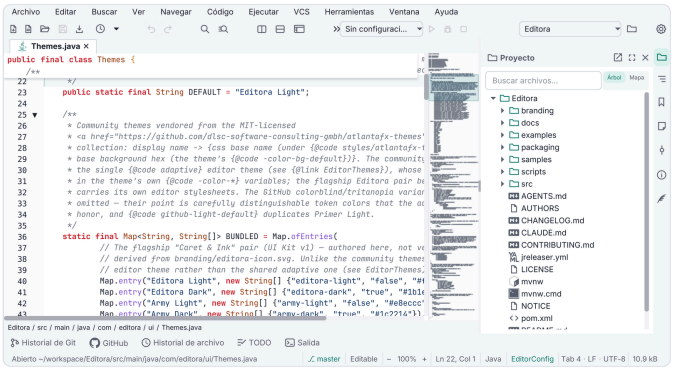
- `INTL-01` `MUST` The app ships a message catalog for each of the six languages above, English as the base.
  - *Check (code):* the six `messages*.properties` files are in the app's `i18n` resources.
  - *Reference:* Editora `src/main/resources/com/editora/i18n/` .

Adding a language:

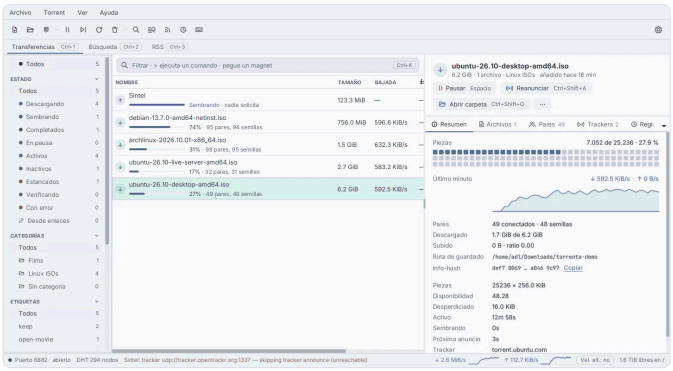
- `INTL-02` `SHOULD` A new language is a `messages_<code>.properties` with every key, listed in the Settings language picker and in the parity test.
  - *Check (code):* the language's code is in `Messages` ' language list (which the picker shows) and in the catalog list the parity test walks.
  - *Reference:* Editora `i18n/Messages.java` (`LANGUAGES` , `available()`); `MessagesTest` 's `LOCALES` and `ALL_CATALOGS` , after which `translationCatalogsHaveExactlyTheBaseKeySet` holds it to the same keys as the others.
- `INTL-03` `SHOULD` Every value is translated for real: no English value is shipped in another language's catalog.
  - *Check (code):* compare each catalog's values with the base; a value identical to the English one is a name, a term or a bare pattern that stays the same in that language ("Git: Pull (Rebase)", "{0} · {1}"), never an untranslated sentence. No test catches it, so it is also checked in review.
  - *Reference:* Editora `messages_*.properties` .
- `INTL-04` `SHOULD` The layout holds in every language at 1024 px wide: rows wrap, never cut. German is today's longest.
  - *Check (by hand):* run the app in each language at 1024 px wide (see Quality).
  - *Reference:* Editora `messages_de.properties` .
- `INTL-05` `SHOULD` A Chinese, Japanese or Korean catalog comes with a font fallback for its script.
  - *Why:* Inter has no CJK glyphs.
  - *Check (by hand):* run the app in that language and look for missing-glyph boxes.
  - *Reference:* no app yet.
- `INTL-06` `SHOULD` An Arabic or Hebrew catalog sets `NodeOrientation.RIGHT_TO_LEFT` on every scene.
  - *Check (code):* the scene setup sets the node orientation from the current language.
  - *Reference:* no app yet.

Rules

- `INTL-07` `SHOULD` No raw English literal is passed to a control: every string comes from `tr("some.key")` in one `Messages` class.
  - *Check (test):* a test scans the sources for `tr("…")` calls and fails when a key is missing from the base catalog.
  - *Reference:* Editora `i18n/Messages.java` , `MessagesTest.everyLiteralKeyUsedInTheSourcesExists` .
- `INTL-08` `MUST` Every catalog has every key of the base catalog, and no more, each with a non-empty value.
  - *Check (test):* a parity test fails the build when a key is missing, extra or empty in any catalog.
  - *Reference:* Editora `MessagesTest.translationCatalogsHaveExactlyTheBaseKeySet` , `translationsAreNonEmptyAndDistinctFromKeys` .
- `INTL-09` `SHOULD` In a value with a placeholder, a literal apostrophe is doubled; in a value without one, it is written once.
  - *Why:* a value with a placeholder is a `MessageFormat` pattern, where a lone apostrophe starts a quoted section and can swallow the placeholder after it; a value without one is shown exactly as written.
  - *Check (test):* catalog tests fail a lone apostrophe beside a placeholder and a doubled one without.
  - *Reference:* Editora `MessagesTest.aValueWithAPlaceholderNeverHasALoneApostrophe` , `aValueWithoutAPlaceholderNeverDoublesAnApostrophe` .
- `INTL-10` `SHOULD` Identifiers such as line numbers and ports use `{0,number,#}` , so they are not digit-grouped.
  - *Check (test):* a test formats each identifier argument in every locale and fails on a grouped number.
  - *Reference:* Editora `MessagesTest.identifierArgumentsAreNeverDigitGrouped` .
- `INTL-11` `SHOULD` Technical identifiers and the `--help` and `--version` text stay untranslated, outside the catalogs.
  - *Check (code):* the usage text is a literal in the launcher, not a `tr` call.
  - *Reference:* Editora `App.java` (the usage text).



Editora in Spanish: menus, tool windows and the status bar all come from the catalogs.



Torrenta in Spanish.

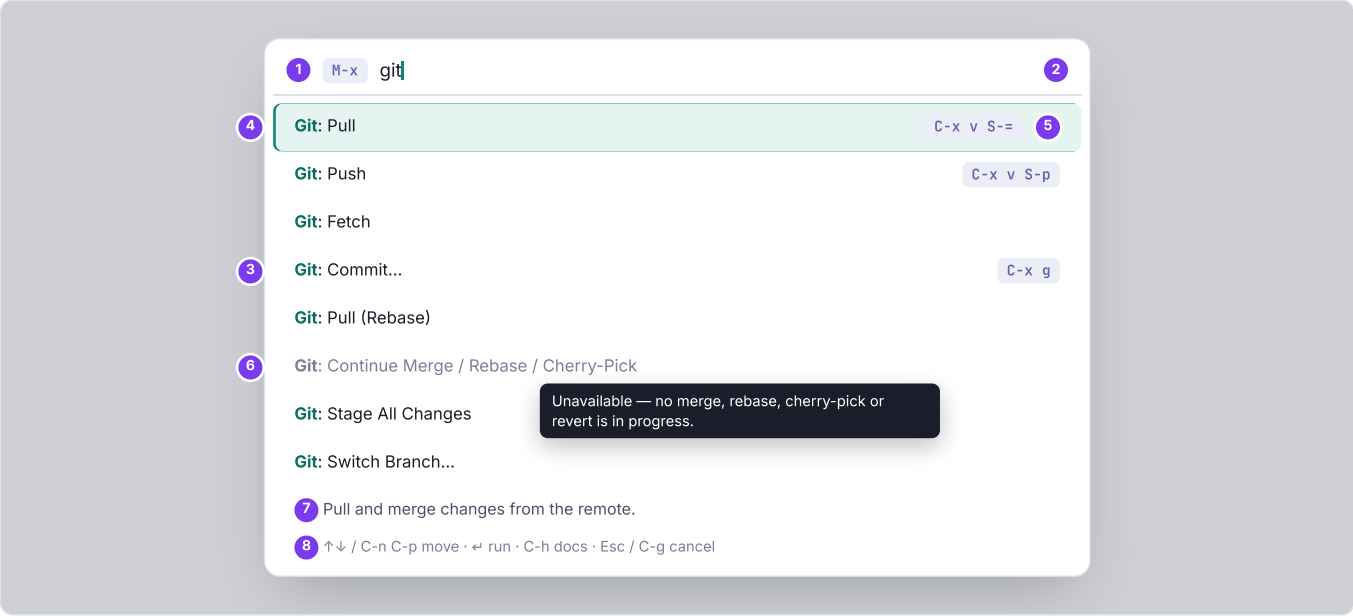
Command Palette

One palette per window lists every registered command, so a command that exists is a command people can find. The pure parts (`FuzzyMatch` , `MatchText` , the key helper, `PickerKeys` in Editora and `PaletteKeys` in the copies, and the caret-key helper, `TextInputKeymap` in Editora and `TextInputKeys` in the copies) are shared with Termina and Mirada. Torrenta shares `MatchText` and ranks with its own `CommandMatcher` .

Rules

- **PAL-01** `MUST` Every app has one Command Palette per window, opened by `palette.show` : `M-x` in the Emacs keymap and `Ctrl+Shift+P` (`Cmd+Shift+P` on macOS) in the others.
  - *Check (code):* the window builds one `CommandPalette` , and each bundled keymap binds that chord to `palette.show` .
  - *Reference:* Editora `ui/CommandPalette.java` , built once in `MainController` ; `keymaps/emacs.json` (`M-x`), `cua.json` and `vscode.json` (`C-S-p`), `cua.mac.json` (`Cmd-S-p`).
- **PAL-02** `MUST` The palette lists every registered command, with its title from `command.<id>` and its description from `command.<id>.desc` .
  - *Check (code):* the palette's rows come from the whole registry, filtered only by the query; a command that cannot run stays listed, grayed. A catalog parity test fails on a command title with no `.desc` .
  - *Reference:* Editora `CommandPalette.filter` (`registry.all()`), `MessagesTest` ; the Spec table below.
- **PAL-03** `MUST` The palette is an in-scene overlay in the window's root `StackPane` , over a faint scrim, not a `javafx.stage.Popup` .
  - *Why:* on Windows a popup does not reliably take keyboard focus.
  - *Check (code):* the palette shows its card through the overlay host installed in the root `StackPane` ; no `Popup` is created for it.
  - *Reference:* Editora `ui/OverlayHost.java` (`install`), `CommandPalette.show` .

Anatomy



1. **Opener chip.** The chord that opened the palette, read from the active keymap. No chip when the user has unbound it, never a wrong one.
2. **Query field.** The first row is the input; there is no title. Prompt "Run command...". Emacs caret keys work inside it.
3. **Match highlight.** The characters the fuzzy matcher scored, bold in the accent color. The highlight and the ranking come from the same call, so they never disagree.
4. **Selected row.** Tinted background plus a 2 px accent bar on its inside edge. The bar is what makes the selection readable in a long list.
5. **Key chip.** The command's binding in the active keymap, in the keyboard color (brace periwinkle). Absent when unbound.
6. **Unavailable command.** Listed, grayed, never selected and inert to clicks. Its tooltip says why and what to do ("Unavailable — no merge, rebase, cherry-pick or revert is in progress.").
7. **Description.** The selected command's `command.<id>.desc` , one line with an ellipsis, always one line tall so the card never jumps.
8. **Key legend.** The palette's own keys, rendered from the active keymap.

Spec

Property	Value	Reference
Kind	In-scene overlay in the window's root `StackPane` , over a faint scrim. **Not** a `javafx.stage.Popup` : on Windows a popup does not reliably take keyboard focus.	`OverlayHost`

Property	Value	Reference
Width	620 px	`content.setPrefWidth(620)`
Rows	36 px high, 8 visible before the list scrolls	`ROW_HEIGHT` , `VISIBLE_ROWS`
Card	`-color-bg-overlay` , 1 px `-color-border-default` , radius 12, padding 6, one wide drop shadow	`.command-palette` in `app.css`
Row text	12.5 px; padding 7 × 16	`.command-palette .list-cell`
Scrim	`rgba(0,0,0,0.18)` ; a click on it closes the palette	`.overlay-backdrop`
Content	Every registered command: title from `command.<id>` , description from `command.<id>.desc`	`CommandRegistry` , `MessagesTest`
Ranking	Fuzzy score over title, id and description; a query that names a title outranks a fuzzy hit	`FuzzyMatch` , `titlePhraseScore`

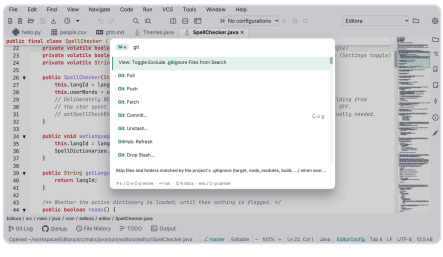
Behavior

- **PAL-04** `SHOULD` The palette opens from the `palette.show` command, by its chord or a menu item, with an empty query that lists every command.
  - *Check (code):* `show` clears the field and filters on the empty query before the card appears.
  - *Reference:* Editora `CommandPalette.show` .
- **PAL-05** `SHOULD` Typing filters the list, and the first *enabled* match is selected.
  - *Check (test):* a test with a disabled command at the top finds the cursor on the first enabled row.
  - *Reference:* Editora `CommandPalette.selectFirstEnabled` , `CommandPaletteDisabledFxTest` .
- **PAL-06** `SHOULD` ↑ ↓, `C-n` `C-p` and Page Up/Down move the selection; Enter, or a click on an enabled row, runs the command. The palette closes first, then the command runs.
  - *Check (code):* the run path hides the card before it calls the registry, and does nothing on a disabled row.
  - *Reference:* Editora `CommandPalette.runSelected` , `PickerKeys` .
- **PAL-07** `SHOULD` Esc, `C-g` or a click on the scrim cancels, and focus returns to where it was.
  - *Check (code):* the cancel key and a press on the backdrop both hide the overlay, and hiding focuses the node that had focus before it opened.
  - *Reference:* Editora `PickerKeys` (`CANCEL`), `OverlayHost` (`previousFocus`).
- **PAL-08** `SHOULD` `C-h` , or the first of `C-h` , `F1` , `Shift+F1` the active keymap leaves free, opens the selected command's page on the website, for the running version.
  - *Check (code):* the docs chord is the first free one of the three, and the URL is built from the app's versioned docs address.
  - *Reference:* Editora `CommandPalette` (`PickerKeys.freeChord` , `docsUrl`), `AppInfo.docsUrl` .
- **PAL-09** `SHOULD` A live keymap switch re-renders the key chips, the opener chip and the key legend.
  - *Check (code):* applying a new keymap calls the palette's rebind method, which rebuilds the opener, the legend and the rows.
  - *Reference:* Editora `CommandPalette.refreshBindings` , called from `SettingsReapply` .

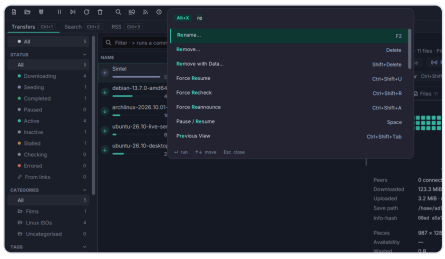
Copy

Key	English
`palette.prompt`	Run command…
`palette.disabled.<reason>`	Unavailable — *what is missing. What to do first.*

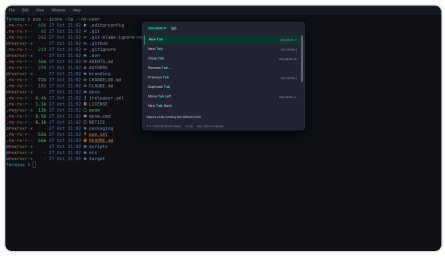
In the apps



Editora.



Torrenta.



Termina.

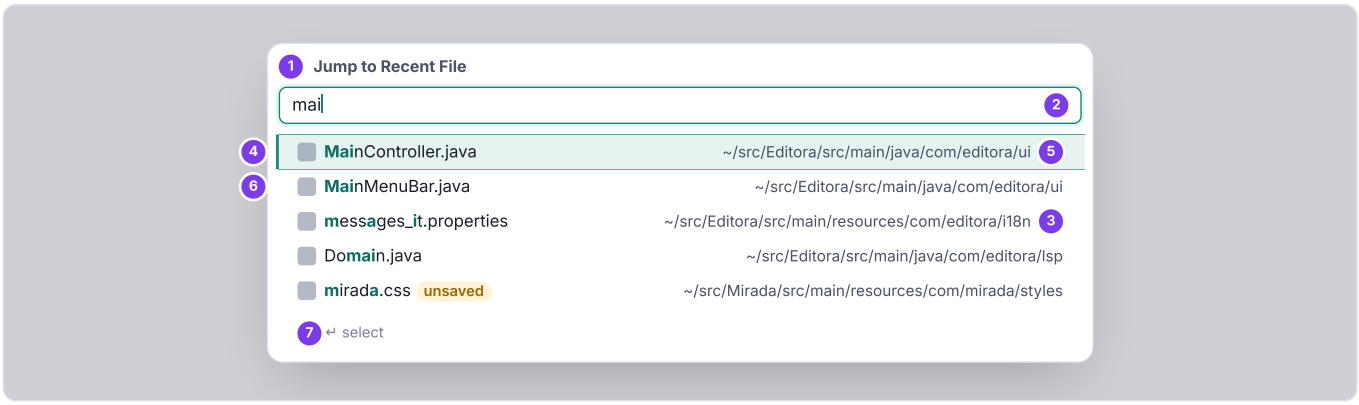
QuickOpen picker

The picker for every "choose one from a list" moment: recent files, themes, keymaps, a setting's values, a branch to compare with. It is the Command Palette card with a title over the field and a prompt in it, so a list choice is made the way a command is run: type a few letters, Enter. The keyboard page states the rule; this page is the shape.

Rules

- `QOPEN-01` `SHOULD` One picker serves every list choice: recent files, themes, keymaps, a setting's values, a branch to compare with.
  - *Check (code):* those choices are instances of one generic picker class; there is no second list-picker class or `ChoiceDialog` for them.
  - *Reference:* Editora `ui/QuickOpen.java` (one generic class, instantiated more than seventy times), driven by `PickerKeys`.
- `QOPEN-02` `SHOULD` The picker is the palette's card in the shared in-scene overlay host, with a title naming the command over the field and a prompt in the field; never a `javafx.stage.Popup`.
  - *Check (code):* the picker shows its card through the overlay host and sets a title and a prompt text.
  - *Reference:* Editora `QuickOpen.setOverlayHost`, `ui/OverlayHost.java`; the Spec table below.

Anatomy



1. **Title.** The command that opened the picker ("Jump to Recent File", "Set App Theme"), bold 12 px in the muted color. This is what tells the picker apart from the palette, which has no title.
2. **Query field** with a prompt per picker: "Type to filter recent files...", "Type to filter themes...". The Emacs caret keys work in it (`TextInputKeymap.installShared`).
3. **Match highlight.** The characters the fuzzy matcher scored, bold in the accent color; the same `MatchText.runs` as the palette, so the highlight and the rank never disagree.
4. **Selected row.** The first match, or with nothing typed the value in force when the picker knows it (`setCurrentItem`), so a settings picker opens on the current value.
5. **Detail**, right-aligned and muted: the file's folder, a branch's upstream, "current". It gives way with an ellipsis; the title keeps its width and never wraps.
6. **Icon** (optional, `setItemIcon`): a file-type glyph from `FileIcons`. A per-row class marks state (an unsaved buffer in amber).
7. **Key legend.** "↵ select" plus any row action's key, rebuilt from the active keymap on every showing.

Spec

Property	Value	Reference
Kind	The palette's card in the shared in-scene `OverlayHost`, over the same scrim; never a `javafx.stage.Popup`	`QuickOpen.setOverlayHost`, `OverlayHost`
Width	620 px; a picker may ask for its own width and a minimum number of rows (a registry browser with one entry still gets a roomy box)	`DEFAULT_WIDTH`, `setPreferredSize`
Rows	26 px fixed cells; the list hugs its match count, at least 1 row, 12 before it scrolls	`CELL_HEIGHT`, `MAX_VISIBLE`, `resizeList`
Title	Bold 12 px, `-color-fg-muted`, over the field	`.command-palette .palette-title`
Row	Title as matched runs, an optional icon before it, the detail in the palette's muted right-detail style	`ItemCell`, `.keybinding`
Legend	11 px muted under the list	`.command-palette .palette-hint`
Card	`-color-bg-overlay`, radius 12, 1 px `-color-border-default`, padding 6: the palette's	`.command-palette`
Ranking	`FuzzyMatch` over the search key (the label unless the picker names another, a note's body for instance); equal scores keep the supplier's order	`filter`

Behavior

- **QOPEN-03** `SHOULD` The items are read fresh on every showing and snapshotted, so typing filters a stable list. A source that answers late (an outline the language server has not returned yet) calls `reload()`.
  - *Check (code):* `show` calls the items supplier; `reload` re-reads it and keeps the query.
  - *Reference:* Editora `QuickOpen.show`, `QuickOpen.reload`.
- **QOPEN-04** `SHOULD` With nothing typed the list keeps the supplier's own order (recency for recent files, document order for symbols), with the current value pre-selected when there is one.
  - *Check (code):* the empty query passes the items through unsorted and selects the `setCurrentItem` value.
  - *Reference:* Editora `QuickOpen.filter`, `setCurrentItem`.
- **QOPEN-05** `SHOULD` Typing ranks by fuzzy score, and ties keep the supplier's order, so rows do not shuffle under the cursor between keystrokes.
  - *Check (code):* the sort's last tiebreak is the supplier's index.
  - *Reference:* Editora `QuickOpen.filter`.
- **QOPEN-06** `SHOULD` The active keymap's line and page chords (`nav.lineDown`, `nav.pageUp`, ...) move the selection, as do ↑ ↓, Page Up/Down and `Ctrl+Home` / `Ctrl+End` (`Cmd` on macOS), which always work; plain Home and End stay with the field's caret. Enter or a click on a row chooses; the card hides first, then the choice runs, so focus is already back where it belongs.
  - *Check (code):* the accept path hides the card before it calls the choice handler.
  - *Reference:* Editora `QuickOpen` (`hide()` then `onChoose`), `PickerKeys.resolve`.
- **QOPEN-07** `SHOULD` Esc, `C-g` or a click on the scrim cancels, and the host restores focus. A picker that previewed something undoes it.
  - *Check (test):* a test previews a row, dismisses the picker and finds the preview undone.
  - *Reference:* Editora `OverlayHost` (`previousFocus`), `QuickOpenPreviewFxTest` (`dismissingRestoresWhatWasPreviewed`).
- **QOPEN-08** `MAY` A picker previews the highlighted row as the selection moves and undoes the preview on cancel (Recent Locations shows each place and returns to the origin on Esc). A preview without an undo is worse than none, and nothing is previewed for the row the picker opens on, only for one the user moved to. The theme and keymap pickers do not preview; they apply on Enter.
  - *Check (test):* opening previews nothing, moving previews, choosing does not restore.
  - *Reference:* Editora `QuickOpen.setPreview(onPreview, onCancelled)`, `QuickOpenPreviewFxTest`.
- **QOPEN-09** `MAY` A row action acts on the highlighted row and keeps the picker open (the breakpoints picker enables or removes one), re-reading the list afterwards and keeping the cursor in place.
  - *Check (test):* a row action runs on the highlighted row and the picker stays up.
  - *Reference:* Editora `QuickOpen.addRowAction(key, legend, action)`, `QuickOpenPreviewFxTest` (`aRowActionActsOnTheHighlightedRowAndKeepsThePickerOpen`).
- **QOPEN-10** `SHOULD` A picker that can open on an empty list shows a placeholder sentence in it, never a blank card. A picker with nothing to offer may instead say so in the status line and not open.
  - *Check (code):* each picker whose source can be empty either calls `setPlaceholder` or returns before `show` with a status message.
  - *Reference:* Editora `QuickOpen.setPlaceholder` (`BreakpointsPicker`); `NavigationCoordinator` (`status.nav.noRecentLocations`).
- **QOPEN-11** `SHOULD` The host shows one card at a time; opening a picker replaces whatever card was up.
  - *Check (code):* the host's `show` removes the previous card before it adds the new one, and keeps the original focus owner.
  - *Reference:* Editora `OverlayHost.show`.
- **QOPEN-12** `SHOULD` The picker's relatives move, accept and cancel with the same keys as the picker.
  - *Check (code):* each relative maps its keys through `PickerKeys`.
  - *Reference:* Editora `Switcher`, `FileFinder`, `MultiSelectPicker`, `BranchPopup`, `GitStashPopup`.

The relatives: the Open Files `Switcher` lists tabs in tab order, opens on the current one and closes a file with `C-d` or Backspace. `FileFinder` is the Emacs `find-file` field with Tab completion. `MultiSelectPicker` ticks several rows with Space where exactly one would be wrong. `BranchPopup` and `GitStashPopup` are sectioned pickers with their own actions.

Uses

App	Picked with it	Reference
Editora	Recent files (with file icons), recent locations (previewed), bookmarks, the structure outline, tool windows, open files, projects, keymap, app and editor theme, spell language, a setting's values (opened on the current value), branches, tags and commits to compare with, remotes, commit message history, the kill ring, Occur lines, external tools, macros, the debug main class, breakpoints, agent models	`NavigationCoordinator`, `EditorSettingsCoordinator`, `DiffCoordinator`, `DebugCoordinator`, `BreakpointsPicker`
Mirada	Go to bookmark..., Open recent folder...: the palette shown over a list of names with a detail per row and an "open" legend	`CommandPalette.Choices`, `ViewerWindow`
Termina, Torrenta, Modula, nux	Commands only; profiles and other lists are menus or combo boxes today	`CommandPalette` in each

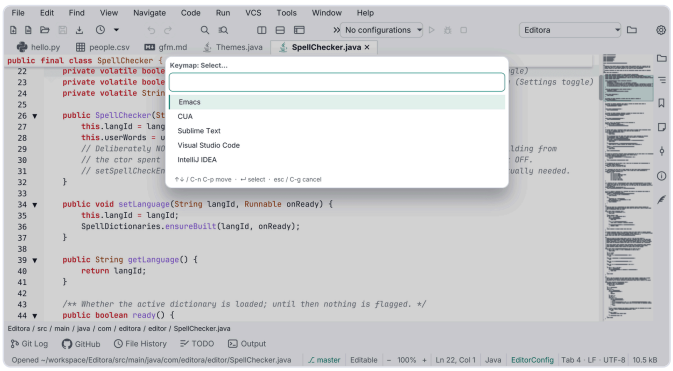
Copy

Key	English
`nav.recentFiles.title` / `.prompt`	Jump to Recent File / Type to filter recent files...
`palette.theme.appTitle`, `palette.theme.editorTitle` / `palette.theme.prompt`	Set App Theme, Set Editor Theme / Type to filter themes...
`command.keymap.select` / `palette.keymap.prompt`	Keymap: Select... / Type to filter keymaps...

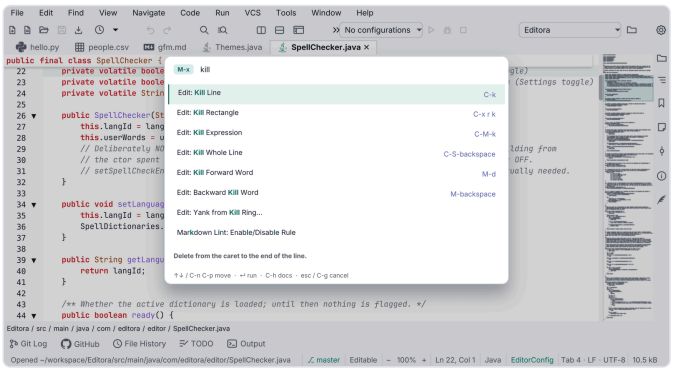
Key	English
`palette.setting.pick`	Type to filter…
`pickerKeys.select` , `pickerKeys.open` , `pickerKeys.cancel`	{0} select, {0} open, {0} cancel
`switcher.header`	Open Files

A title names the command; a prompt starts with "Type to filter" and ends with an ellipsis. A placeholder says what is missing and what to do ("No breakpoints set. Click a line number to add one.").

In the apps



Editora: Keymap: Select… over the editor, the title above the field.



The palette the picker is built from.

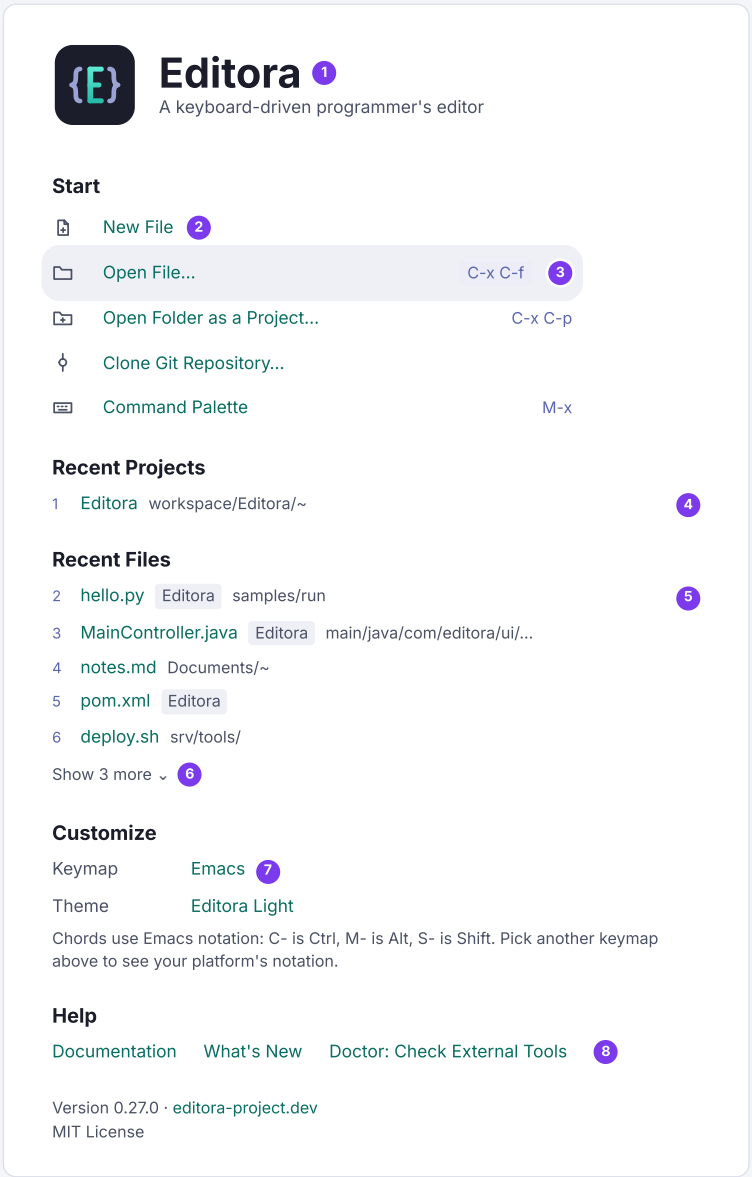
Welcome page

The page shown when there is no session to restore, and the answer to "what do I do now?". The text rules are in Editora's `ui/WelcomeText.java` and the strings under `welcome.*`.

Rules

- `WELC-01` `SHOULD` An app that opens files or projects shows a Welcome page when there is no session to restore.
  - *Check (code):* startup with nothing restored opens the Welcome page.
  - *Reference:* Editora `ui/WelcomePane.java`, `MainController.addWelcomeTab`.
- `WELC-02` `SHOULD` An app whose window *is* its content shows its own empty state instead, with the same ingredients: the ways in, each with its chord.
  - *Check (by hand):* start the app with nothing in it and read the empty state's keys.
  - *Reference:* Torrenta `ui/Welcome.java` (see In the apps below).

Anatomy



1. **Identity.** The app icon at 64 px beside the name (2.4 em, bold) and the README's one-line tagline, muted. One centered column, at most 640 px wide and never narrower than 340; a longer window scrolls.
2. **Start actions**, each an existing command: New File, Open File, Open Folder as a Project (only with Projects on), Clone Git Repository (only with Git on), and the Command Palette always last, the gateway to everything else. Accent links with no box; the icon sits in a 26 px column so every label starts at the same x, and the whole row answers a click.

3. **Chord**, right-aligned past a 32 px gap, in the keyboard color, from the active keymap. Unbound: no chord. Open File shows find-file's chord in the Emacs keymap, where that is the key to learn. The hovered row tints and the chord becomes a chip.
4. **Entry number.** The first nine recent entries, projects first, are numbered in the keyboard color; typing that digit opens the entry. Projects come in the order of their most recent file.
5. **Recent file row.** The name keeps its width; a chip names the project (or the host of a remote file); the folder is muted, relative to the project root, with `~` for home, and elides from its start so the part that differs stays visible. The full path is the row's tooltip; right-click offers Copy Path and Remove from recent files.
6. **Disclosure.** Five rows per list; the rest wait behind "Show N more", which becomes "Show fewer".
7. **Customize.** The active keymap and app theme by name; each link runs the picker command. Under an Emacs-style keymap a caption explains the notation. On a first run, with nothing recent, this block moves up under Start.
8. **Help and footer.** Documentation, What's New (the release notes) and the Doctor. Then the version, the home page and the license; a dev build adds its commit on its own line once the lookup, which runs `git` off the FX thread, has answered.

Spec

Property	Value	Reference
Kind	A real tab content in its own closeable "Welcome" tab, so the tab strip handles switching and closing; not an overlay	`WelcomePane implements TabContent`, `MainController.addWelcomeTab`
Column	640 px max, 340 px min, 48 px margin, centered in a scroll pane; below the minimum the page scrolls sideways	`COLUMN_WIDTH`, `MIN_COLUMN_WIDTH`, `MARGIN`
Zoom	Every px size is multiplied by the text zoom and the fonts are `em`, so zoom scales the layout, not only the words	`setFontScale`, `.welcome-*`
Surface	`-color-bg-default`, the UI surface, not the editor's; its tab takes the same fill	`.welcome-pane`, `.ui-surface-tab`
Header	Icon 64 px; title 2.4 em bold; tagline `-color-fg-muted`	`.welcome-title`, `.welcome-caption`
Section heading	1.15 em bold, padding 28 0 8 0	`.welcome-section`
Links	`Hyperlink` in `-color-accent-fg`, no border, underlined only on hover or keyboard focus; visited stays the accent	`.welcome-action`, `.welcome-link`
Action row	Icon column 26 px, gap 12 to the label, at least 32 px to the chord; hover fills `-color-bg-subtle` with 6 px radius, insets 8 px past the row	`WelcomePane.action`, `.welcome-action-row`
Chord	`-state-key`, 0.9 em; on hover a `-state-key-bg` chip, radius 4	`.welcome-key`
Recent row	Number `-state-key` 0.8 em; folder `-color-fg-muted` 0.9 em with a leading ellipsis; project chip muted on `-color-bg-subtle`, radius 3	`.welcome-entry-number`, `.welcome-recent-dir`, `.recent-project-name`
Lists	5 rows, then a flat disclosure button	`COLLAPSED_RECENT_LIMIT`, `addCollapsed`
Focus	The first recent entry, or New File when there is none, with no focus ring; Enter and Space open it	`requestFocus`, `focusPage`

Behavior

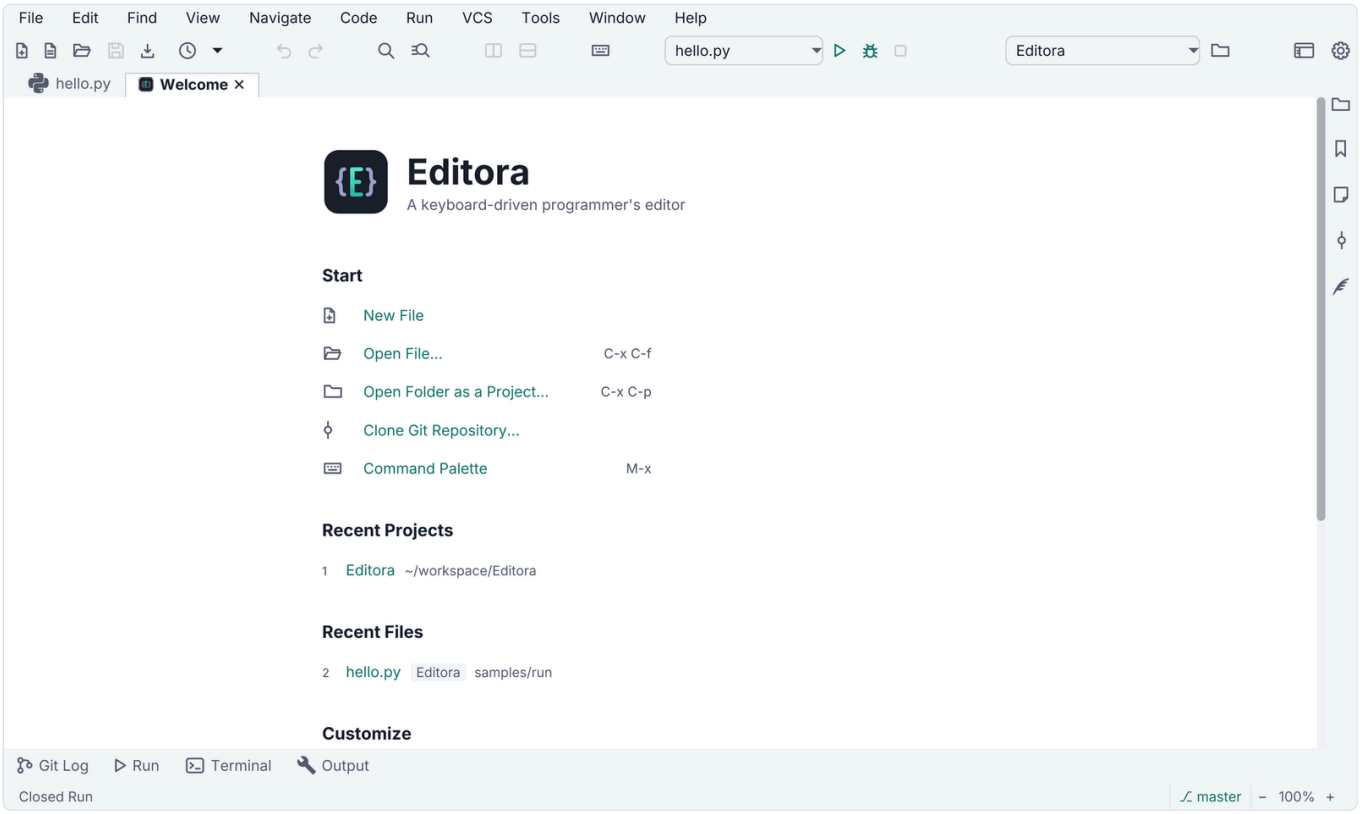
- `WELC-03` `SHOULD` The page is shown at startup when there is no session to restore (so always with `--no-session`), by the `view.welcome` command (Help → Welcome Page), and when the tab strip is empty after a project swap. Standalone diff startup suppresses it while its worker reads the files.
  - *Check (code):* those three paths open the one Welcome tab; the diff startup holds a flag that skips it.
  - *Reference:* Editora `MainController.addWelcomeTab`, `showWelcomeIfNoTabs`; `WindowSessionCoordinator` (`suppressWelcome`).
- `WELC-04` `SHOULD` Closing the last file tab does not bring the page back. The editor area shows its own quiet hint instead: Open File, Open Folder, Command Palette and Welcome Page, each with its chord, and takes focus so chords still reach the window.
  - *Check (test):* closing the last tab shows the hint and opening a tab hides it.
  - *Reference:* Editora `ui/EditorEmptyState.java`, `EditorEmptyStateFxTest`.
- `WELC-05` `SHOULD` The page is rebuilt on every show, so the recents, the Projects and Git toggles, the keymap and the theme are current. A refresh keeps the focus where it was.
  - *Check (code):* showing the page calls `refresh`, which re-requests focus when the page had it.
  - *Reference:* Editora `WelcomePane.refresh`.
- `WELC-06` `SHOULD` The recent lists offer no file that no longer exists and no project whose root is gone. Remote files are kept unchecked.
  - *Check (code):* the recent files pass through the same filter as the toolbar's dropdown; projects are kept only when their root is a directory.
  - *Reference:* Editora `RecentFiles.showable`, `WelcomePane`.
- `WELC-07` `SHOULD` Remove from recent files asks first, through the caller, then refreshes the page.
  - *Check (code):* the remover the page is given shows a confirmation before it removes the entry.
  - *Reference:* Editora `WelcomePane.setRecentRemover`; `MainController` (`dialog.removeRecent.*`).
- `WELC-08` `SHOULD` The digits 1 to 9 open the numbered entries: typed characters only, with no modifier, so chords and Enter are untouched.
  - *Check (code):* the digit handler is on `KEY_TYPED` and ignores Ctrl, Alt and Meta.
  - *Reference:* Editora `WelcomePane.onKeyTyped`.

- **WELC-09** `SHOULD` Saved SFTP sites appear as their own section; a click opens the connection form prefilled. The section is hidden when there are none.
  - *Check (code):* the page adds a Remote Sites section only when the saved-sites list is not empty.
  - *Reference:* Editora `WelcomePane` (`remoteSites` , `welcome.remote`).
- **WELC-10** `SHOULD` Customize runs `keymap.select` and `theme.setAppTheme` ; a change made there re-renders the page, chords included.
  - *Check (code):* the Customize links run those two commands.
  - *Reference:* Editora `WelcomePane` (`customizeRow`).
- **WELC-11** `SHOULD` The Welcome tab is not a document: its menu is Close only, with no Save, Rename or Pin, and the title bar shows the app or project name, not a file.
  - *Check (code):* the Welcome tab's context menu holds one Close item.
  - *Reference:* Editora `MainController.addWelcomeTab` .

Copy

Key	English
`welcome.tab`	Welcome
`welcome.tagline`	A keyboard-driven programmer's editor
`welcome.start`	Start
`welcome.newFile` / `welcome.openFile`	New File / Open File...
`welcome.openFolder` / `welcome.clone`	Open Folder as a Project... / Clone Git Repository...
`welcome.recentProjects` / `welcome.recentFiles` / `welcome.remote`	Recent Projects / Recent Files / Remote Sites
`welcome.noRecent`	No recent files yet. Open a file or a folder to start, or run editora FILE in a terminal.
`welcome.recent.showMore` / `welcome.recent.showFewer`	Show {0} more / Show fewer
`welcome.entryNumber`	Press {0} to open *(accessible help)*
`welcome.customize` / `welcome.keymap` / `welcome.theme`	Customize / Keymap / Theme
`welcome.notation.emacs`	Chords use Emacs notation: C- is Ctrl, M- is Alt, S- is Shift. Pick another keymap above to see your platform's notation.
`welcome.help` / `welcome.documentation` / `welcome.whatsNew` / `welcome.doctor`	Help / Documentation / What's New / Doctor: Check External Tools
`welcome.version`	Version {0}
`welcome.page`	Welcome Page *(the menu item)*

In the apps



Editora: Start actions with their chords, Recent Projects and Recent Files numbered 1 and 2, Customize below.

Torrenta has no documents to come back to, so its welcome is an **empty state over the table** until the first torrent (`ui/Welcome.java` , bound to the list being empty): a drop zone that says "Drop a .torrent, a magnet or a link anywhere", the four keys that matter with their chords (open a .torrent file, add what is on the clipboard, filter or run a command, Settings), the clipboard's magnet or link offered with an *Add it* button when there is one, and where downloads go with a *change* link. Nothing on it takes focus: the keys are the way in. Termina, Mirada, Modula and nux open straight into their content and have neither.

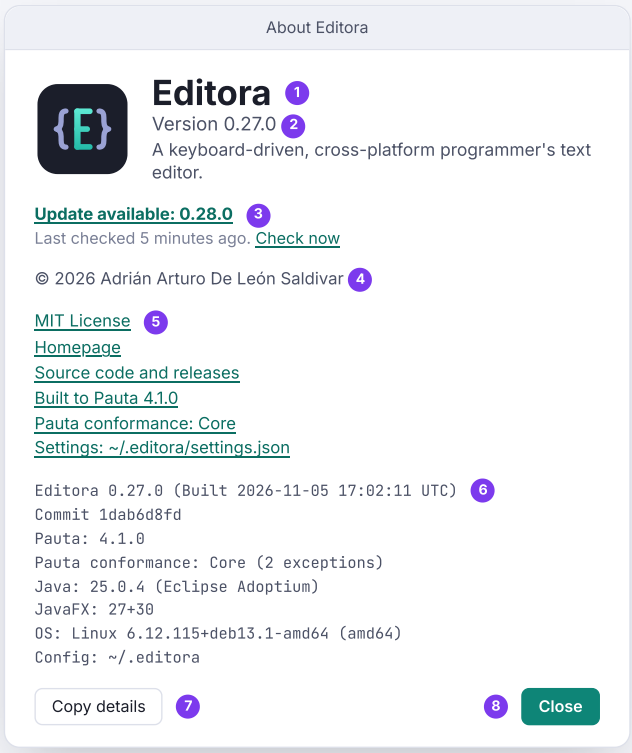
About window

The About window answers "what exactly am I running?" in one place, and its details are what a bug report needs. Torrenta (`AboutDialog`), Termina, Modula, Mirada and nux (`AboutWindow`) follow Editora's order.

Rules

- **ABOUT-01** `MUST` Every app has an About window, reached by Help → About, the `help.about` command, and on macOS the application menu's About item.
  - *Check (code):* `help.about` is registered and listed in the Help menu, and the macOS application menu's About item runs it.
  - *Reference:* Editora `ui/AboutDialog.java` , `ui/MenuBarModel.java` (`help.about` in the Help menu).
- **ABOUT-02** `MUST` About shows the version and build, the Pauta edition (see below), the runtime, the license, links, and the result of the last update check.
  - *Check (code):* the window reads the version, build and edition from `AppInfo` , the same text as `--version` , and shows the update checker's last answer.
  - *Reference:* Editora `ui/AboutDialog.java` , `AppInfo` ; the Spec table below.
- **ABOUT-03** `SHOULD` *Copy details* copies the environment block and leaves the window open; its label becomes "Copied".
  - *Check (code):* the button sits in the content, not the dialog's button bar, so pressing it can't close the window.
  - *Reference:* Editora `AboutDialog` (`about.copyDetails` , `about.copied`).
- **ABOUT-04** `SHOULD` The update status claims nothing before a check has answered: with no answer, or with the check switched off, it says it has not checked, never "up to date".
  - *Check (code):* the status falls back to the not-checked text when the checker has no result.
  - *Reference:* Editora `AboutDialog` (`about.notChecked`); the states below.

Anatomy



1. **Identity.** The app icon at 72 px beside the name (26 px, bold). No alert header band, no info graphic: the panel reads as one block, like a native About panel.
2. **Version and tagline.** Version 14 px muted; a snapshot build appends its Git branch, "0.28.0-SNAPSHOT (feat/map)". The tagline is the README's one-line pitch, 13 px muted.
3. **Update status.** What the last update check found (see the states below), when it ran, and *Check now*. Nothing is claimed before a check has answered.
4. **Copyright**, muted.
5. **Links.** License, homepage, source code and releases, the Pauta edition the app is built to and its conformance, which opens the Pauta window (see below), and the settings file, which opens in the app (or in the file manager for apps with no editor).
6. **Environment.** Monospace 11.5 px, muted: version and build time, commit, Pauta edition and conformance, Java and JavaFX versions with vendor, OS with architecture, config folder. Home folders shown as `~` .
7. **Copy details** copies the environment block and stays open; its label becomes "Copied". It sits in the content, not the button bar, so pressing it does not close the panel.

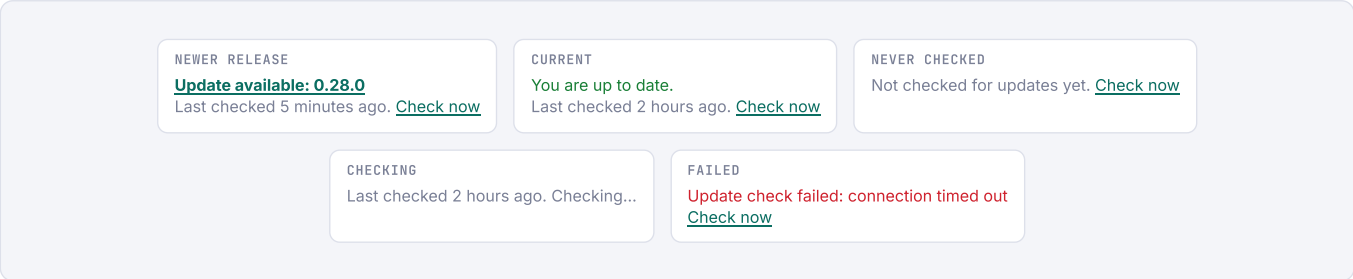
8 **Close**, the one button, and the default: Enter and Esc both close.

Pauta edition and conformance

About names the edition of these guidelines the app implements and how fully it conforms (Conformance), so a user, a bug report and a reviewer all know which rules the app answers to. No app does it yet; the app template names its edition in its README and `AGENTS.md` only.

- **ABOUT-05** `MUST` The edition appears in two places: a link, *Built to Pauta 2.0.0*, among the others, and a line `Pauta: 2.0.0` in the environment block, right after the commit, so *Copy details* carries it.
  - *Check (code):* the links use the `about.pauta` key, and the environment block writes a `Pauta:` line after the commit.
  - *Reference:* no app yet.
- **ABOUT-06** `MUST` The link opens the edition's own address, `https://pauta-project.dev/<edition>/` (for example `https://pauta-project.dev/2.0.0/`), which never changes, not `latest/`, which moves on to newer editions.
  - *Check (code):* the URL is built from the edition number and never contains `latest`.
  - *Reference:* no app yet.
- **ABOUT-07** `MUST` The number is the edition the app meets, not the newest. Moving to a newer edition means reading its changelog for new or stronger MUSTs, meeting them, then changing the number. An app at Partial names the edition it is working toward instead, and says so (ABOUT-09).
  - *Check (by hand):* audit the app against the MUSTs of the edition it names.
  - *Reference:* no app yet.
- **ABOUT-08** `MUST` The edition has one source: a `pauta.version` property in `pom.xml`, filtered into `build-info.properties` and read by `AppInfo`, like the app's own version. The README's Pauta badge and its "Built to" line name the same edition and change with it.
  - *Check (code):* `pom.xml` defines `pauta.version`, `build-info.properties` carries it, `AppInfo` reads it, and the README names the same number.
  - *Reference:* no app yet.
- **ABOUT-09** `MUST` About shows the app's conformance, Complete, Core or Partial, as the helper computes it from `pauta.json` (PAUTA-02) and never higher: a link, *Pauta conformance: Core*, right after the edition's, that opens the Pauta window, and a line `Pauta conformance: Core (2 exceptions)` right after the `Pauta:` line. At Partial the edition's link reads *Working toward Pauta 4.1.0*: the app does not meet that edition yet, so it does not say it is built to it.
  - *Why: Never claim what is not true.*
  - *Check (code):* About calls the conformance helper and holds no conformance name of its own; the link runs `help.pauta`; at Partial the edition's link uses `about.pautaWorkingToward`.
  - *Reference:* no app yet.

Update status states



"Update available" links to that release's page. An update check that is switched off must never read as "up to date": with no answer, the panel says it has not checked.

Spec

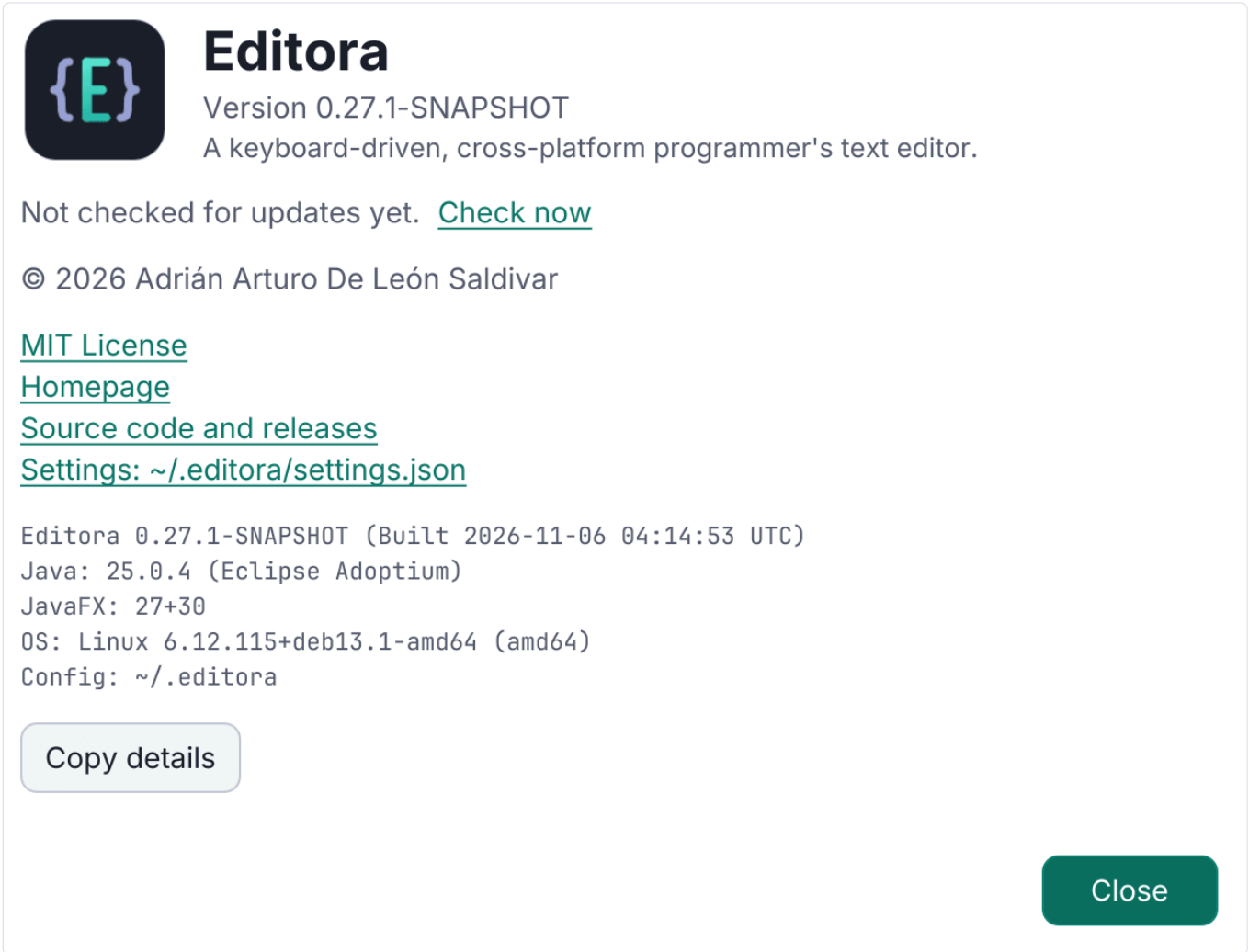
Property	Value	Reference
Kind	A modal dialog owned by the window, styled through `Dialogs.styled` so it gets `app.css` and the UI font	`AboutDialog.show`
Layout	One column, 14 px between blocks; content padding 6 8 2 8	`.about-content`
Icon	The app icon PNG at 72 px	`icon-128.png`
Name / version / tagline	26 px bold / 14 px muted / 13 px muted	`.about-name`, `.about-version`, `.about-tagline`
Update colors	Available: accent, bold. Up to date: green. Failed: red. Last checked: muted	`.about-update*`
Environment	JetBrains Mono 11.5 px, muted	`.about-env`
Source of the version	Maven-filtered `build-info.properties`, read by `AppInfo`: the same text as `--version` and a problem report	`AppInfo`
Pauta edition	`pauta.version` in `pom.xml`, filtered into `build-info.properties`; the link opens `https://pauta-project.dev/<edition>/`	no app yet

Property	Value	Reference
Pauta conformance	Computed from `pauta.json` by the conformance helper; the link opens the Pauta window	Conformance

Copy

Key	English
`dialog.about.title`	About {0}
`about.version`	Version {0}
`about.updateAvailable`	Update available:
`about.upToDate`	You are up to date.
`about.lastChecked`	Last checked {0}.
`about.notChecked`	Not checked for updates yet.
`about.checkNow` / `about.checking`	Check now / Checking…
`about.homepage` / `about.releases`	Homepage / Source code and releases
`about.pauta` / `about.pautaWorkingToward`	Built to Pauta {0} / Working toward Pauta {0}
`about.pautaConformance`	Pauta conformance: {0}
`about.copyDetails` / `about.copied`	Copy details / Copied

In the apps



Editora.

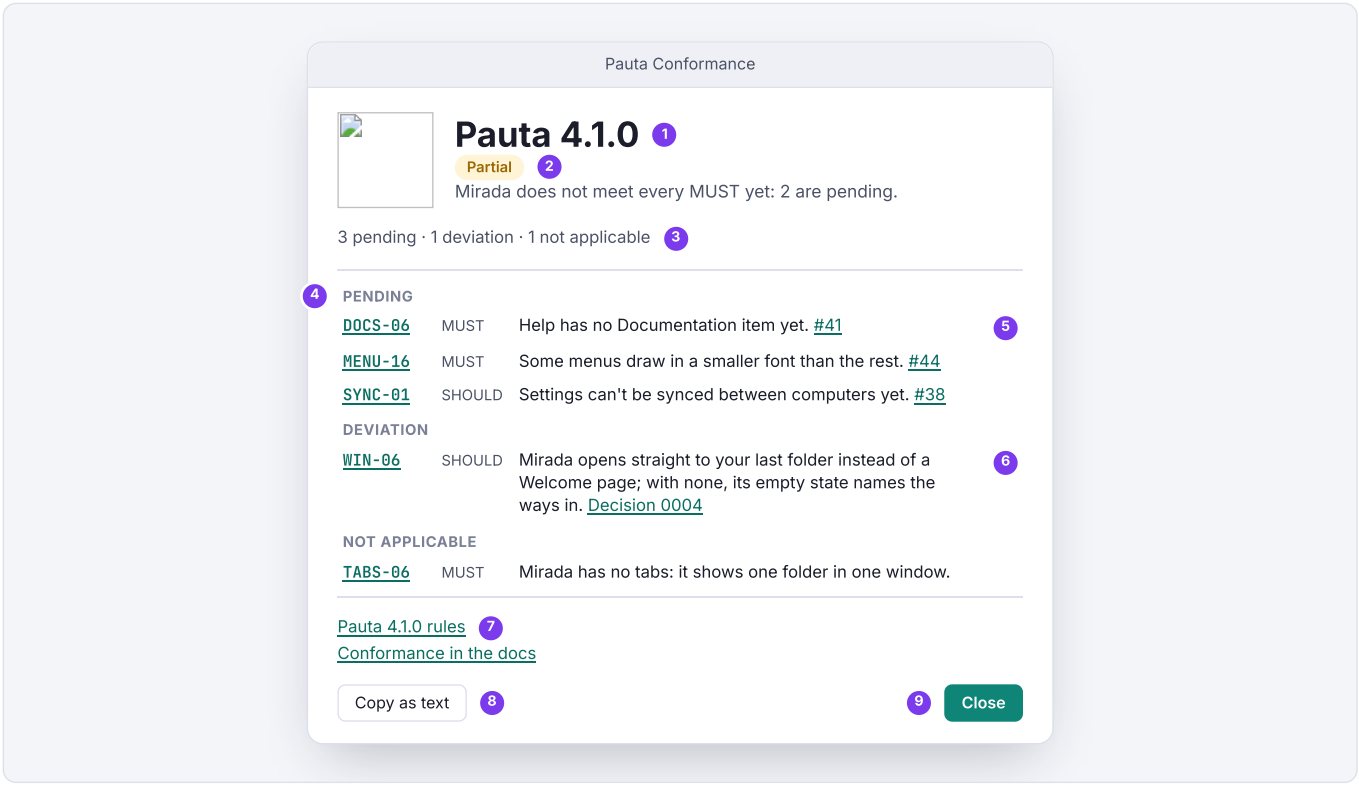
Pauta window

About says which edition of Pauta the app is built to and how fully it conforms; the Pauta window shows the evidence. It lists every exception the app records in its `pauta.json` (see Conformance): the rules that do not apply, the ones it does differently on purpose, and the ones still pending. It is built like the About window: one modal panel, read-only, with a copyable summary. No app has one yet.

Rules

- **PWIN-01** `MUST` Every app has a Pauta window, opened by the `help.pauta` command: from the Command Palette like every command, from the Help menu's *Pauta Conformance* row just above About, and from About's *Pauta conformance* link.
  - *Check (code):* `help.pauta` is registered, the Help `MenuSpec` lists it before `help.about`, and About's *Pauta conformance* link runs it through the registry.
  - *Reference:* no app yet; build it like Editora's `ui/AboutDialog.java`.
- **PWIN-02** `MUST` The window shows the edition, the conformance with one sentence that says what it means, a count of exceptions by kind, and every exception: the rule's ID, its level (`MUST` or `SHOULD`), the kind, the reason, and for a deviation or a pending exception its link.
  - *Check (by hand):* open the window and compare it with `pauta.json`, entry by entry.
  - *Reference:* no app yet; the Anatomy below.
- **PWIN-03** `MUST` It reads from `pauta.json` and `AppInfo` alone, and takes the conformance from the same helper as About (PAUTA-02). Nothing in it is written in the window's code.
  - *Check (code):* the window loads the exceptions through one loader and calls the conformance helper; it holds no rule IDs or conformance names of its own.
  - *Reference:* no app yet.
- **PWIN-04** `MUST` Every link opens a fixed address: the edition at `https://pauta-project.dev/<edition>/`, each rule ID at its address in that edition (the entry's `url`), never `latest/`.
  - *Why:* an app built to 4.1.0 answers to 4.1.0's rules; `latest/` moves on.
  - *Check (code):* the edition URL is built from `pauta.version`; rule links use each entry's `url`.
  - *Reference:* ABOUT-06.
- **PWIN-05** `SHOULD` Pending exceptions come first, then deviations, then not-applicable; within a kind, MUSTs before SHOULDs, then by ID.
  - *Why:* what the app still owes is what a reader came to find.
  - *Check (test):* a unit test of the sort.
  - *Reference:* no app yet.
- **PWIN-06** `SHOULD` *Copy as text* copies the edition, the conformance and the list as Markdown, one line per exception, and leaves the window open; its label becomes "Copied".
  - *Check (code):* the button sits in the content, not the button bar, like About's *Copy details*.
  - *Reference:* Editora `AboutDialog` (`about.copyDetails`, `about.copied`).
- **PWIN-07** `SHOULD` A *Conformance in the docs* link opens the app's docs page (PAUTA-13) for this version.
  - *Check (code):* the link is built from the app's docs base, like Help → Documentation.
  - *Reference:* Editora `AppInfo.docsUrl()`.
- **PWIN-08** `SHOULD` With no exceptions, the list's place says so in one line, "No exceptions: Mirada meets every `MUST` and `SHOULD` of Pauta 4.1.0.", and the copy button still works.
  - *Check (by hand):* with an empty `exceptions` array, open the window.
  - *Reference:* no app yet.

Anatomy



The entries above show the layout with Mirada's name; they are not Mirada's record.

1. **Edition.** The Pauta mark at 72 px beside "Pauta" and the edition, in About's name style.
2. **Conformance.** A pill: Complete green, Core accent, Partial amber. Below it, one sentence in the tagline style that says what it means for this app.
3. **Counts.** How many exceptions of each kind, muted. A kind with none is left out.
4. **Group header.** One per kind present: Pending, Deviation, Not applicable, in that order. The list scrolls past about ten rows; the rest of the window stays put.
5. **Exception row.** The rule's ID in monospace, linked to the rule in this edition; its level, `MUST` or `SHOULD`, muted; the reason; for a pending exception, the issue number as a link.
6. **Decision link.** A deviation links its decision record by number.
7. **Links.** The edition's rules at its own address, and the app's conformance page in its docs.
8. **Copy as text** copies the summary as Markdown and stays open; its label becomes "Copied".
9. **Close**, the one button and the default: Enter and Esc both close.

Spec

Property	Value	Reference
Kind	A modal dialog owned by the window, styled through `Dialogs.styled` like About	`AboutDialog.show`
Width	560 px; the list scrolls at about ten rows	`.pauta-list`
Head	The Pauta mark PNG at 72 px; "Pauta" and the edition 26 px bold	`.about-name`
Conformance pill	Complete `-color-success-*`, Core `-color-accent-*`, Partial `-color-warning-*`	`.pauta-conformance`
Rule ID	JetBrains Mono 12 px, accent, a link	`.pauta-rule`
Source	`pauta.json` in the app's resources and `pauta.version` through `AppInfo`	Conformance
Conformance	The conformance helper, never stored	PAUTA-02

Copy

Key	English
`command.help.pauta`	Pauta Conformance

Key	English
`command.help.pauta.desc`	Show which edition of Pauta {0} implements, how fully it conforms, and the rules it does not meet.
`dialog.pauta.title`	Pauta Conformance
`pauta.edition`	Pauta {0}
`pauta.conformance.complete` / `.core` / `.partial`	Complete / Core / Partial
`pauta.conformance.complete.desc`	{0} meets every `MUST` and `SHOULD` of this edition.
`pauta.conformance.core.desc`	{0} meets every `MUST` of this edition; {1} SHOULDs are pending.
`pauta.conformance.partial.desc`	{0} does not meet every `MUST` yet: {1} are pending.
`pauta.kind.pending` / `.deviation` / `.notApplicable`	Pending / Deviation / Not applicable
`pauta.rules` / `pauta.docs`	Pauta {0} rules / Conformance in the docs
`pauta.none`	No exceptions: {0} meets every `MUST` and `SHOULD` of Pauta {1}.
`pauta.copy` / `pauta.copied`	Copy as text / Copied

The reasons themselves come from `pauta.json`, in English (PAUTA-08); the window's own words are in the catalogs like every other string.

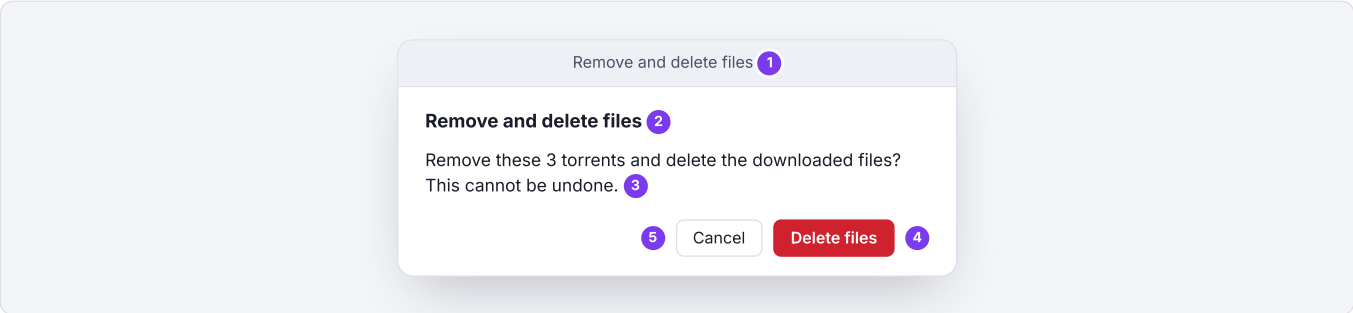
Confirm dialog

One question before anything the user made is removed, deleted, discarded or overwritten, asked the same way everywhere: the text names the thing and what happens to it, the confirming button is the action's own verb in red, and Cancel is the way out. An action that can be taken back needs no question (see Window shell).

Rules

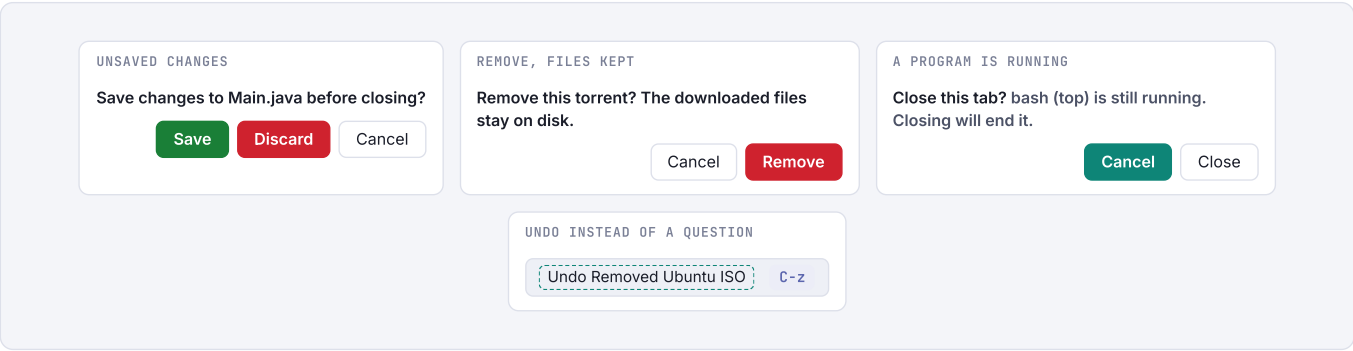
- `CONF-01` `MUST` Before anything the user made is removed, deleted, discarded or overwritten, the app asks first, unless the action can be taken back instead.
  - *Check (code):* each destructive action calls a confirmation before it changes anything, or records an undo.
  - *Reference:* Editora `ui/CloseCoordinator.java` (unsaved changes, Quit) and `GitCoordinator.confirmDestructive`; Torrenta `ui/RemoveConfirm.java`, which asks it for the torrent list, the RSS tab and every list in Settings.
- `CONF-02` `MUST` The question names the thing and what happens to it.
  - *Check (code):* every confirmation string names the thing, by an argument for its name or count or in its own words ("Quit Editora?"), and says what happens to it.
  - *Reference:* Editora `dialog.unsaved.header`, `dialog.discard.tracked`; Torrenta `RemoveConfirm.question`, `Actions.removeQuestion`.
- `CONF-03` `MUST` The confirming button carries the action's own verb (Delete files, Remove, Discard, Quit), never OK or Yes, in the `danger` style.
  - *Check (code):* the action's `ButtonType` is named from the catalog and its button gets the `danger` style class.
  - *Reference:* Editora `GitCoordinator.confirmDestructive`, `MainController.styleUnsavedChangesButtons`, `styleQuitButtonAsDanger`; Torrenta `RemoveConfirm.ask`.
- `CONF-04` `MUST` Cancel is the way out: Escape and the window's close button both answer Cancel, and nothing is lost on a dismissal.
  - *Check (code):* Cancel has the `CANCEL_CLOSE` role, and the result defaults to Cancel when the dialog closes any other way (`showAndWait().orElse(cancel)`).
  - *Reference:* Editora `GitCoordinator.confirmDestructive`, `CloseCoordinator.unsavedAlert`; Torrenta `RemoveConfirm.ask`.

Anatomy



1. **Window title**: the header's words again ("Remove feed", "Unsaved changes"). A plain confirmation, no icon in the title and no app name.
2. **Header**, when the question is long enough to need one. Editora's Git confirmations set none and put the whole message in the content; Torrenta's header names the action and the content asks the question.
3. **The question names the thing and the consequence.** How many ("these 3 torrents", a `{0,choice,…}` plural), which one ("the feed “Linux ISOs” and its articles"), what stays ("The downloaded files stay on disk."), and "This cannot be undone." only when it is true. Termina's multi-line paste shows the text itself in an expandable area, so the decision is made on what will be sent.
4. **The action button** carries the action's verb (Delete files, Remove, Discard, Quit, Close), never OK or Yes, and the `danger` style: red, not the accent. It is the dialog's one colored button.
5. **Cancel**, in the catalog's word, with the `CANCEL_CLOSE` role: Escape and the window's close button both answer Cancel.

States



- `CONF-05` `SHOULD` When saving is an option the question has three answers, in this order: Save (the `success` style, green), Discard (`danger`) and Cancel. A Save that fails leaves the tab open.
  - *Check (code):* the unsaved-changes alert sets Save, Discard and Cancel in that order and styles the first two.

- *Reference:* Editora `CloseCoordinator.unsavedAlert`, `MainController.styleUnsavedChangesButtons`.

- **`CONF-06`** `SHOULD` A question about ending a running program, which is not the user's data, words the button Close and makes Cancel the default button, so dismissing the dialog by any route keeps the program alive.
  - *Check (code):* the close-running-program alert calls `setDefaultButton(true)` on Cancel.
  - *Reference:* Termina `ui/TerminalWindow.java` (`dialog.closeAnyway`).
- **`CONF-07`** `SHOULD` Where the action can be taken back, the app asks nothing and offers Undo instead.
  - *Check (by hand):* close a clean tab and reopen it with Reopen Closed Tab; in Mirada, trash an image and undo.
  - *Reference:* Mirada moves an image to the trash at once and offers Undo; Torrenta asks before removing a torrent but also keeps the removal on its undo stack for ten minutes (`UndoStack.KEEP_MS`); Editora `ClosedTabs`.

Spec

Property	Value	Reference
Kind	`Alert(CONFIRMATION)` owned by the window (`initOwner`), shown with `showAndWait`; the answer defaults to Cancel when the dialog is closed any other way	`GitCoordinator.confirmDestructive`, `RemoveConfirm.ask`
Buttons	A `ButtonType` with the action's name and the `OK_DONE` role, and Cancel with `CANCEL_CLOSE` from the catalog, never JavaFX's stock OK and Cancel, which are worded in the JVM's locale	`RemoveConfirm.ask`, Torrenta's `StockButtons`
Action button	`danger` style class: the theme's `.button.danger` fills it `-color-danger-emphasis` with emphasis text. Save in the unsaved question: `success`	`MainController.styleUnsavedChangesButtons`, `styleQuitButtonAsDanger`
Default button	The action, where it has the `OK_DONE` role (Editora's Git confirmations, Torrenta's Remove): Enter runs it, and red is what says so. Editora's unsaved-changes and Quit questions give their buttons no role, so Enter does nothing. Termina's close-running-program question makes Cancel the default	`confirmDestructive`, `CloseCoordinator.unsavedAlert`, Termina `TerminalWindow`
Font	The UI font sheet attached to the dialog pane, because a dialog is its own scene and inherits nothing from the window	Torrenta `Fonts.installUiFont(DialogPane)`, Editora `App.installUiFont` (every window)
Stylesheet	`Dialogs.styled` adds `app.css` to a dialog that needs the app's own rules (About); the `-color-*` tokens and `.button.danger` resolve from the theme either way	`Dialogs`
Text	The question in the content; a header only when the question needs a name over it. Counts as plurals, names in quotes, the consequence spelled out	`Actions.removeQuestion`, `confirm.remove.<kind>`
Strings	One key per kind, header and question both, tested to exist for every kind	`RemoveConfirm.Kind`, `RemoveConfirmTest`

Behavior

- **`CONF-08`** `SHOULD` The question comes before anything changes, and the action runs only on the action button.
  - *Check (code):* the helper returns true on the action button alone, and the caller acts only on true.
  - *Reference:* Torrenta `RemoveConfirm.ask`; Editora `GitCoordinator.confirmDestructive`.
- **`CONF-09`** `SHOULD` One gesture asks one question. An action on many things asks once with the count ("Remove these 3 torrents?", Torrenta's "Apply to many torrents" for a bulk command). Closing every tab asks per unsaved buffer, selecting each tab first so the user sees what the question is about, and remembers a Discard answer for exactly the state that was shown.
  - *Check (code):* a bulk command builds one question with a plural count; the close-all sweep selects each tab before it asks.
  - *Reference:* Torrenta `Actions.removeQuestion`; Editora `CloseCoordinator.confirmSweep`.
- **`CONF-10`** `SHOULD` A delete that cannot be undone says so; one that leaves the files says that instead.
  - *Check (code):* "This cannot be undone." appears only in the strings of actions that cannot be undone.
  - *Reference:* Editora `dialog.discard.tracked`; Torrenta `actions.removeQuestion.delete`.
- **`CONF-11`** `SHOULD` A dangerous alternative (Force with Lease, Delete Anyway) is offered beside the safe one (Pull, then Push), in red, and only once the safe one has failed.
  - *Check (code):* the force path is reached only from the safe command's failure result.
  - *Reference:* Editora `GitBranchCoordinator.offerAfterRejection` (Force with Lease after a rejected push), `confirmForceDelete` (Delete Anyway after `NOT_FULLY_MERGED`).
- **`CONF-12`** `SHOULD` The question can be tested without a screen: its words come from pure code, and a capture hook answers the dialog, so a UI test never presses a native button.
  - *Check (test):* a unit test covers the header and question for every kind, and UI tests set the capture answer.
  - *Reference:* Torrenta `RemoveConfirm.header`, `question`, `answerForCapture`; `RemoveConfirmTest`.

Copy

Key	English
`dialog.unsaved.title` / `dialog.unsaved.header`	Unsaved changes / Save changes to {0} before closing?
`dialog.save` / `dialog.discard` / `dialog.cancel`	Save / Discard / Cancel
`dialog.quit.header` / `dialog.quit.button`	Quit Editora? / Quit

Key	English
`dialog.discard.tracked`	Discard all changes to "{0}"? This cannot be undone.
`dialog.discardAll.action`	Discard All
`dialog.deleteBranch.force`	Delete Anyway
`confirm.remove.feed.header` / `confirm.remove.feed`	Remove feed / Remove the feed “{0}” and its articles? The torrents already added stay.
`actions.removeQuestion.delete`	Remove this torrent and delete the downloaded files? This cannot be undone.
`window.remove.button.delete` / `common.remove` / `common.cancel`	Delete files / Remove / Cancel
`dialog.closeRunning` / `dialog.closeAnyway` (Termina)	Close this tab? / Close

In the apps

Editora asks for unsaved changes, Quit with busy terminals, pinned tabs, Git discards and branch deletions, and Local History purges, all through `Alert` with named buttons. Torrenta's `RemoveConfirm` serves the torrent list, feeds, rules, watched folders, search providers, engine profiles, categories, tags and saved views with one shape. Termina asks before closing a tab with a running program and before a multi-line paste. Mirada and Modula ask nothing: Mirada's trash is undoable, and Modula's preset chips remove on one click, which this template says they should not.

Find bar

The bar for searching text: in an editor, a read-only console, a terminal's scrollback. It is part of the window and searches as you type. Mirada has no text to search: its *Find Image...* is a picker, see the QuickOpen template.

Rules

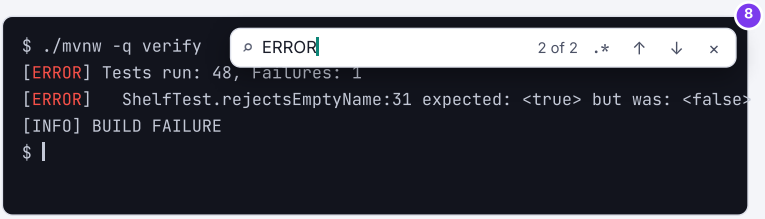
- `FIND-01` `MUST` Wherever there is text to search, the app has a find bar that is part of the window, never a popup.
  - *Check (code):* the bar is a node in the window's scene graph (docked or floating); no `Popup` or `Stage` is created for it.
  - *Reference:* Editora `ui/FindReplaceBar.java` (the editor's find and replace), with `ui/ConsoleFindBar.java` and `ui/TerminalFindBar.java` as the same shape over a console and a terminal; Termina `ui/SearchBar.java`.
- `FIND-02` `MUST` The bar searches as you type and shows a match count.
  - *Check (code):* the query field's text listener runs the search (debounced) and updates the count label.
  - *Reference:* Editora `FindReplaceBar` (`debounce`, `find.count`); Termina `SearchBar` (`search.count`).
- `FIND-03` `MUST` A list app filters instead of finding, with the same field and the same Escape.
  - *Check (code):* the list's filter field filters as it is typed, and Escape in it empties it or gives focus back to the list.
  - *Reference:* Torrenta `Omnibox`, `FilterQuery`, `MainWindow.onOmniboxKey`.

Anatomy



1. **Query field**, focused on open with its text selected, so typing replaces the last query. A single-line selection in the editor is the first query; a multi-line one becomes the scope instead and switches *Sel* on. A clear button appears while the field has text.
2. **Count** "3 of 12", 11 px muted. Empty when there is no query, "No matches" when there are none. Wide enough that the buttons beside it do not shuffle as the digits grow.
3. **Previous and next**, 28 px flat icon buttons, the same as Shift+Enter and Enter.
4. **Options** as compact toggles with a two-letter caption (match case, regular expression, whole word, preserve case, in selection); the tooltip and the accessible name carry the full meaning. Disabled while the query is empty, except *Sel*, which is a scope.
5. **Replace** replaces the active match and steps to the next; *All* replaces every match and reports "Replaced 12 occurrence(s)" in the status line. A console or a terminal has no replace row.
6. **Close**, top right whatever the bar wraps to. Escape does the same.
7. **Matches** are all highlighted while the bar is open, the active one distinctly, and the highlights follow the text as it is edited. They go when the bar closes.

The same bar over a terminal floats at the top right of the pane it searches, in the palette's materials, so the two read as one family: a lens, the field as the card's own text, the count, a regex switch, and up, down and close.



8. **Floating card**, 380 px wide, radius 8, over the output being read. Enter steps *up* to older output and Shift+Enter down: a terminal is searched for something recent, so the search starts at the bottom (iTerm2's convention). The arrow keys do the same.

Spec

Property	Value	Reference
Kind	In-scene, never a popup. Editora docks the bar above the editor area; Termina floats it inside the `TerminalView`, so each pane of a split has its own and a search never wonders which terminal it is about	`MainController` (`topBox`), `SearchBar`
Docked bar	`-color-bg-subtle`, padding 4 8, spacing 6; icon buttons 28 × 28, flat; the groups (find, replace, options) wrap onto a second line in a narrow window instead of shrinking the fields	`.find-bar`, `.find-bar-icon`, `WrapRow`
Floating card	`-color-bg-overlay`, 1 px `-color-border-default`, radius 8, 380 px wide, padding 3 4 3 8, shadow 18 at offset 6; the field is the card's own text at 12.5 px	`.search-bar`, `.search-field`
Options	Toggles with a short caption; tooltip and accessible name carry the meaning; selected: `-color-accent-subtle` with `-color-accent-fg`	`OptionToggle`, `.search-toggle:selected`
Count	`{0} of {1}`, 11 px `-color-fg-muted`, right-aligned; minimum width 64 px	`find.count`, `search.count`, `COUNT_MIN_WIDTH`
Debounce	The search runs 150 ms after the last keystroke, and an edit to the text re-highlights after the same 150 ms without moving the caret	`debounce`, `editDebounce`, `TYPING_SETTLE`, `searchTyping`
Threading	A document up to 512 K characters is searched where the request is made; larger ones on one shared daemon thread, latest query only, the result applied on the FX thread. A scrollback is always scanned off the FX thread	`SYNC_SEARCH_CHARS`, `SEARCH_POOL`, `LatestOnly`, `TerminalView.searchAsync`
Bounds	A page of matches around the active one, not every match of a query that matches millions of times; a regular expression has a time budget. Termina keeps the newest 1,000 matches	`SearchMatcher.DEFAULT_PAGE`, `find.tooComplex`, `ScrollbackSearch.MAX_MATCHES`
Bad regex	Said in place, never by a dialog: the status line ("Bad regex: …") or, in Termina, the field in `-color-danger-fg` and the count "Invalid"	`find.badRegex`, `.search-field:invalid`
Focus	The bar and its buttons are out of the Tab order; in a terminal Tab belongs to the shell	`setFocusTraversable(false)`

Behavior

- **FIND-04** `SHOULD` The find command (`find.show`) opens the bar: `C-s` in the Emacs keymap and `Ctrl/Cmd+F` in CUA, Sublime, VS Code and IntelliJ, with `F3` as well in CUA and Sublime. A terminal uses `Ctrl+Shift+F`, `Cmd+F` on macOS, since a plain `Ctrl+letter` belongs to the shell. The command while the bar is already open finds the next match (Editora) or puts the caret back in the field (Termina).
  - *Check (code):* each bundled keymap binds that chord to the find command; the command steps when the bar is shown.
  - *Reference:* Editora `keymaps/*.json`, `MainController.findShowOrNext`; Termina `TerminalWindow` (`menu.find`, `MenuAction.appChord`).
- **FIND-05** `SHOULD` Typing highlights every match and selects and scrolls to the first one at or after the caret. The anchor is the caret at open time, so refining the query does not run away down the document.
  - *Check (code):* the search starts from an anchor saved when the bar opens, not from the current selection.
  - *Reference:* Editora `FindReplaceBar` (`searchAnchor`, `recompute`).
- **FIND-06** `MUST` Enter steps to the next match and Shift+Enter to the previous one, wrapping around at the ends. In a terminal the directions are up and down the scrollback.
  - *Check (code):* the query field's key handler steps forward on Enter and backward on Shift+Enter.
  - *Reference:* Editora `ConsoleFindBar`, `TerminalFindBar`; Termina `SearchBar`. Editora's editor bar (`FindReplaceBar`) does not yet: its field steps forward on Enter, and back only with the previous button, `find.showBackward` (`C-r`, `Shift+F3`) or `find.previous`. Alt+Enter there turns every match into a caret.
- **FIND-07** `MUST` Escape or the close button closes the bar: the highlights go, a search still pending is canceled so nothing moves after the bar is gone, and focus returns to the editor, the console or the shell.
  - *Check (code):* the close path stops the debounces, cancels the running search, clears the highlights and focuses the text it searched.
  - *Reference:* Editora `FindReplaceBar.hideBar`.
- **FIND-08** `SHOULD` In a terminal, Escape is consumed in the field and never reaches the shell, and the terminal's own key and mouse filters leave every event aimed at the bar alone.
  - *Check (code):* the field consumes Escape, and the terminal's filters return early for events from the bar.
  - *Reference:* Termina `SearchBar`, `TerminalView.fromSearchBar`.
- **FIND-09** `SHOULD` The keymap's caret and editing chords (`C-a`, `C-e`, `C-k`, `C-y`, `Ctrl+V/X/C/Z`) act on the field, not on the document. Find's own chords stay global, so next, previous, replace and close work while the field has focus.
  - *Check (code):* the find fields install the shared text-input keymap.
  - *Reference:* Editora `TextInputKeymap.installShared`; Termina's and Mirada's `TextInputKeys`.
- **FIND-10** `SHOULD` Nothing found is one status-line message ("Not found: x") or the count "No matches". No beep, no dialog.
  - *Check (code):* the no-match path sets `find.notFound` or `search.none` and opens nothing.
  - *Reference:* Editora `FindReplaceBar` (`find.notFound`); Termina `SearchBar` (`search.none`).
- **FIND-11** `SHOULD` Replace keeps each match's case when *AB* (preserve case) is on.
  - *Check (test):* a test replaces with preserve case on and finds each match's casing kept.
  - *Reference:* Editora `FindReplaceBarFxTest` (`preserveCaseFollowsEachMatchesCasing`).
- **FIND-12** `SHOULD` A replacement that is too slow stops with "Replace stopped: the pattern is too slow" instead of hanging the window.
  - *Check (test):* a catastrophic regex in Replace All aborts.
  - *Reference:* Editora `find.replaceTimeout`, `FindReplaceBarFxTest` (`catastrophicRegexInReplaceAllAbortsInsteadOfHanging`).

- **FIND-13** `SHOULD` A list app's filter box filters the names as plain words are typed, turns `key:value` terms into chips and runs a command after `>`. `Ctrl+K` focuses it and is drawn in the box as a key badge.
  - *Check (code):* the box's text listener filters, and its badge is read from the filter command's chord.
  - *Reference:* Torrenta `Omnibox`, `MainWindow.buildFilterBar`.
- **FIND-14** `SHOULD` Escape in a filter box peels one layer: a box with something in it empties, an empty one gives the list back.
  - *Check (code):* the box's Escape handler clears a non-empty box and otherwise focuses the list.
  - *Reference:* Torrenta `MainWindow.onOmniboxKey`.
- **FIND-15** `SHOULD` The filter box is out of the Tab order, so a fresh launch does not land every keystroke in it, and the window's own chords (undo, `Ctrl+Alt+1…9`) do not fire while a text field has focus: `Ctrl+Z` there is the field's own undo.
  - *Check (code):* the box is not focus-traversable, and the window's chord handlers return early when the focus owner is a `TextInputControl`.
  - *Reference:* Torrenta `MainWindow.buildFilterBar`, `MainWindow` window chords.

Copy

Key	English
`find.prompt`	Find
`find.replacePrompt`	Replace
`find.count`	{0} of {1}
`find.notFound`	Not found: {0}
`find.badRegex`	Bad regex: {0}
`find.tooComplex`	Search stopped early: the pattern is too complex for this text
`find.replaced`	Replaced {0} occurrence(s)
`search.caseTip` · `search.regexTip` · `find.wholeWord`	Match case · Regular expression · Whole word
`search.older` · `search.newer` · `search.close` (Termina)	Previous match, up (Enter) · Next match, down (Shift+Enter) · Close (Escape)
`search.none` · `search.invalid` (Termina)	No matches · Invalid
`window.filter.prompt` (Torrenta)	Filter · > runs a command · paste a magnet

In the apps

- **Editora:** find and replace over the editor (`FindReplaceBar`), with *Find in Files* (`search.inFiles`) as a separate results window; the console and terminal tool windows carry the same bar without the replace row.
- **Termina:** the floating card over each pane, with the regex switch; matches are found across soft-wrapped lines, so a word that reached the right edge and carried on is still found.
- **Torrenta:** the omnibox above the transfers, which filters, runs commands and adds what is pasted.
- **Mirada, Modula, nux:** no free text to search; Mirada finds an image through a picker.

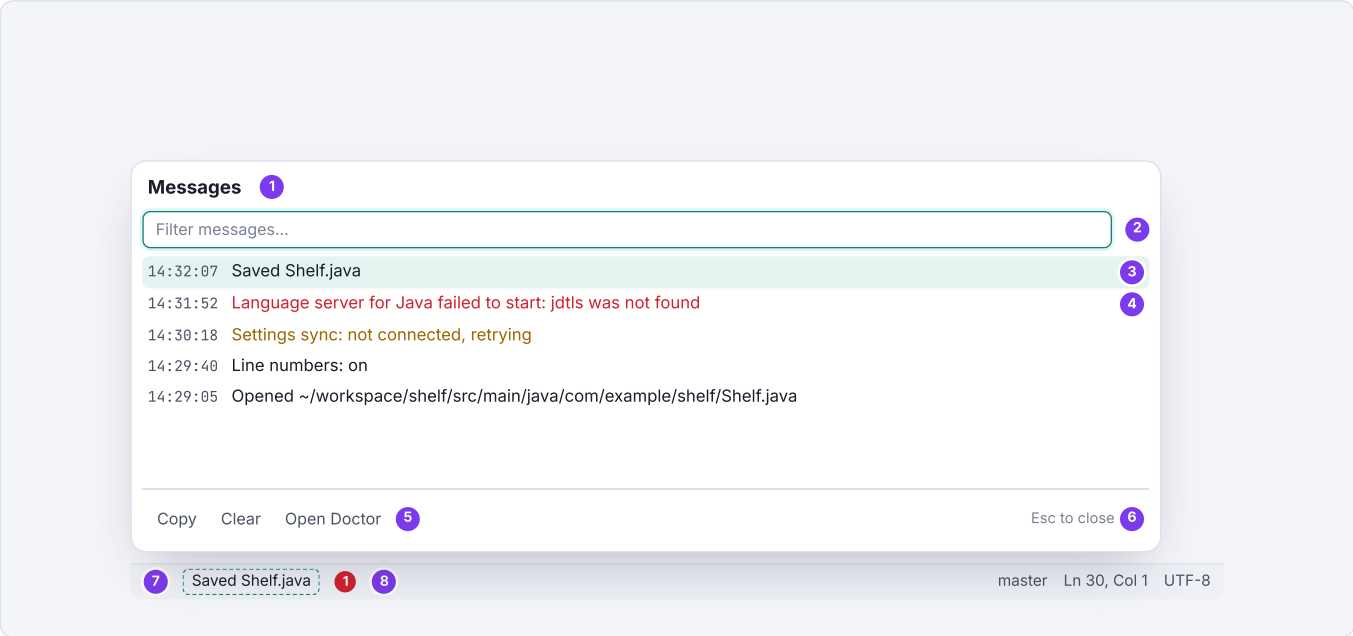
Message Log

The status bar's echo line shows one message and replaces it with the next; the Message Log keeps them, so nothing the app said is lost because it scrolled past.

Rules

- `MLOG-01` `MUST` Every app with a status bar keeps a Message Log: the session's status-bar messages, newest first, opened from the status bar.
  - *Check (code):* every status-bar message is also added to one log model, and a view lists that model.
  - *Reference:* Editora `ui/MessageLog.java` (the model, no JavaFX) and `ui/MessageLogPopup.java` (the view); Torrenta carries the same pair.

Anatomy



1. **Title** "Messages", 14 px bold.
2. **Filter**, focused on open. Filters as you type; ↓ or Enter moves into the results without leaving the field.
3. **Entry.** The time as `HH:mm:ss` (11 px, monospace, muted) and the message as it appeared in the echo line. Newest first.
4. **Severity** colors the message, not the row: errors red, warnings amber, information in the default color. The time stays muted.
5. **Actions:** Copy (the visible entries as text), Clear (empties the session log and its unread count), and Open Doctor: messages carry no actions of their own, and a missing tool or a server that would not start is where Doctor helps.
6. **Key hint**, muted, 11 px.
7. **Echo line.** The status bar's latest message; a click opens the log.
8. **Unread errors.** A red count beside the echo line while errors have arrived since the log was last opened; a click opens the log.

Spec

Property	Value	Reference
Kind	A card in the shared in-scene overlay host, anchored just above the status bar's echo area, over the window	`MessageLogPopup.show`, `OverlayHost`
Size	773 px wide; the list 336 px high, scrolling past that	`WIDTH`, `LIST_HEIGHT`
Card	`-color-bg-overlay`, radius 12, like the palette	`.message-log`
Rows	Padding 2 4, radius 5; selected row `-state-sel`	`.message-log-list .list-cell`
Capacity	The newest 200 messages; older ones drop off	`MessageLog.MAX_ENTRIES`
Repeats	A message repeated within 2 s moves to the top as one entry, unread once	`REPEAT_MILLIS`
Lifetime	This session only, in memory. Not written to disk: the Debug Log is the persistent record	`MessageLog`
Blank messages	Not recorded (a blank message only clears the echo line)	`MessageLog.add`

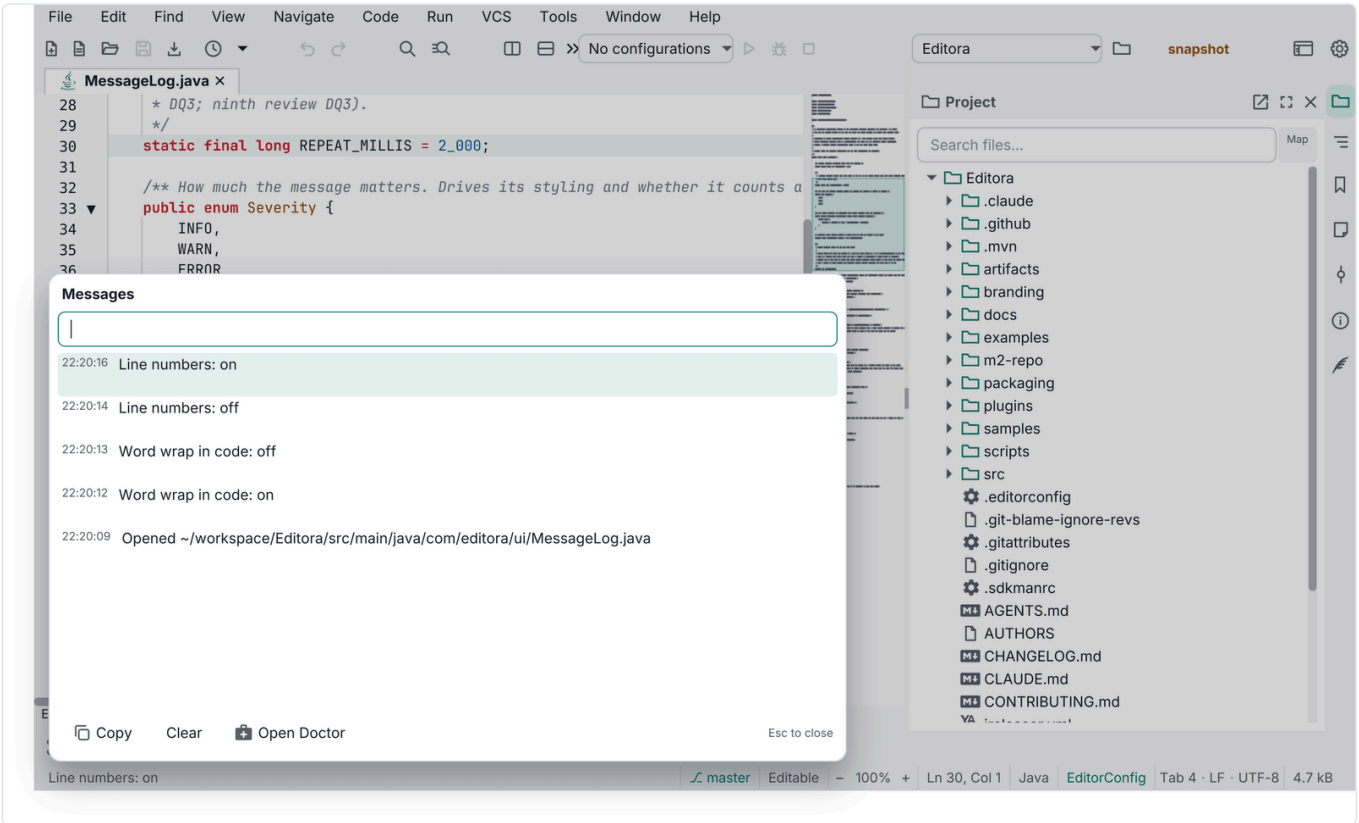
Behavior

- `MLOG-02` `SHOULD` A click on the echo line or on the unread-error count opens the log, as does the `view.messageLog` command.
  - *Check (code):* both status-bar nodes run `view.messageLog` on a click.
  - *Reference:* Editora `StatusBar` (`echo` , `unreadErrors`), `StatusBar.showMessageLog` .
- `MLOG-03` `SHOULD` Esc, `C-g` or a click outside closes the log.
  - *Check (code):* the card's key handler hides it on Escape and `C-g` ; the overlay's backdrop hides it on a press.
  - *Reference:* Editora `MessageLogPopup` , `OverlayHost` .
- `MLOG-04` `SHOULD` `Ctrl+C` (`Cmd+C` on macOS) copies the selected entries.
  - *Check (code):* the card's key handler copies the selection on the shortcut-down C.
  - *Reference:* Editora `MessageLogPopup.copySelection` .
- `MLOG-05` `SHOULD` A message's severity comes from its key: a `status.*` key whose last segment contains `fail` or `error` is an error, one containing `cannot` or `invalid` a warning. Name new keys accordingly, or report through the error channel directly.
  - *Check (test):* a severity test classifies keys by those rules.
  - *Reference:* Editora `ui/StatusSeverity.java` (`ofKey`), `StatusSeverityTest` .
- `MLOG-06` `SHOULD` Opening the log, or clearing it, resets the unread-error count.
  - *Check (code):* opening the log marks it read, and `clear` zeroes the count.
  - *Reference:* Editora `MessageLog.markRead` , `MessageLog.clear` , `StatusBar.showMessageLog` .
- `MLOG-07` `SHOULD` An unread error is never evicted silently: the count keeps it even when the entry drops off the end.
  - *Check (test):* an error pushed past the capacity still counts as unread.
  - *Reference:* Editora `MessageLogSeverityTest` (`anEvictedErrorStillCounts`).

Copy

Key	English
`messagelog.title`	Messages
`messagelog.filter`	Filter messages...
`messagelog.empty`	No messages yet
`messagelog.copy` / `messagelog.clear`	Copy / Clear
`messagelog.doctor`	Open Doctor
`messagelog.hint`	Esc to close

In the apps



Editora.

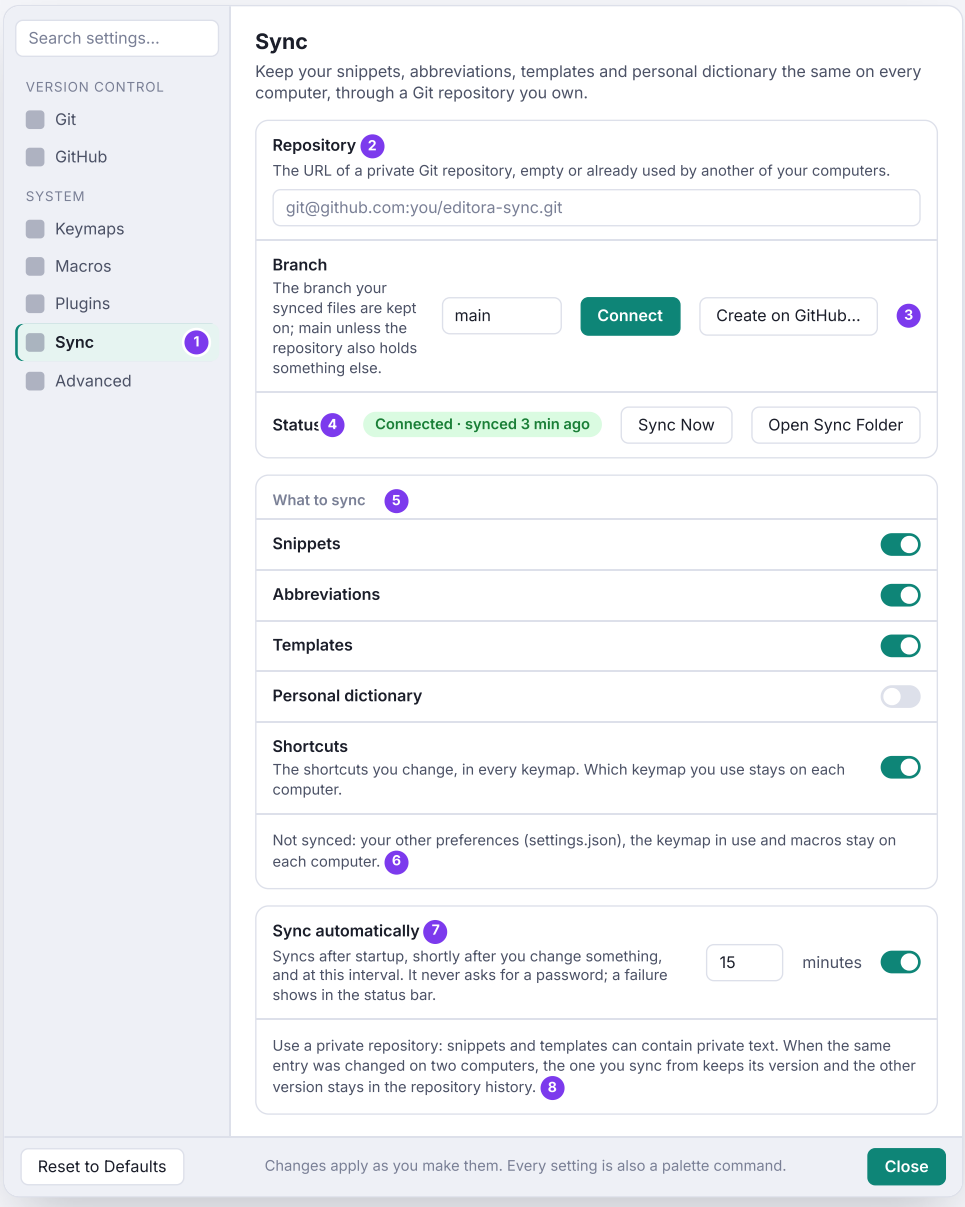
Settings Sync

Keeps the user's data (snippets, templates, shortcuts, profiles, saved views) the same on every computer. No server and no account of ours: the store is a private Git repository the user owns.

Rules

- **SYNC-01** `SHOULD` An app with user data worth carrying between computers offers Settings Sync, on a page of its own in the Settings window that the Settings search finds by "sync".
  - *Check (by hand):* open Settings, search "sync", and land on the Sync page.
  - *Reference:* Editora `ui/SettingsSync.java` (the service) and `ui/SettingsSyncPage.java`, built by `SettingsWindow` (`Category.SYNC`).
- **SYNC-02** `SHOULD` The store is a Git repository the user owns, reached through their own Git credentials (a credential helper or an SSH key), with no server or account of ours. A URL with a password or token in it is refused.
  - *Check (code):* the sync package runs `git` against the user's URL, and the page refuses a URL with credentials in it.
  - *Reference:* Editora `sync/` package (`SyncEngine`, `SyncMerge`, `SyncAccount`), `SyncEngine.hasCredentials`; Termina carries the same package, Torrenta its own `sync/` with `SyncEngine` and `SyncAccount`; the Spec table below.

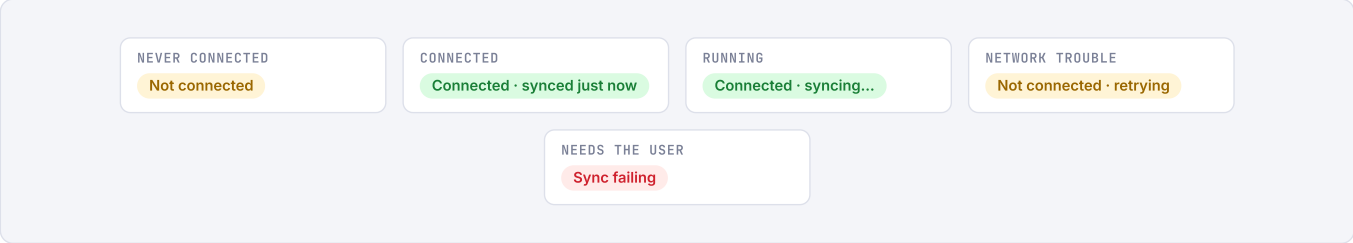
Anatomy



1 **Its own page** in the Settings sidebar, under System, found by the Settings search ("sync", "git", "github").

2. **Repository URL.** SSH or HTTPS. A URL with a password or token in it is refused, with the reason and the fix (a credential helper such as `gh auth setup-git`, or an SSH URL).
3. **Branch, Connect, Create on GitHub….** Connect clones or adopts the repository. Create on GitHub makes a private repository through the `gh` CLI and fills the URL in.
4. **Status pill** plus Sync Now and Open Sync Folder (see the states below).
5. **What to sync:** one switch per kind of data, each with a description when its scope is not obvious.
6. **What is not synced**, said plainly: machine-local preferences, the keymap in use, anything with paths.
7. **Automatic sync** with its interval in minutes (default 15, at least 1). Automatic runs never prompt.
8. **Privacy and conflicts**, in one note: use a private repository; on a same-entry conflict the computer you sync from wins, and the other version stays in the history.

Status pill states



Green, amber and red are the Settings status-pill colors. Amber means "will recover by itself" (offline, the remote unreachable); red means "the user must act" (Git missing, the repository written by a newer version, a sync that would remove most of a kind of data and waits for confirmation). The pill's accessible text is "Sync status: …".

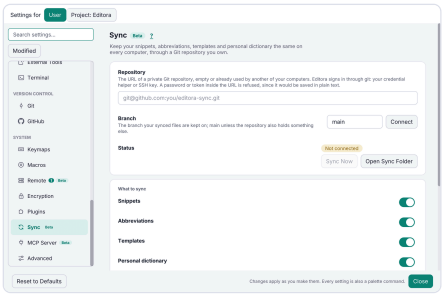
Spec

Property	Value	Reference
Store	A Git repository the user owns, cloned into `<config>/sync/`; one folder per kind of data	`SyncEngine`, `SyncTarget`
Sign-in	The user's own Git credentials (credential helper or SSH key); the GitHub account that can read the repository is pinned per clone	`SyncAccount`
Automatic runs	After startup, 30 s after a local change, and every *n* minutes (default 15)	`SettingsSync.DIRTY_DELAY_SECONDS`, `syncIntervalMinutes`
Conflicts	Per entry; this computer wins; the other version stays in history	`SyncMerge`
Safety	Files a sync replaces are copied to the sync folder first; a sync that would remove most of a kind of data stops and asks	`SyncReport`
One runner	Sync runs in the first instance only; a second instance says so	`status.sync.secondaryInstance`
Failures	To the status bar and the Message Log, said once per failure, not once per interval	`SettingsSync`
Palette	Settings Sync: Set Up…, Sync Now, and Toggle Sync Automatically	`sync.setup`, `sync.now`, `sync.toggleAuto`

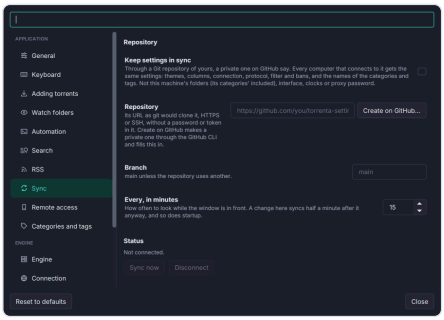
Copy

Key	English
`settings.sync.subtitle`	Keep your snippets, abbreviations, templates and personal dictionary the same on every computer, through a Git repository you own.
`settings.sync.connect` / `settings.sync.createGithub`	Connect / Create on GitHub…
`settings.sync.now` / `settings.sync.openFolder`	Sync Now / Open Sync Folder
`settings.sync.pill.*`	Connected · Not connected · Sync failing · syncing… · retrying · synced {0} min ago
`status.sync.notSetUp`	Settings sync is not connected. Set it up in Settings ▸ Sync.

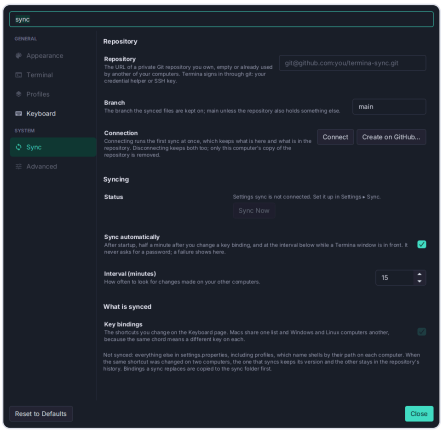
In the apps



Editora.



Torrenta.



Termina.

Packaging, signing and release

Targets

Each target builds on its own GitHub-hosted runner. jpackage and JavaFX natives are host-specific, so there is no cross-building.

Target	Runner	Downloads
Linux x64	`ubuntu-latest`	`.deb`, `.rpm`, `.AppImage`, `.tar.gz`, fat jar
Linux arm64	`ubuntu-24.04-arm`	`.tar.gz` (and `.deb` where the app's tooling allows), fat jar
macOS arm64	`macos-15`	`.dmg`
macOS x64	`macos-15-intel`	`.dmg` (the last Intel image, good to about Aug 2027)
Windows x64	`windows-latest`	`.msi`, fat jar
Windows arm64	—	**Not possible yet**: OpenJFX publishes no `win-aarch64` natives (JDK-8314064). ARM users run the x64 MSI under emulation.

- `REL-01` `MUST` jpackage builds the installers for the five targets in the table above, each on its own runner, through the Maven `dist` profile.
  - *Check (code):* the release workflow's build matrix lists `linux-x64`, `linux-arm64`, `macos-arm64`, `macos-x64` and `windows-x64` on those runners, and `pom.xml` has a `dist` profile.
  - *Reference:* Editora `release.yml` (the `build` job), `pom.xml` (`dist`).
- `REL-02` `SHOULD` Downloads are named `<App>-<version>-<target>.<ext>`.
  - *Check (code):* the release-asset check builds the expected names this way.
  - *Reference:* Editora `scripts/release/CheckRelease.java` (`expectedAssets`).
- `REL-03` `MUST` **JReleaser** (`jreleaser.yml`) only creates the GitHub release: it builds nothing, and attaches every file and `checksums_sha256.txt`.
  - *Check (code):* `jreleaser.yml` attaches the build matrix's files and runs after the build jobs.
  - *Reference:* Editora `jreleaser.yml`, the `release` job in `release.yml`.
- `REL-04` `SHOULD` The release notes are the version's section of `CHANGELOG.md`.
  - *Check (code):* the release job hands JReleaser that section as the changelog.
  - *Reference:* Torrenta `scripts/release/ReleaseNotes.java`. Editora's release notes are still its commit list.
- `REL-05` `MUST` The Maven version is the single source of truth: a tag `vX.Y.Z` releases the version in `pom.xml`.
  - *Check (code):* the release workflow fails when the tag and the pom version differ.
  - *Reference:* Editora's "Check the tag against the pom version" step in `release.yml`.
- `REL-06` `SHOULD` `master` carries a `-SNAPSHOT` version, and a post-release job opens the pull request that bumps it to the next snapshot.
  - *Check (code):* the pom's version on `master` ends in `-SNAPSHOT`, and `release.yml` has the bump job.
  - *Reference:* Editora's `bump` job in `release.yml`.
- `REL-07` `SHOULD` A **release-asset check** fails the run before publishing when an expected file is missing.
  - *Why:* GitHub releases are immutable.
  - *Check (code):* a step before JReleaser compares the gathered files with the expected list.
  - *Reference:* Editora's "Verify the expected release assets" step in `release.yml`.
- `REL-08` `SHOULD` On Linux, a `.desktop` file's `StartupWMClass` is the JavaFX app class.
  - *Why:* the running window then belongs to the menu entry and shows its icon.
  - *Check (code):* `packaging/linux/<App>.desktop` sets `StartupWMClass=com.<app>.App`.
  - *Reference:* Editora `packaging/linux/Editora.desktop`, Torrenta `packaging/linux/Torrenta.desktop`.
- `REL-09` `SHOULD` On Linux, AppStream metainfo has the id `io.github.adriandeleon.<App>`.
  - *Check (code):* `packaging/linux/io.github.adriandeleon.<App>.metainfo.xml` carries that `<id>`.
  - *Reference:* Torrenta `packaging/linux/`. Editora's metainfo id (`com.editora.Editora`) predates the rule.
- `REL-10` `SHOULD` Packaging trains an **AOT cache** (JDK 25 Leyden) to cut start-up time; Linux legs run the training under `xvfb`.
  - *Check (code):* the `dist` build runs a training launch with `-XX:AOTCacheOutput`, and the Linux release legs install `xvfb`.
  - *Reference:* Editora `scripts/aot_build.java`.

Install methods

- `REL-11` `MUST` The GitHub release offers every download on every platform: the `.dmg`, `.msi`, `.deb`, `.rpm`, `.AppImage` and `.tar.gz`, with checksums.
  - *Check (code):* `jreleaser.yml` attaches each of these types.
  - *Reference:* Editora `jreleaser.yml`.
- `REL-12` `SHOULD` On macOS (Apple Silicon) the app installs with `brew install --cask adriandeleon/tap/<app>`: JReleaser's `brew` packager writes the cask to `adriandeleon/homebrew-tap`. A cask takes exactly one artifact, so it serves Apple Silicon, and Intel Macs use the `macos-x64` DMG.
  - *Why:* the official homebrew-cask repository has notability rules a new project does not meet, so the family shares one personal tap.
  - *Check (code):* `jreleaser.yml` has a `brew` packager for that tap; `jreleaser package --basedir .` writes the cask locally, to check it without releasing.
  - *Reference:* Torrenta and Termina `jreleaser.yml`; Editora has no packager yet.
- `REL-13` `SHOULD` On Windows x64 the app installs with `winget install AdrianDeLeon.<App>`: JReleaser's `winget` packager opens a pull request on `microsoft/winget-pkgs` from the `adriandeleon` fork. The package identifier is `AdrianDeLeon.<App>`, the moniker the app's lower-case name.

- *Check (code):* `jreleaser.yml` has a `winget` packager with that identifier; `jreleaser package --basedir .` writes the manifests locally.
- *Reference:* Torrenta and Termina `jreleaser.yml`; Editora has no packager yet.

- **REL-14** `SHOULD` Both packagers read a `PACKAGES_TOKEN` secret, a classic personal access token with `repo` scope. Without it JReleaser skips both, and the GitHub release still publishes.
  - *Why:* the workflow's own token cannot write to the tap or open a pull request on the fork.
  - *Check (code):* `release.yml` passes `PACKAGES_TOKEN` to JReleaser and leaves the packagers out when it is empty.
  - *Reference:* Torrenta and Termina `release.yml`.

winget is reviewed by Microsoft: the first submission for a new publisher is checked by a person, not merged automatically.

Status on 2026-11-06: the tap exists and holds no casks yet. Torrenta and Termina have both packagers configured, waiting for the token. Mirada's first winget submission (microsoft/winget-pkgs#471599) is open.

Signing

- **REL-15** `MUST` The macOS `.dmg` is signed with the Developer ID Application certificate (team 8JKSF797L8), with the hardened runtime and per-app entitlements, notarized with `notarytool`, stapled and checked with `spctl`; then the notarized app is launched once.
  - *Check (code):* `release.yml` imports `MACOS_CERT_P12` and `MACOS_CERT_PASSWORD`, notarizes with `APPLE_API_KEY_P8`, `APPLE_API_KEY_ID` and `APPLE_API_ISSUER`, staples, runs `spctl` and launches the app.
  - *Reference:* Editora's "Import signing certificate (macOS)" and "Notarize DMG (macOS)" steps in `release.yml`, `packaging/macos/Editora.entitlements`. Done in all six apps.
- **REL-16** `MUST` Linux downloads are signed with the one shared OpenPGP release key ("Adrian De Leon (release signing)") in a separate `sign-linux` job: `rpmsign` for the `.rpm`, a debsigs origin signature for the `.deb`, and an armored `.asc` beside every file. The public key is committed as `packaging/linux/signing-key.asc` and attached to each release.
  - *Check (code):* `release.yml` has a `sign-linux` job that reads `LINUX_GPG_KEY` and `LINUX_GPG_PASSPHRASE` and runs the shared scripts, and `packaging/linux/signing-key.asc` exists.
  - *Reference:* Editora `scripts/release/sign-linux-packages.sh`, `scripts/release/check-linux-signatures.sh`. Done in Editora, Torrenta, Termina and Modula; Mirada partly; nux not yet.
- **REL-17** `SHOULD` The Windows `.msi` is signed with Authenticode, once the certificate is chosen.
  - *Check (code):* the Windows leg signs the `.msi` before it is uploaded.
  - *Reference:* no app yet. **Open decision**: SignPath Foundation (free for OSS), a Certum Open Source certificate, or Azure Trusted Signing. Until then MSIs are unsigned and SmartScreen warns.
- **REL-18** `MUST` Every download has a GitHub **build provenance attestation** (`actions/attest-build-provenance`), which `gh attestation verify` checks.
  - *Check (test):* a test keeps the attested files and the files JReleaser attaches the same list.
  - *Reference:* Editora's "Attest build provenance" step in `release.yml`, `ReleaseSupplyChainTest`. Done in Editora, Termina and Mirada.

The certificate renews before 2027-11-30; the Apple membership before 2027-10-31.

CI/CD (GitHub Actions)

- **REL-19** `MUST` `ci.yml` runs on every push to `master` and every pull request: **Format** (`spotless:check`) and **Build & Test** (`./mvnw verify`, coverage gate included) on JDK 25. Both are required status checks for `master`.
  - *Check (code):* `ci.yml` triggers on `push` to `master` and on `pull_request`, with the two jobs; the repository's rules for `master` require both.
  - *Reference:* Editora `ci.yml`.
- **REL-20** `SHOULD` `full-tests.yml` runs the full suite on Linux, macOS and Windows daily, on a pull request labeled `full-ci`, and before every release, and lists flaky tests in the job summary.
  - *Check (code):* `full-tests.yml` has a `schedule` and a `pull_request` trigger gated on the label, and the release preflight waits for its run on the tagged commit.
  - *Reference:* Editora `full-tests.yml`, `scripts/ci/ReportFlakes.java`, and the "Check that the Full tests passed on this commit" step in `release.yml`.
- **REL-21** `MUST` `release.yml` runs on a `v*` tag that matches the pom version, with `workflow_dispatch` as a dry run. It runs the five-target matrix, the signing jobs, attestations, the release-asset check, JReleaser, then the bump to the next snapshot. A dry run builds and signs and publishes nothing.
  - *Check (code):* `release.yml` triggers on the tag pattern and `workflow_dispatch`, and a dispatch runs JReleaser with `--dry-run`.
  - *Reference:* Editora `release.yml`.
- **REL-22** `MUST` Every workflow pins each third-party action to a commit SHA, with its version in a comment (`actions/setup-java@b843ebf… # v6.1.0`).
  - *Check (code):* every `uses:` in `.github/workflows/` ends in a 40-character SHA and a `# vX.Y.Z` comment.
  - *Reference:* Editora `.github/workflows/`.
- **REL-23** `MUST` Dependabot updates `github-actions` and `maven` weekly.
  - *Check (code):* `.github/dependabot.yml` lists both ecosystems with `interval: weekly`.
  - *Reference:* Editora `.github/dependabot.yml`.
- **REL-24** `MUST` Every workflow starts from `permissions: contents: read`; only a job that needs write access asks for it, in its own block.
  - *Check (code):* each workflow's top-level `permissions` is `contents: read`, and every `write` sits in a job's own `permissions`.
  - *Reference:* Editora `release.yml`: `release` (publish), `bump` (the pull request) and `preflight` (start a Full tests run).
- **REL-25** `SHOULD` Every workflow sets up JDK 25 with `actions/setup-java` and `cache: maven`.
  - *Check (code):* each `setup-java` step has `java-version: '25'` and `cache: maven`.
  - *Reference:* Editora uses Temurin in `ci.yml` and `full-tests.yml`, and Liberica for the release build, for linux-aarch64 coverage.
- **REL-26** `MUST` Every job has `timeout-minutes`, and `concurrency` cancels a superseded pull-request run, never a run on `master`.
  - *Check (code):* every job sets `timeout-minutes`; `cancel-in-progress` is true only for `pull_request` events.
  - *Reference:* Editora `ci.yml` (`cancel-in-progress: ${{ github.event_name == 'pull_request' }}`).
- **REL-27** `MUST` CI uploads the surefire reports even when tests fail, and the JaCoCo report; it writes the coverage totals to the job summary.

- *Check (code):* the test reports upload under `if: always()`, and a step appends the totals to `$GITHUB_STEP_SUMMARY`.
- *Reference:* Editora's "Upload test reports and coverage data" and "Summarise coverage" steps in `ci.yml`.

- **REL-28** `MUST` Signing skips cleanly when the secrets are absent (forks, dry runs), so they still build; a tag without them fails.
  - *Check (code):* each signing step checks its secrets first, skips without them, and fails on a tag push.
  - *Reference:* Editora's "Import signing certificate (macOS)" step checks `MACOS_CERT_P12` first.
- **REL-29** `MUST` The packaged app is launched once on each OS before a release (a smoke test).
  - *Check (code):* `release.yml` runs the packaged launcher on every target before JReleaser.
  - *Reference:* Torrenta's "Smoke-test the packaged app" step runs the launcher in every target's image. Editora runs only the notarized macOS app (`--version`, `--check-terminal`).

Feature matrix

Surveyed on 2026-11-05 from each repository's default branch (the rows added in editions 1.2.0, 1.3.0, 1.4.0 and 2.0.0 on 2026-11-06). ● has it, ◐ partly or differently, — not found.

What the apps are.

App	What it is	Homepage	Source
Editora	Programmer's text editor (the reference)	editora-project.dev	adriandeleon/Editora
Torrenta	BitTorrent client	torrenta-project.dev	adriandeleon/torrenta
Termina	Terminal emulator	termina-project.dev	adriandeleon/Termina
Modula	Software-defined-radio receiver	—	adriandeleon/Modula
Mirada	Fast image viewer (GraalVM native image)	mirada-project.dev	adriandeleon/Mirada
nux	UPS monitor for Network UPS Tools, in the tray	—	adriandeleon/nux

Platform and repository

	Editora	Torrenta	Termina	Modula	Mirada	nux
JDK 25+ and JavaFX 27+, JPMS module, `mvnw`	●	●	●	●	●	●
AtlantaFX 3	●	●	◐ vendored CSS	● Primer	● Cupertino	● Primer
Inter UI font `SHOULD`	●	●	●	○ Plex Mono	○	○
NullAway + every package `@NullMarked`	●	●	●	●	●	○
Spotless (Palantir)	●	●	●	●	●	○
JaCoCo gate	● 0.90 L/M	● 0.90 L	◐ per-package	○	○	○
Headless FX tests	●	●	●	○	●	●
Six or more message catalogs	●	●	●	○	○	●
README badges + screenshot	●	●	●	●	◐ no shot	●
Keep a Changelog format	●	◐	◐	●	◐	●
MIT `LICENSE` + `NOTICE`	●	●	●	●	●	●
`AGENTS.md` canonical, `CLAUDE.md` a pointer	●	◐ `CLAUDE.md` only	●	◐ `CLAUDE.md` only	●	◐ copy
`CONTRIBUTING.md`	●	○	○	○	○	○
`docs/` and `branding/`	●	●	●	●	●	●
`docs/README.md` indexing the guides	●	○	○	○	○	○
User docs on the website (`/docs/`)	● 43 pages, versioned	● 19 pages	● 16 pages	○ README only	● 11 pages	○ README only
Help → Documentation command	●	○	○	○	○	○
`bug_report.yml` issue form	●	●	○	○	○	○
Dependabot	●	●	○	●	○	●

Release and signing

	Editora	Torrenta	Termina	Modula	Mirada	nux
JReleaser	●	●	●	○	○	○
Five-target matrix (Linux x64/arm64, macOS arm64/x64, Windows x64)	●	◐	●	◐ no Linux arm64	●	●

	Editora	Torrenta	Termina	Modula	Mirada	nux
macOS signed + notarized `.dmg`	●	●	●	●	●	●
Linux OpenPGP `.asc` signatures	●	●	●	●	◐ rpm + checksums	○
Windows `.msi` Authenticode	○	○	○	○	○	○
Build provenance attestations	●	○	●	○	○	○
Homebrew cask / winget manifests	○	●	●	○	◐ winget	○
`.desktop` file + AppStream metainfo	●	●	●	●	◐ no metainfo	◐ no metainfo
AOT start-up training	●	○	●	○	●	○

User-facing features

	Editora	Torrenta	Termina	Modula
Command registry + Command Palette (in-scene overlay)	●	●	●	●
QuickOpen-style pickers for lists	●	○ palette only	○ palette only	○ palette only
Find bar: in-scene, incremental, count, Esc returns focus	● find and replace	◐ filter box	● floating card	○
Keymap presets	● 9	○	○	○
Keybinding editor / shortcuts reference	●	●	●	○
An icon on every menu row	◐ about half the menu bar	◐ not the RSS menus	◐ right-click menus only	○
Chord beside each menu row, right-click menus included	◐ menu bar only	◐ menu bar and torrent menu	◐ menu bar only	● right-click menu
Menu font pinned (Inter 14 px on `.context-menu`)	●	○	◐ root font only	◐ size only
Right-click menu: one at a time, any outside press closes it	● every window	◐ stock auto-hide	◐ terminal view	◐ radio pane
Icon-only buttons carry an accessible name	● `Icons.name`	●	○ tooltip only	○ tooltip only
Toolbar tooltips show the command's chord	●	●	○ no toolbar	○
Tab strip: unsaved and pinned states, drag to reorder, reopen closed	●	○	◐ reorder and reopen, no pin	○
Panels toggled by commands, placement remembered	● stripes, dock, float	● sidebar and detail pane	◐ panes in a tab	○
Settings window	●	●	●	●
Settings search	●	●	●	○
Settings sync (Git / GitHub via `gh`)	●	●	●	○
Message Log (status-bar history)	●	●	○	○
Debug Log + uncaught exception handler	●	●	●	○
Update notification (GitHub releases)	●	●	●	●

	Editora	Torrenta	Termina	Modula
Report a Problem → GitHub issue (+ `--report`)	●	●	○	○
About window with runtime versions	●	●	●	●
About names the Pauta edition it implements	○	○	○	○
Pauta window and `pauta.json` exceptions	○	○	○	○
Welcome page	●	●	○	○
Confirm dialog: named red action button, Escape cancels, or undo instead	●	● `RemoveConfirm` + undo	◐ asks, button not red	○ presets remove on one click ◐
Doctor / diagnosis	●	◐ per torrent	○	◐ native libs
Single instance (hand off to running app)	●	●	○	○
`--config-dir` and other CLI flags	●	●	●	○
Light / dark / follow system, live switch	◐ live, no system choice	◐ live, no system choice	◐ live, no system choice	◐ live day/night, no system choice
Caret & Ink theme pair (Editora Light / Dark)	● authored	● vendored	● vendored	○
Icons as themed SVG paths, one style class per family	● two families	● two families	● filled, menus	◐ transport glyphs
State colors from theme roles, contrast-tested	● `StyleContrastTest`	◐ roles, no test	◐ roles; terminal schemes tested	○ fixed hexes
Extended title bar (`HeaderBar`, `StageStyle.EXTENDED`)	●	◐ opt-in	●	○
Desktop notifications	●	●	●	●
Tray icon (SNI over D-Bus, AWT fallback)	○	●	○	●
Close to tray, only while an icon is up	○	●	○	●
Start at login (XDG, LaunchAgent, `HKCU` Run) and `--minimized`	○	●	○	○
Window geometry restore	●	●	●	○
Config export / import	●	○	○	○
Plugins	●	○	○	○

Shared features

Beyond the list this guide started from, these already exist in two or more apps and belong in the guideline:

1. **`AppInfo` + `build-info.properties`.** Version, build time and commit come from a Maven-filtered resource, read by About, `--version`, the update check and problem reports. *(all six)*
2. **Shared `update/` package** (`UpdateCheck` pure parser, `UpdateService`, `ReleaseInfo`). *(5 apps)*
3. **Pure palette core shared**: `FuzzyMatch`, `MatchText`, `KeyChord`, `PaletteKeys` and the caret-key helper (`TextInputKeymap` in Editora, `TextInputKeys` in the copies). Emacs-style caret keys work inside the palette's query field. *(Editora → Termina → Mirada; Torrenta takes `MatchText` only)*
4. **Shortcuts reference window**: a searchable list of every key, opened from Help. *(Termina, Mirada)*
5. **Extended title bar** with JavaFX 27 `HeaderBar` and `StageStyle.EXTENDED`, falling back to a decorated window; the macOS system menu bar via `useSystemMenuBar`. *(Editora, Termina, Mirada, Torrenta opt-in)*
6. **macOS app-menu integration**: About and Settings in the application menu, `openFiles` Apple Events reaching the running app (`MacOpenFiles` / `MacHandlers`). *(Editora, Mirada, Torrenta)*
7. **Tray package** (`AwtTray`, `SniTray`, `StatusNotifierItem` over dbus-java) plus **autostart** (XDG autostart, LaunchAgent, `HKCU\…\Run`) and `--minimized`. *(Torrenta, Modula, nux)*
8. **Desktop notifications** through one `Notifier`: `notify-send` / `gdbus` on Linux, `osascript` on macOS, a PowerShell toast on Windows, the tray as fallback. *(Torrenta, Termina, Modula, nux, Editora)*
9. **Window geometry that stays on screen**: `WindowGeometry` / `WindowPlacement` restore size and position but move a window back when its monitor is gone. *(Editora, Torrenta, Termina, Mirada)*
10. **Linux desktop integration**: a `.desktop` file whose `StartupWMClass` matches the JavaFX app class (`com.<app>.App`), AppStream metainfo `io.github.adriandeleon.<App>`, multi-size PNG icons.
11. **Package-manager channels**: Homebrew cask and winget manifests generated by JReleaser. *(Torrenta, Termina, Mirada)*
12. **Atomic config writes** with a file lock, and values clamped to a valid range when read. *(Termina, nux, Mirada)*
13. **Dev capture / screenshot mode** (`DevCapture`, nux `--screenshot=`): renders the real UI headless at 2× for README, website and bug-report images. *(Torrenta, Termina, nux; Editora uses an external headless harness)*
14. **Smoke launch in CI**: the packaged app is started and checked (still running after 20 s, or `--version`) on every OS before release. *(all six)*
15. **Release dry run**: `workflow_dispatch` builds and signs with throwaway credentials and publishes nothing. *(Editora, Modula, Mirada, nux, Termina)*
16. `SECURITY.md` with how to report a vulnerability privately. *(Modula; worth adding everywhere)*
17. **Simple mode / density**: fewer menus and a compact layout for new users. *(Editora, Torrenta)*
18. **Zoom levels** stored per window kind, `Ctrl/Cmd +/-/0`. *(Editora, Termina)*
19. **Undo for destructive list actions** (remove a torrent, close a tab) and **reopen closed tab**. *(Torrenta `UndoStack`, Termina and Editora `ClosedTabs`)*

Gaps per app

App	Missing MUSTs
Editora	Windows MSI signing; the Pauta edition and conformance in About, `pauta.json` and the Pauta window; a follow-the-system theme; AI keys only in the keychain, written off the FX thread; plugin signature checking off the Settings UI; no fixed hexes in component rules; `@Tag("fx")` on every FX test; no waiting on the common pool (`DapManager`); accessible names on the main toolbar's buttons, and those buttons out of the Tab order; Shift+Enter in the editor's find bar; a previous-tab command; a palette command for every setting; an echo message for an uncaught error; an icon on every menu row, and chords in right-click menus and drop-downs.
Torrenta	Help → Documentation; `docs/README.md` ; Keep-a-Changelog format; `AGENTS.md` (only `CLAUDE.md` is committed); `CONTRIBUTING.md` ; Windows signing; attestations; the Pauta edition and conformance in About, `pauta.json` and the Pauta window; a follow-the-system theme; a config file that can't be read kept as a backup, never stopping startup; proxy, Torznab and remote-profile secrets in the keychain; an echo message for an uncaught error; an icon on every menu row (the RSS, priority and Set Category menus), chords in right-click menus, and the menu font pinned.
Termina	Help → Documentation; `docs/README.md` ; Message Log; Report a Problem; single instance; `bug_report.yml` ; `CONTRIBUTING.md` ; full coverage gate; Windows signing; the Pauta edition and conformance in About, `pauta.json` and the Pauta window; a follow-the-system theme; an echo message for an uncaught error; icons on the menu bar's rows, chords in right-click menus, and the menu font pinned.
Modula	User docs on a website; Help → Documentation; `docs/README.md` ; JaCoCo; headless FX tests; i18n; Settings search and sync; Message/Debug Log; Report a Problem; `AGENTS.md` ; `CONTRIBUTING.md` ; `bug_report.yml` ; JReleaser; Linux arm64; the Pauta edition and conformance in About, `pauta.json` and the Pauta window; a follow-the-system theme; a config file that can't be read kept as a backup, with a test; icons on its menu rows, and the menu font pinned.
Mirada	Help → Documentation; `docs/README.md` ; JaCoCo; i18n; Settings search and sync; Message/Debug Log; uncaught handler; update check; Report a Problem; `CONTRIBUTING.md` ; `bug_report.yml` ; JReleaser; the Pauta edition and conformance in About, `pauta.json` and the Pauta window; a config file that can't be read kept as a backup, with a test; the menu font pinned to Inter 14 px.
nux	User docs on a website; Help → Documentation; `docs/README.md` ; NullAway; Spotless; JaCoCo; Settings search and sync; Message/Debug Log; Report a Problem; `CONTRIBUTING.md` ; `bug_report.yml` ; Linux signatures; JReleaser; the Pauta edition and conformance in About, `pauta.json` and the Pauta window; a config file that can't be read kept as a backup, never stopping startup, and every value clamped on load; icons on its menu buttons' rows, and the menu font pinned.